6CCSAACV
Advanced Computer Vision

Revision Notes

Academic Year 2025/26 — Semester 2

Compiled on May 1, 2026

These notes are structured to help you prepare for exams.
Each week includes: learning objectives, core concepts, theory, and worked examples.

Contents

Module Overview **6**

1 Week 1: Introduction **8**
1.1 Topic Summary . . . 8
1.2 Core Concepts . . . 8
1.2.1 What is Computer Vision? . . . 8
1.2.2 Why Study Computer Vision? . . . 9
1.2.3 Why is Computer Vision Difficult? . . . 10
1.2.4 How Do We Tackle the Problem of Vision? . . . 12
1.3 Worked Examples . . . 13
1.4 Summary . . . 14

2 Week 2: Image Formation **16**
2.1 Topic Summary . . . 16
2.2 Core Concepts . . . 16
2.2.1 Physics of Image Formation . . . 16
2.2.2 Optics: Pinhole and Lens Cameras . . . 18
2.2.3 Geometry of Image Formation . . . 18
2.2.4 Intrinsic and Extrinsic Parameters . . . 19
2.2.5 Digital Images . . . 20
2.2.6 Camera Sensors and Demosaicing . . . 21
2.2.7 The Eye . . . 22
2.3 Worked Examples . . . 24
2.4 Summary . . . 25

3 Week 3: Low-Level Vision (Artificial) **26**
3.1 Topic Summary . . . 26
3.2 Core Concepts . . . 26
3.2.1 Convolution . . . 27
3.2.2 Smoothing . . . 28
3.2.3 Differencing . . . 29
3.2.4 Edge Detection . . . 30
3.2.5 Multi-Scale Feature Detection . . . 32
3.3 Worked Examples . . . 33
3.4 Summary . . . 36

4 Week 4: Low- and Mid-Level Biological Vision **38**
4.1 Topic Summary . . . 38
4.2 Core Concepts . . . 39
4.2.1 The Biological Visual System . . . 39
4.2.2 Primary Visual Cortex (V1) . . . 40

4.2.3 Gabor Functions . . . 41
4.2.4 Non-Classical Receptive Fields . . . 43
4.2.5 Introduction to Mid-Level Vision . . . 44
4.2.6 Gestalt Laws (Bottom-up Grouping Principles) . . . 45
4.2.7 Border Ownership . . . 45
4.3 Worked Examples . . . 46
4.4 Summary . . . 48

5 Week 5: Mid-Level Vision — Segmentation 50
5.1 Topic Summary . . . 50
5.2 Core Concepts . . . 50
5.2.1 Feature Space and Similarity . . . 51
5.2.2 Thresholding . . . 52
5.2.3 Region-Based Segmentation . . . 53
5.2.4 Clustering Methods . . . 54
5.2.5 Fitting Methods . . . 56
5.3 Worked Examples . . . 58
5.4 Summary . . . 60

6 Week 6: Feature Matching 62
6.1 Topic Summary . . . 62
6.2 Core Concepts . . . 62
6.2.1 The Correspondence Problem . . . 63
6.2.2 Correlation-Based Methods . . . 63
6.2.3 Feature-Based Methods: Detector + Descriptor . . . 65
6.2.4 Corner Detection: Harris . . . 66
6.2.5 Scale-Invariant Detectors: Harris-Laplacian and SIFT . . . 67
6.2.6 Feature Descriptors . . . 68
6.2.7 The Matching Problem: Outliers and RANSAC . . . 69
6.2.8 Putting It All Together . . . 70
6.3 Worked Examples . . . 70
6.4 Summary . . . 74

7 Week 7: Stereo Vision and Depth 76
7.1 Topic Summary . . . 76
7.2 Core Concepts . . . 77
7.2.1 Why Stereo? Image Formation Recap . . . 77
7.2.2 Coplanar Cameras (Simplest Case) . . . 77
7.2.3 The Correspondence Problem . . . 78
7.2.4 Field of View (FoV) . . . 80
7.2.5 Non-Coplanar (Verging) Cameras . . . 80
7.2.6 Epipolar Geometry . . . 81

7.2.7 Cues to Depth . . . 82
7.3 Worked Examples . . . 83
7.4 Summary . . . 86

8 Week 8: Motion and Tracking 88
8.1 Topic Summary . . . 88
8.2 Core Concepts . . . 88
8.2.1 Video, Motion Field, and Optic Flow . . . 89
8.2.2 Brightness Constancy and the Optic Flow Constraint . . . 89
8.2.3 The Aperture Problem . . . 90
8.2.4 The Video Correspondence Problem . . . 90
8.2.5 Estimating Optic Flow . . . 91
8.2.6 Applications of Optic Flow . . . 92
8.2.7 Motion Segmentation: Image Differencing and Background Subtraction . . . 93
8.2.8 Tracking . . . 93
8.2.9 Trackers: Kalman, Particle, Mean-Shift, KLT . . . 94
8.3 Worked Examples . . . 95
8.4 Summary . . . 98

9 Week 9: Artificial Object Recognition (High-Level Vision) 100
9.1 Topic Summary . . . 100
9.2 Core Concepts . . . 101
9.2.1 Recognition tasks . . . 101
9.2.2 Recognition requirements and generic procedure . . . 102
9.2.3 Template matching . . . 103
9.2.4 Sliding window . . . 104
9.2.5 Edge matching . . . 104
9.2.6 Model-based recognition . . . 104
9.2.7 Intensity histograms . . . 105
9.2.8 Implicit Shape Model (ISM) . . . 105
9.2.9 SIFT feature matching . . . 106
9.2.10 Bag-of-Words (BoW) . . . 107
9.2.11 Geometric invariants . . . 107
9.2.12 Classification of methods . . . 108
9.3 Worked Examples . . . 109
9.4 Summary . . . 112

10 Week 10: Biological Object Recognition (High-Level Vision) 115
10.1 Topic Summary . . . 115
10.2 Core Concepts . . . 116
10.2.1 Object-Based vs Image-Based Theories . . . 116
10.2.2 Configural vs Featural Theories . . . 117

10.2.3 Categorisation: Rules, Prototypes, Exemplars 117
10.2.4 Cortical Visual System: Pathways 119
10.2.5 Feed-Forward Hierarchical Models: HMAX and CNN 120
10.2.6 Recurrent Models and Top-Down Processing 121
10.2.7 Bayesian Inference 121
10.2.8 Big Picture: How Cortex Combines These Ideas 122
10.3 Worked Examples 123
10.4 Summary 127

Module Overview

About This Module

This module provides students with a comprehensive understanding of the fundamental concepts and techniques involved in **interpreting and extracting information from visual data**.

Module code: 6CCSAACV **Semester:** 2 (AY 2025/26)

Module Lecturer: Dr. Luis Carlos Garcia Peraza Herrera

Module Aims

The module aims to:

- Provide a comprehensive understanding of fundamental concepts and techniques in computer vision.
- Develop skills in interpreting and extracting information from visual data.
- Cover both traditional and state-of-the-art machine learning techniques for vision tasks.

Learning Outcomes

By the end of this module, you will be able to:

Code	Learning Outcome
ACV1	Identify and solve complex computer vision tasks, selecting and implementing the most suitable algorithms and methodologies.
ACV2	Design and develop innovative methods and algorithms for addressing various computer vision challenges, critically evaluating their strengths and limitations.
ACV3	Analyse, assess, and effectively deploy a range of traditional and state-of-the-art machine learning techniques to implement robust solutions to diverse computer vision tasks.

Recommended Reading

There is no required reading for this module. However, the following books are recommended:

1. *Computer Vision: Algorithms and Applications* (2nd ed). Richard Szeliski. Springer, 2022.
2. *Seeing: The Computational Approach to Biological Vision* (2nd ed). John P. Frisby and James V. Stone. MIT Press, 2010.
3. *Computer Vision: Models, Learning, and Inference.* Simon J. D. Prince. Cambridge University Press, 2012.
4. *Deep Learning: Foundations and Concepts.* Christopher Bishop. Springer, 2023.

Assessment

[TODO] Add assessment details when provided (coursework + exam breakdown).

Weekly Topics

[TODO] Add weekly topic overview once lecture slides are provided.

1 Week 1: Introduction

1.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Describe the goals and challenges of the Computer Vision field.
- Recognise real-world scenarios where Computer Vision is used.
- Identify and list various Computer Vision applications.

Big Picture. This week introduces the field of Computer Vision: what it is, why it matters, why it's hard, and how we approach solving vision problems. Vision is our primary sense for experiencing the world, yet teaching machines to "see" remains one of the great challenges in AI. The key insight is that vision is an ***ill-posed inverse problem***—many possible 3D scenes can produce the same 2D image. Solving this requires ***priors*** (constraints/expectations) that make some interpretations more likely than others.

What Examiners Like to Ask:

- Define computer vision and distinguish it from related fields (image processing, computer graphics).
- Explain why vision is an "ill-posed inverse problem."
- Give examples of priors and how they constrain interpretations.
- List and categorise CV applications by type of information extracted.
- Explain why deep learning is not always the best solution.

1.2 Core Concepts

Roadmap:

1. What is Computer Vision? (definitions, related fields)
2. Why study vision? (importance, applications)
3. Why is vision difficult? (ill-posed, exponentially large)
4. How do we tackle the problem? (priors, engineering approaches)

1.2.1 What is Computer Vision?

Computer Vision

Computer Vision is a field of study that focuses on creating methods capable of extracting information about the world from images and videos.

Classic definitions:

- "Computing properties of the 3D world from one or more digital images" (Trucco & Verri)
- "Making useful decisions about real physical objects and scenes based on sensed images" (Stockman & Shapiro)
- "Extracting descriptions of the world from pictures or sequences of pictures" (Forsyth & Ponce)

Alternative names: machine vision, image analysis, image understanding, computational vision.

Related Disciplines and Their Transformations:

Field	Input	→	Output
Image Processing	Image	→	Image
Computer Vision	Image	→	Description
Computer Graphics	Description	→	Image

Exam Tip: Field Distinctions

A common question asks you to label arrows in a diagram showing transitions between "Real World," "Image," and "Description." Remember:

- Real World → Image: **Imaging/Photography**
- Image → Description: **Computer Vision**
- Description → Image: **Computer Graphics**
- Image → Image: **Image Processing**

Other related fields:

- **Pattern Recognition**: recognising and classifying stimuli in images and other datasets.
- **Photogrammetry**: obtaining measurements from images.
- **Biological Vision**: understanding visual perception in humans/animals (neuroscience, psychology, psychophysics).

1.2.2 Why Study Computer Vision?

Motivation:

- Vision is the main way we experience the world.
- Want machines to interact with and understand the world (robotics, AI).
- Digital images are everywhere—lots of information to extract.
- Many practical applications with real-world impact.

Applications by Information Extracted:

Application	Information Extracted
OCR / Character Recognition	Category of each letter/digit
Face Tracking (webcam)	Location of face in image
Face Recognition / Biometrics	Identity of individual
Content-Based Image Retrieval	Category of items, relationships between items
Driver Assistance	Categorisation (pedestrians, road edge) + 3D location
3D Modelling	3D locations of all image points
Object Detection	Location + class of objects
Object Segmentation	Pixel-level object boundaries
Action Recognition	Temporal classification of actions
Visual SLAM	Camera pose + 3D map of environment

Application Categories:

- **Recognition**: OCR, face recognition, iris/fingerprint biometrics, landmark recognition
- **Detection & Tracking**: face detection, people tracking, object tracking (e.g., Hawk-Eye in sports)
- **Retrieval**: content-based image retrieval, reverse image search
- **3D Reconstruction**: 3D models from images, structure from motion
- **Autonomous Systems**: driver assistance, self-driving cars, space exploration (Mars rovers)
- **Medical**: analysis of MRI, CT, ultrasound; surgical robotics
- **Entertainment**: AR/VR, TikTok filters, eye contact rectification
- **Industrial**: quality control inspection, barcode/QR scanning

---

1.2.3 Why is Computer Vision Difficult?

Two Major Challenges

1. **One image → many interpretations**: the problem is ***ill-posed***.
2. **One object → many images**: the problem is ***exponentially large***.

Challenge 1: Vision is an Ill-Posed Inverse Problem

Forward vs Inverse Problems

- **Forward problem**: know causes, predict outcomes (well-posed, unique solution).
- **Inverse problem**: know outcomes, infer causes (often ill-posed, multiple solutions).

- **Imaging** (forward): 3D world → 2D image is **unique** (well-posed).
- **Vision** (inverse): 2D image → 3D world is **NOT unique** (ill-posed).

For any given image, many different 3D scenes could have generated it. Humans typically perceive only one interpretation because our brains apply ***priors*** that make some interpretations more likely.

Classic Ambiguous Figures

- **Necker Cube**: two equally valid 3D interpretations; perception flips spontaneously.
- **Rubin's Vase**: face or vase? Two interpretations, never perceived simultaneously.
- **Convex/Concave Craters**: prior expectation of light from above determines interpretation.

Challenge 2: Vision Scales Exponentially

Consider recognising an object that can appear at:

- l locations in the image
- s different scales (sizes)
- o orientations
- c colours

This single object can generate $l \times s \times o \times c$ different images. The number of possible images grows **exponentially** with the number of parameters.

Factors Affecting Appearance:

Factor	Effect
Viewpoint	Same object from different angles looks very different
Illumination	Different lighting changes appearance drastically
Non-rigid deformation	Object can bend, stretch (e.g., faces, animals)
Within-category variation	Objects in same category look different (e.g., different chairs)
Background clutter	Multiple objects in scene
Occlusion	Objects partially hidden by other objects

Exam Tip: Discrimination vs Recognition

"Objects that look very similar can be represented and recognized as different objects, whereas objects that look very different can be recognized as the same basic-level objects" (Bar, 2004). This captures the dual challenge: distinguishing similar things while grouping dissimilar things.

Why Current Systems Are Limited:

- Most CV systems work in **specific domains**: specific task + specific environment.
- Solutions are **not robust** and **do not generalise** well.
- Code that works for one task may fail for similar tasks or same task under different conditions.
- The challenge: develop **general-purpose vision systems** that match human performance.

Challenging Examples

- **Chihuahua or blueberry muffin?** Texture similarity fools classifiers.
- **Sheepdog or mop?** Shape/texture confusion.
- **How many cows?** Segmentation fails with unusual poses/occlusions.

These examples show that even state-of-the-art deep learning struggles with adversarial or unusual cases.

1.2.4 How Do We Tackle the Problem of Vision?

Prior (Constraint)

A ***prior*** is additional information that helps reduce the number of possible solutions, making some interpretations more likely than others. Priors are essential for solving the ill-posed inverse problem of vision.

Sources of Priors:

1. **Prior knowledge / familiarity**:
   - Learned familiarity with certain objects
   - Knowledge of image formation process (e.g., perspective, shadows)
2. **Prior exposure / motion / priming**:
   - Recent or preceding sensory input
   - Change blindness: disrupting temporal continuity hides changes
3. **Current context**:
   - Surrounding visual scene
   - Concurrent input from other senses

Effects of Priors on Human Perception:

Prior Type	Example Effect
Illumination	Checker shadow illusion: squares A and B appear different but are same intensity (shadow prior)
Light direction	Craters appear convex/concave depending on assumed light source (from above)
Perspective	Ames room: people appear to change size due to perspective expectations
Prior knowledge	Dalmatian dog: invisible until you know what to look for
Context	"THE CAT"—middle letter is H in first word, A in second (same shape)
Face detection	We see faces everywhere (Virgin Mary toast sold for $28,000!)

Illusions Reveal Assumptions

Illusions are not "mistakes" by the visual system. They reveal the **assumptions/priors** the visual system uses to solve the under-constrained problem of vision. These assumptions work well in typical situations because they reflect how the world usually is.

Two Engineering Approaches:

Approach	Description
Forward Engineering	Determine requirements → design system → implement and refine. "Top-down": start with computational theory, fill in details.
Reverse Engineering	Find a system that works (e.g., brain) → analyse how it works → implement using same mechanisms. "Bottom-up": start with mechanisms, build model.

This course considers **both** approaches:

- **Computational (machine) vision**: How can we get computers to see?
- **Biological (human) vision**: How do people see?

Exam Tip: Deep Learning Is Not Always the Answer

The lecture emphasises that traditional/classical CV methods sometimes outperform deep learning. Example: for endoscope content area segmentation, classical methods achieved better accuracy than DL approaches. Always consider the problem domain and data availability.

1.3 Worked Examples

Example 1: Defining Computer Vision

Q: Give a definition of "Computer Vision."

A: Computer Vision is a field of study that focuses on creating methods capable of extracting information about the world from images and videos.

Example 2: Field Transitions Diagram

Q: Label each arrow with the field that deals with that transition:

$$\text{Real World} \leftrightarrow \text{Image} \leftrightarrow \text{Description}$$

A:

- Real World → Image: **Imaging/Photography**
- Image → Real World: (Physical interaction, not a field)
- Image → Description: **Computer Vision**
- Description → Image: **Computer Graphics**
- Image → Image: **Image Processing**

Example 3: Information Extracted by Application

Q: What type of information is extracted in each application?

A:

- **OCR**: category of each letter
- **Face tracking webcam**: location of face in image
- **Face recognition/biometrics**: identity of individual
- **Content-based image retrieval**: category of items, relationships between items
- **Driver assistance**: categorisation (pedestrians, road edge) + 3D location

- **3D modelling**: 3D locations of all image points

Example 4: Ill-Posed Inverse Problem

Q: Vision is often described as an "ill-posed, inverse problem." Explain these terms and their opposites.

A:

- **Forward problem**: Know the causes, predict the outcomes (e.g., know forces, calculate acceleration).
- **Inverse problem**: Know the outcomes, infer the causes (e.g., know pixel intensities, infer 3D scene).
- **Well-posed problem**: Has one unique solution.
- **Ill-posed problem**: Has multiple solutions (or no solution).

Vision is an **inverse problem** because we know pixel intensities (outcomes) and want to infer the 3D scene (causes). It is **ill-posed** because multiple 3D scenes can produce the same 2D image.

Example 5: Priors in Vision

Q: What is a "prior" and how does it help solve the ill-posed inverse problem of vision? Give examples.

A: A prior is additional information that reduces the number of possible solutions, making some interpretations more likely. Examples include:

- **Knowledge of the world**: objects are usually solid, light comes from above
- **Image formation knowledge**: perspective geometry, how shadows work
- **Prior exposure**: recently seen objects are more easily recognised
- **Context**: surrounding scene constrains possible object identities

---

1.4 Summary

Key Takeaways

1. **Computer Vision** extracts descriptions/information from images (Image → Description).
2. Vision is **difficult** because it is:
   - **Ill-posed**: one image can have many 3D interpretations
   - **Exponentially large**: one object can generate many images
3. **Priors/constraints** are essential for narrowing down interpretations.
4. Human vision uses priors extensively—illusions reveal these assumptions.
5. We tackle CV using both **forward engineering** (design systems) and **reverse engineering** (study biological vision).
6. Deep learning is powerful but **not always the best solution**—classical methods can outperform in specific domains.

Concept	**Definition**	**Implication**
Inverse problem	Infer causes from outcomes	Multiple solutions possible
Ill-posed	No unique solution	Need priors to constrain
Prior	Additional constraining info	Makes some interpretations likely
Forward engineering	Top-down design	Computational theory first
Reverse engineering	Bottom-up modelling	Study biological systems

2 Week 2: Image Formation

2.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Explain the end-to-end process of digital image capture.
- Analyse the biological mechanisms of visual perception in the human eye.
- Compare the different strategies used by digital cameras and the human retina to efficiently encode spatial and colour information.

Big Picture. This week covers how images are formed—both in cameras and in the eye. We examine the ***physics*** (light, reflectance, optics), ***geometry*** (projection, camera models), ***digitisation*** (sampling, quantization), and ***biological vision*** (photoreceptors, ganglion cells). The key insight is that recovering surface properties from images is ill-posed because $L = f(E, R)$—we observe luminance but need to infer reflectance without knowing illumination.

What Examiners Like to Ask:

- Explain the relationship $L(\lambda) = f(E(\lambda), R(\lambda))$ and why colour constancy is hard.
- Derive the perspective projection equations and explain intrinsic/extrinsic parameters.
- Compare pinhole cameras vs lensed cameras (trade-offs).
- Explain demosaicing and different interpolation methods.
- Describe centre-surround receptive fields and their functional benefits.
- Compare foveal vs peripheral vision.

2.2 Core Concepts

Roadmap:

1. Physics of image formation (light, reflectance, optics)
2. Geometry of image formation (camera models, projective geometry)
3. Digital images (digitisation, representation)
4. Camera sensors (CCD, demosaicing)
5. The eye (photoreceptors, ganglion cells)

2.2.1 Physics of Image Formation

Overview of Image Formation:

Images are formed when a **sensor** registers **radiation** that has interacted with **physical objects**. Light from a source is reflected by surfaces, passes through optics, and is recorded by a sensor.

$$\text{Light Source} \rightarrow \text{Surface (Reflectance)} \rightarrow \text{Optics} \rightarrow \text{Sensor}$$

Two Sets of Parameters:

Parameter Type	What It Determines
Radiometric	Intensity/colour at each image location (illumination, surface reflectance, sensor sensitivity)
Geometric	Where scene points appear in image (camera position/orientation, optics, projection geometry)

Light and Colour:

Visible Light

Electromagnetic radiation with wavelength λ between 400–700nm. The sun's intensity peaks in this range, and human eyes evolved sensitivity to it.

- **Illuminance** $E(x, y, \lambda)$: amount of light striking a surface (varies by light source)
- **Reflectance** $R(x, y, \lambda)$: fraction of light reflected at each wavelength (***albedo***)
- **Luminance** $L(x, y, \lambda)$: amount of light reaching the sensor

Fundamental Equation of Image Formation

$$L(x, y, \lambda) = f\big(E(x, y, \lambda), R(x, y, \lambda)\big)$$

The luminance depends on both illuminance and reflectance. This is **ill-posed**: we observe L but need to recover R without knowing E.

Colour Mixing:

Type	Process	Result
Additive (light)	R + G + B lights	White (broad spectrum)
Subtractive (pigment)	R + G + B pigments	Black (absorbs all)

Why Objects Have Colour

An object looks **green** because it absorbs red and blue light, leaving more green in the reflected light. The colour we perceive depends on which wavelengths are *not* absorbed.

Colour Constancy:

Colour Constancy

The ability to perceive surface colour as constant despite changes in illumination. Humans do this well; computers struggle.

Approaches to estimate illumination E:

- Assume average reflectance across scene is known (grey world)
- Fix brightest patch to white
- Gamut falls within known range
- Known reference colour (colour chart, skin)
- Specular reflections have illumination colour

None work reliably in all cases. Human vision discounts illumination effects through learned priors.

2.2.2 Optics: Pinhole and Lens Cameras

Pinhole Camera

A camera that restricts light flow through a small hole so that only one ray from each scene point reaches the sensor. Everything is in focus regardless of depth.

Pinhole Trade-off:

Aperture	Focus	Brightness
Small pinhole	Sharp	Dim (long exposure, blur from motion)
Large pinhole	Blurred	Bright (short exposure)

A **lens** solves this by focusing rays from the same point onto a single image location, keeping the image sharp while gathering more light.

Thin Lens Properties:

- Rays through optical centre O are not refracted
- Rays parallel to optical axis refract through focal point F'
- Rays through F become parallel to optical axis

Thin Lens Equation

$$\frac{1}{f} = \frac{1}{|z|} + \frac{1}{|z'|}$$

where f = focal length, z = object distance, z' = image distance.

Key implications:

- Object at infinity $\Rightarrow$ image at F'
- Object at F or closer $\Rightarrow$ no image formed
- **Depth of focus**: range of object depths that appear acceptably sharp
- Smaller aperture $\Rightarrow$ larger depth of focus (approaches pinhole behaviour)

Exam Tip: Lens vs Pinhole

Know the trade-offs: pinhole has infinite depth of focus but needs long exposure; lens is bright but only objects at specific depths are in focus.

2.2.3 Geometry of Image Formation

Perspective (Pinhole) Camera Model

A mathematical model of geometric projection where a 3D point $P = (x, y, z)$ projects to image point $P' = (x', y', z')$. The image plane Π' is at distance f' from the pinhole along the optical axis.

Equation of Projection:

From similar triangles:

$$\frac{x'}{x} = \frac{f'}{z} \quad \Rightarrow \quad x' = \frac{f'}{z}x$$

$$\frac{y'}{y} = \frac{f'}{z} \quad \Rightarrow \quad y' = \frac{f'}{z} y$$

Since P' lies on the image plane: $z' = f'$.

In matrix form (homogeneous coordinates):

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \frac{f'}{z} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Projective Geometry:

Geometry	Describes	Preserves
Euclidean	Objects "as they are" (3D→3D)	Lengths, angles, parallelism
Projective	Objects "as they appear" (3D→2D)	None of the above

Projective distortions:

- Distant objects appear smaller
- Parallel lines converge at ***vanishing points***
- Equal lengths become unequal
- Right angles become non-right angles

Vanishing Point

The projection of a point at infinity. All parallel lines in 3D share the same vanishing point in the 2D image.

Railway Tracks Example

In the world: tracks parallel, ties equal length, perpendicular, evenly spaced.

In the image: tracks converge at vanishing point, ties get shorter with distance, not perpendicular to tracks, get closer together.

2.2.4 Intrinsic and Extrinsic Parameters

Converting Camera Coordinates to Pixels:

Intrinsic Parameters

Camera-dependent properties that map the image plane [mm] to pixel coordinates [pixel]:

- o_x, o_y: principal point (image centre) in pixels
- $\alpha = -f'/s_x$, $\beta = -f'/s_y$: magnification factors
- s_x, s_y: pixel size [mm/pixel]
- Aspect ratio $= \alpha/\beta$

Pixel coordinates:

$$u = \alpha \frac{x}{z} + o_x, \qquad v = \beta \frac{y}{z} + o_y$$

In matrix form:

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \begin{pmatrix} \alpha & 0 & o_x \\ 0 & \beta & o_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Extrinsic Parameters

Camera location-dependent properties that transform world coordinates to camera coordinates:

- $\mathbf{R}$: 3×3 rotation matrix
- $\mathbf{t}$: 3×1 translation vector

World to camera transformation:

$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{R}^T & -\mathbf{R}^T\mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Complete Projection Pipeline

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{z} \underbrace{\begin{pmatrix} \alpha & 0 & o_x \\ 0 & \beta & o_y \\ 0 & 0 & 1 \end{pmatrix}}_{\text{Intrinsic (2D→2D)}} \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_{\text{Projection (3D→2D)}} \underbrace{\begin{pmatrix} \mathbf{R}^T & -\mathbf{R}^T\mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix}}_{\text{Extrinsic (3D→3D)}} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

Exam Tip: Forward vs Inverse

3D → 2D projection is **well-posed** (unique). 2D → 3D recovery is **ill-posed** (depth lost). Need extra information: stereo vision or scene priors.

2.2.5 Digital Images

Digital Image

A 2D array (matrix) of numbers $I(x, y)$ where each element represents the light intensity at a discrete location called a ***pixel*** (picture element).

Two discretisation processes:

- **Pixelisation**: intensity values averaged within each grid location (spatial sampling)
- **Quantization**: intensity represented using finite discrete values (intensity sampling)

Image Formats:

Format	Values/Pixel	Bits	Range
Binary/Monochrome	1 binary	1	$\{0, 1\}$
Greyscale	1 real	8	$[0, 255]$ or $[0, 1]$
Colour (RGB)	3 reals	24 (8×3)	3 channels × 256 levels

Convention: 0 = black, 255 (or 1.0) = white. Origin at top-left corner.

Format Conversions:

RGB to Greyscale (weighted average):

$$I_{\text{grey}} = \frac{r \cdot I_R + g \cdot I_G + b \cdot I_B}{r + g + b}$$

Greyscale to Binary (thresholding):

$$I_{\text{bin}}(p) = \begin{cases} 1 & \text{if } I_{\text{grey}}(p) > t \\ 0 & \text{otherwise} \end{cases}$$

2.2.6 Camera Sensors and Demosaicing

Charge Coupled Device (CCD)

A semiconductor with a 2D matrix of photo-sensors. Each sensor accumulates charge proportional to incident light intensity and exposure time. Charge pattern corresponds to the image.

Colour from CCDs:

Photo-sensors are not wavelength-selective. Colour is achieved via filters:

Solution	Pros	Cons
3 CCDs + prism	High quality, full sampling	Expensive, bulky
1 CCD + colour mask	Cheap, compact	Coarser sampling, needs demosaicing

---

Bayer Mask

The most common colour filter array. Has 2× as many green filters as red/blue (matches human sensitivity). Results in subsampled colour channels that need interpolation.

Demosaicing

The process of filling in missing colour values so each pixel has R, G, and B values. A form of interpolation.

Interpolation Methods:

Method	Description	Trade-off
Nearest Neighbour	Copy adjacent pixel from same channel	Fast, inaccurate
Bilinear	Average of 2 or 4 nearest same-channel pixels	Fast, blurs edges
Smooth Hue Transition	Interpolate ratio (hue) R/G or B/G	Better colour accuracy
Edge-Directed	Interpolate along axis with lowest gradient	Preserves edges

Edge-Directed Interpolation for Green

To calculate G_8:

1. Compute horizontal gradient: $\Delta_H = |G_7 - G_9|$
2. Compute vertical gradient: $\Delta_V = |G_3 - G_{13}|$
3. If $\Delta_H < \Delta_V$: $G_8 = (G_7 + G_9)/2$ (interpolate horizontally)
4. If $\Delta_H > \Delta_V$: $G_8 = (G_3 + G_{13})/2$ (interpolate vertically)
5. If equal: $G_8 = (G_3 + G_7 + G_9 + G_{13})/4$

2.2.7 The Eye

Eye Anatomy:

- **Cornea**: bulk of refraction (fixed focus)
- **Lens**: further refraction, can change shape to adjust focal length
- **Iris**: regulates light amount (like camera aperture)
- **Retina**: contains photoreceptors
- **Optic nerve**: ∼1 million fibres transmit to brain

Photoreceptor Types:

Type	Sensitivity	Colour	Location
Rods	High (dim light)	None (monochrome)	Periphery
Cones (S)	Low (bright light)	Blue (∼420nm)	Fovea
Cones (M)	Low (bright light)	Green (∼540nm)	Fovea
Cones (L)	Low (bright light)	Red (∼570nm)	Fovea

Distribution:

- **Fovea**: no rods, high cone density. #blue $\ll$ #red $\leq$ #green
- **Periphery**: mostly rods, few cones
- **Blind spot**: no photoreceptors (optic nerve exit)

Foveal vs Peripheral Vision:

Property	Fovea	Periphery
Resolution	High (dense receptors)	Low (sparse receptors)
Colour	Yes (cones)	No (rods)
Sensitivity	Low (cones)	High (rods)
Brain resources	Disproportionately large	Smaller

Retinal Processing—Ganglion Cells:

Receptive Field (RF)

The area of visual space from which a neuron receives input. More generally, the stimulus properties that produce a response.

- ∼1 million ganglion cells vs ∼100 million photoreceptors
- Each ganglion cell collects from multiple photoreceptors
- Large convergence in periphery, smaller in fovea (another reason for lower peripheral acuity)

Centre-Surround Receptive Fields

Ganglion cells have centre-surround RFs:

- **On-centre, off-surround**: active if centre brighter than surround
- **Off-centre, on-surround**: active if centre darker than surround

Functional Benefits of Centre-Surround RFs

1. **Edge enhancement**: weak response on uniform regions, strong near edges
2. **Invariance to ambient light**: measures contrast, not absolute intensity
3. **Efficient coding**: only signals where intensity changes ("redundancy reduction")

Centre-Surround Response

On a uniform surface: input to centre and surround cancels ⇒ weak response.

At an edge: input unequal ⇒ strong response (increased or decreased depending on type).

Under different lighting: if illumination doubles, both centre and surround double, so contrast unchanged ⇒ response unchanged.

Colour Opponent Cells:

Ganglion cells combine cone inputs to create colour-opponent responses:

- **Red-Green opponent**: Red ON / Green OFF (or vice versa)
- **Blue-Yellow opponent**: Blue ON / Yellow OFF (yellow = average of R and G cones)

Difference of Gaussians (DoG)

The standard mathematical model for centre-surround RFs:

$$\mathrm{DoG}(x, y) = G_{\sigma_1}(x, y) - G_{\sigma_2}(x, y)$$

where G_σ is a Gaussian with standard deviation σ, and $\sigma_1 < \sigma_2$.

2.3 Worked Examples

Example 1: Colour Constancy Problem

Q: Why is recovering surface colour from an image an ill-posed problem?

A: The luminance $L(\lambda)$ we observe depends on both illuminance $E(\lambda)$ and reflectance $R(\lambda)$: $L = f(E, R)$. We want to recover R (surface colour) but only measure L and don't know E (light source). Different combinations of E and R can produce the same L, making the problem ill-posed.

Example 2: Thin Lens Calculation

Q: A lens has focal length $f = 50$mm. An object is at distance $z = 200$mm. Where does the image form?

A: Using the thin lens equation:

$$\frac{1}{f} = \frac{1}{|z|} + \frac{1}{|z'|} \quad \Rightarrow \quad \frac{1}{50} = \frac{1}{200} + \frac{1}{|z'|}$$

$$\frac{1}{|z'|} = \frac{1}{50} - \frac{1}{200} = \frac{4-1}{200} = \frac{3}{200}$$

$$|z'| = \frac{200}{3} \approx 66.7\text{mm}$$

Example 3: Perspective Projection

Q: A point $P = (30, 40, 100)$ in camera coordinates. Focal length $f' = 10$mm. Find image coordinates.

A:

$$x' = \frac{f'}{z}x = \frac{10}{100} \times 30 = 3\text{mm}$$

$$y' = \frac{f'}{z}y = \frac{10}{100} \times 40 = 4\text{mm}$$

Image point: $P' = (3, 4, 10)$ mm.

Example 4: Intrinsic Parameters

Q: Convert camera coordinates $(x', y') = (3, 4)$mm to pixel coordinates. Given: $s_x = s_y = 0.01$mm/pixel, $f' = 10$mm, image centre $(o_x, o_y) = (320, 240)$.

A:

$$\alpha = -\frac{f'}{s_x} = -\frac{10}{0.01} = -1000, \quad \beta = -\frac{f'}{s_y} = -1000$$

From projection: $x'/z' = 3/10 = 0.3$, $y'/z' = 4/10 = 0.4$

Actually, using the formula directly with x/z:

$$u = \alpha\frac{x}{z} + o_x = -1000 \times 0.3 + 320 = 20$$

$$v = \beta\frac{y}{z} + o_y = -1000 \times 0.4 + 240 = -160$$

(Negative v indicates point projects above image boundary.)

Example 5: Centre-Surround Benefits

Q: Explain why centre-surround RFs provide invariance to ambient lighting.

A: Centre-surround cells measure **contrast**—the difference between centre and surround intensities. If ambient light doubles, both centre and surround receive twice the light, but their *difference* (relative to their sum) remains constant. The response depends on $I_{\text{centre}} - I_{\text{surround}}$, which encodes local contrast, not absolute intensity.

2.4 Summary

Key Takeaways

1. **Image formation**: $L = f(E, R)$—luminance depends on illumination and reflectance; recovering R is ill-posed.
2. **Optics**: Pinhole = sharp but dim; Lens = bright but limited depth of focus. Thin lens: $1/f = 1/|z| + 1/|z'|$.
3. **Projection**: $x' = (f'/z)x$, $y' = (f'/z)y$. Parallel lines $\rightarrow$ vanishing points.
4. **Camera parameters**: Intrinsic (focal length, pixel size, centre) + Extrinsic (rotation, translation).
5. **Digital images**: Pixelisation (spatial) + Quantization (intensity). Bayer mask needs demosaicing.
6. **Eye**: Fovea (high-res, colour, low-sensitivity) vs Periphery (low-res, mono, high-sensitivity).
7. **Ganglion cells**: Centre-surround RFs provide edge enhancement, lighting invariance, efficient coding.

Camera	**Eye**
Uniform pixel grid	Non-uniform (fovea vs periphery)
Bayer mask for colour	3 cone types for colour
Demosaicing interpolates	Brain processes incomplete data
Raw pixel output	Centre-surround preprocessing
Fixed sensitivity	Rods (dim) vs Cones (bright)

3 Week 3: Low-Level Vision (Artificial)

3.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Define and compute 2D ***convolution*** of an image with a mask, including boundary handling and separability.
- Distinguish convolution from cross-correlation and explain why convolution is preferred.
- Construct and apply ***smoothing*** (box, Gaussian) and ***differencing*** (1st/2nd derivative, Laplacian) masks.
- Build edge detectors that combine smoothing and differencing (LoG, DoG, derivative-of-Gaussian, Canny).
- Explain multi-scale feature detection via Gaussian and Laplacian ***image pyramids***.

Big Picture. Low-level vision is image processing applied as a front-end to higher-level CV tasks. The unifying tool is ***linear filtering***: each output pixel is a weighted sum of input pixels in a neighbourhood, computed by ***convolution*** with a ***mask*** (kernel). Choosing the weights selects what we extract: **smoothing** masks (positive, sum-to-1) average pixels and suppress noise/high spatial frequencies; **differencing** masks (mixed sign, sum-to-0) approximate derivatives and respond to intensity changes (edges). Real edge detection *combines both*: smooth at a chosen ***scale*** σ, then differentiate (LoG/DoG/derivative-of-Gaussian, e.g. Canny). To find features without knowing their size, we run a fixed-size mask over a ***Gaussian image pyramid*** (resampling instead of resizing the mask).

What Examiners Like to Ask:

- Compute the result of $H * I$ by hand (rotate mask, slide, multiply-accumulate; pad with zeros).
- State the convolution formula and contrast with cross-correlation.
- Show a mask is separable: $H = h * h^T = h^T \times h$; explain the efficiency gain.
- Write down 1st- and 2nd-derivative masks for $-\partial/\partial x$, $-\partial/\partial y$, $-\partial^2/\partial x^2$, etc., and the Laplacian.
- List the four causes of intensity discontinuities (depth, orientation, reflectance, illumination).
- Explain why edge detection needs both a smoothing and a differencing operator; why LoG/DoG help.
- Describe the Canny pipeline (smooth $\rightarrow$ derivative $\rightarrow$ magnitude/direction $\rightarrow$ NMS $\rightarrow$ hysteresis).
- Define aliasing and explain why we Gaussian-smooth before downsampling.
- Describe a Gaussian and a Laplacian pyramid and why pyramids are cheaper than scaling the mask.

3.2 Core Concepts

Roadmap:

1. Convolution (definition, method, separability, weights, boundaries).
2. Smoothing (spatial frequency, box, Gaussian).
3. Differencing (1st/2nd derivative masks, Laplacian).
4. Edge detection (causes, idealised edges, LoG/DoG/derivative-of-Gaussian, Canny).
5. Multi-scale feature detection (Gaussian and Laplacian pyramids, aliasing).

3.2.1 Convolution

2D Convolution

For image I and mask (kernel/template) H,

$$I'(i,j) \;=\; (I * H)(i,j) \;=\; \sum_{k,l} I(i+k,\, j+l)\, H(-k,\, -l).$$

The minus signs in $H(-k,-l)$ correspond to a 180° rotation of the mask before sliding.

Method (by hand):

1. **Rotate** the mask H by 180°.
2. For each output pixel: **centre** the rotated mask on the corresponding input pixel.
3. **Multiply** each mask weight by the underlying pixel value.
4. **Sum** the products and write the result into the output pixel.

Two equivalent views:

- Slide the rotated mask across the image, filling outputs as you go (sequential).
- Place a copy of the mask at every pixel and superimpose, weighted by intensity (parallel; matches DoG filters in the retina).

Convolution vs Cross-Correlation:

Operation	Formula	Properties
Convolution $I * H$	$\sum_{k,l} I(i+k, j+l)\, H(-k,-l)$	Commutative, associative
Cross-correlation $I \otimes H$	$\sum_{k,l} I(i+k, j+l)\, H(k,l)$	Neither

- If the mask is **symmetric** ($H(k,l) = H(-k,-l)$), then convolution = cross-correlation.
- Convolution is preferred because it is commutative ($I * H = H * I$) and associative ($(I * H) * G = I * (H * G)$); this lets us combine filters and rearrange pipelines safely.
- In CNNs, the "convolutional" layer is actually cross-correlation: kernels are *learned*, so the rotation is irrelevant and skipping it is faster.

Mask Properties:

Mask as point-spread function. Convolving an isolated white dot on a black background with H yields the rotated mask, shifted to the dot's location. Any image is a sum of such points, so the convolution is the superposition of weighted, rotated masks.

Mask as template. Convolution responds maximally where the image patch matches the rotated mask. Hence "masks are templates": we choose weights that resemble the feature we want to detect.

Two main families of mask weights:

Family	Weights	Effect
Smoothing	all positive, sum to 1	Preserves average intensity, blurs (low-pass)
Differencing	mixed sign, sum to 0	Zero on uniform regions, responds to intensity change

Separability:

Separable Mask

A 2D mask H is separable if it can be written as the convolution of two 1D vectors,

$$H \;=\; h_1 * h_2^T \;=\; h_2^T \times h_1.$$

Then $I * H = (I * h_1) * h_2^T$: a 2D convolution becomes two 1D convolutions.

Why it matters (efficiency): Convolving an $n \times n$ image with an $m \times n$ mask costs:

- $m \cdot n$ multiplications per pixel for the 2D mask.
- $m + n$ multiplications per pixel for the two 1D masks.

Useful identity: the convolution of a row vector with a column vector equals their outer product, $h * h^T = h^T \times h$ (helpful in exam questions).

Boundary Handling. Where the mask falls off the image, the two most common options are:

1. **Zero-pad** the input image (the implicit convention used in lectures): output has same size as input.
2. **Shrink** the output: only compute results where the mask fits entirely inside the input.

Spot it in a question

Phrases like "produce a result the same size as I" $\Rightarrow$ zero-pad. "Only at locations where the mask fits entirely inside" $\Rightarrow$ shrink (output smaller than input).

3.2.2 Smoothing

Spatial frequency. Small-scale texture corresponds to **high** spatial frequency; large smooth regions to **low** spatial frequency. Smoothing acts as a ***low-pass*** filter: it removes high-frequency content (fine detail and noise).

Box (Mean) Filter:

$$H_{\text{box}} \;=\; \frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Each pixel is replaced by the mean of itself and its neighbours. Generates a blurred image. Larger box (9×9, 17×17, ...) blurs more.

Drawback: the box mask has *sharp edges* (constant inside, zero outside), which produces artefacts in the smoothed image.

Gaussian Filter:

2D Gaussian

$$G_\sigma(x, y) \;=\; \frac{1}{2\pi\sigma^2}\,\exp\!\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Falls off gently at the edges; gives more weight to nearby pixels.

Effect of the scale parameter σ: as σ increases:

- Mask width must increase (rule of thumb: $\geq 6\sigma$).
- More pixels contribute to each average.
- Image gets more blurred (more high frequencies suppressed).
- Noise is more effectively suppressed.

Separability of the Gaussian:

$$G_\sigma(x,y) \;=\; \underbrace{\frac{1}{\sqrt{2\pi}\,\sigma}\exp\!\left(-\tfrac{x^2}{2\sigma^2}\right)}_{G_\sigma(x)} \cdot \underbrace{\frac{1}{\sqrt{2\pi}\,\sigma}\exp\!\left(-\tfrac{y^2}{2\sigma^2}\right)}_{G_\sigma(y)}$$

i.e. a 2D Gaussian is the product of two 1D Gaussians $\Rightarrow$ separable $\Rightarrow$ much cheaper.

Useful Gaussian identity (cascade):

$$G_{\sigma_1} * G_{\sigma_2} \;=\; G_{\sqrt{\sigma_1^2+\sigma_2^2}}$$

e.g. $G_\sigma * G_\sigma = G_{\sigma\sqrt{2}}$. Used in pyramid construction.

3.2.3 Differencing

Convolution can also estimate ***derivatives*** (intensity gradients). Smoothing approximates integration; differencing approximates differentiation.

1st derivative (forward difference). Estimate of $\partial I/\partial x$ at a pixel uses neighbours i_1, i_2: gradient $\approx i_2 - i_1$. Multiplying by the row $[-1 \;\; 1]$ would give $-i_1 + i_2$, but *convolution rotates the mask*, so the actual mask used to compute $-\partial/\partial x$ is

$$[\,-1 \;\; 1\,] * I \;\approx\; -\tfrac{\partial I}{\partial x}.$$

2nd derivative (central difference). $(i_3 - i_2) - (i_2 - i_1) = i_1 - 2i_2 + i_3$. The mask $[1 \;\; -2 \;\; 1]$ gives $\partial^2/\partial x^2$; convention uses $[-1 \;\; 2 \;\; -1]$ to compute $-\partial^2/\partial x^2$.

Standard difference masks (orientations):

Operator	Mask	Detects
$-\partial/\partial y$	$[\,-1,\; 1\,]^T$	Horizontal edges (vertical intensity change)
$-\partial/\partial x$	$[\,-1,\;\; 1\,]$	Vertical edges (horizontal intensity change)
$-\partial^2/\partial y^2$	$[\,-1,\; 2,\; -1\,]^T$	Horizontal edges (2nd order)
$-\partial^2/\partial x^2$	$[\,-1,\; 2,\; -1\,]$	Vertical edges (2nd order)

Diagonal versions replace the row/column with the corresponding diagonal pattern (e.g. $\left(\begin{smallmatrix}-1 & 0\\ 0 & 1\end{smallmatrix}\right)$ for a diagonal 1st derivative).

Laplacian Mask:

Laplacian

The Laplacian combines 2nd derivatives in x and y directions:

$$L \;=\; \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix} \;\approx\; -\frac{\partial^2 I}{\partial x^2} - \frac{\partial^2 I}{\partial y^2}.$$

A simpler 4-neighbour variant is $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$. Detects intensity discontinuities at *all* orientations. (Strictly the "true" Laplacian is the additive inverse of these masks, but the convention is to use this sign.)

Spot it in a question

"Sum-to-zero" weights ⇒ differencing mask. "All weights positive, sum to 1" ⇒ smoothing mask. Masks with isotropic (rotationally symmetric) shape detect edges in all orientations (Laplacian, LoG, DoG); separable derivative masks detect a single direction.

3.2.4 Edge Detection

Edge

An image region in which the intensity gradients have a common direction and large magnitudes (in a direction orthogonal to the edge). An edge is an *extended, coherent* intensity discontinuity (not a single noisy pixel).

Idealised edges: ***step*** (light → dark transition), ***bar*** (thin strip at different intensity than its surroundings).

Causes of intensity discontinuities (D–O–R–I):

Cause	Description
Depth	Surfaces at different distances (object boundary in front of background)
Orientation	Two surfaces meeting at an angle
Reflectance	Same surface, different material/colour properties
Illumination	Lighting changes (e.g. shadow boundaries); usually *not* useful for recognition

---

The Edge–Noise Trade-off:

Why we need both smoothing and differencing

- Differencing alone: enhances edges, but also enhances noise (a single bright pixel produces the same local response as a real bar edge).
- Smoothing alone: removes noise, but blurs edges (and so weakens differencing responses).

Edge detection therefore needs:

1. a **smoothing** operator (Gaussian) that establishes a ***spatial scale*** via σ;
2. a **differencing** operator that finds significant intensity changes *at that scale*.

Two equivalent pipelines (by associativity of convolution):

$$\big((I * G_\sigma) * \partial\big) \;=\; I * (G_\sigma * \partial)$$

The right-hand side combines smoothing and differencing into a *single* edge-detection mask, which is more efficient than applying them sequentially.

Edge-detection masks:

Mask	Construction	Detects
Laplacian (no smoothing)	2nd derivative, all directions	Edges + lots of noise
Laplacian of Gaussian (LoG)	$\nabla^2 G_\sigma$ (Gaussian then Laplacian)	Edges at scale σ, all directions
Difference of Gaussians (DoG)	$G_{\sigma_1} - G_{\sigma_2}$, $\sigma_2 > \sigma_1$	Approximates LoG (centre-surround / Mexican hat)
Derivative of Gaussian (1st)	$\partial G_\sigma/\partial x$, $\partial G_\sigma/\partial y$	Oriented edges; combine via magnitude

Laplacian of Gaussian (LoG)

A Gaussian smoothes; a Laplacian differentiates. Combined into one mask:

$$\mathrm{LoG}_\sigma(x,y) \;=\; \nabla^2 G_\sigma(x,y) \;\approx\; -\frac{\partial^2 G_\sigma}{\partial x^2} - \frac{\partial^2 G_\sigma}{\partial y^2}.$$

Looks like a centre-surround / "Mexican hat" (positive centre, negative ring).

Difference of Gaussians (DoG)

$$\mathrm{DoG}(x,y) \;=\; \frac{1}{2\pi\sigma_1^2}\exp\!\left(-\tfrac{x^2+y^2}{2\sigma_1^2}\right) \;-\; \frac{1}{2\pi\sigma_2^2}\exp\!\left(-\tfrac{x^2+y^2}{2\sigma_2^2}\right), \quad \sigma_2 > \sigma_1.$$

Cheap, separable approximation to the LoG. Same role as centre-surround retinal ganglion cells.

Derivative-of-Gaussian magnitude. Convolve image with $\partial G_\sigma/\partial x$ and $\partial G_\sigma/\partial y$ to get I_x, I_y, then

$$M \;=\; \sqrt{I_x^2 + I_y^2}, \qquad D \;=\; \arctan\!\left(\frac{I_y}{I_x}\right).$$

Tends to give the *least noisy* result among the three families and is the basis of Canny.

Effect of scale σ. Choosing σ chooses which edges survive. Small σ: fine detail, more noise. Large σ: only coarse edges, fine details smoothed away. Different σ values can produce completely different edge maps for the same image.

The Canny Edge Detector:

Canny pipeline

1. **Smooth + differentiate**: convolve I with $\partial G_\sigma/\partial x$ and $\partial G_\sigma/\partial y$ to get I_x, I_y.
2. **Magnitude and direction**: $M = \sqrt{I_x^2 + I_y^2}$, $D = \arctan(I_y/I_x)$.
3. **Non-maximum suppression (NMS)**: thin multi-pixel ridges to a single pixel by removing a pixel if a neighbour *perpendicular to the edge direction* has higher magnitude.
4. **Hysteresis thresholding (recursive)**: with a high T_h and low T_l:
   - $M > T_h \Rightarrow$ accept (edge = 1).
   - $M < T_l \Rightarrow$ reject (edge = 0).
   - $T_l \le M \le T_h \Rightarrow$ accept iff connected (recursively) to an accepted pixel.

Three Canny parameters: σ, T_l, T_h. Results are highly parameter-dependent; values that work for one image often fail on another.

Limitations:

- Edge detection finds intensity discontinuities, not only "meaningful" edges. Reflectance / illumination edges are detected just like object boundaries.
- Cannot tell which detected edges correspond to true object contours.

Exam Tip: Canny answer pattern

A common exam question is: "Describe the steps of the Canny edge detector." Use the four-step pipeline above. Mention that step 1 *combines* smoothing and differencing (derivative-of-Gaussian) thanks to convolution being associative.

3.2.5 Multi-Scale Feature Detection

General feature-detection algorithm:

1. Convolve the image with a mask shaped like the (rotated) feature template.
2. Choose a threshold.
3. Mark the feature wherever the convolved image exceeds the threshold.

Problem. The feature scale is unknown (depends on imaging parameters). Two strategies:

Strategy	Idea	Verdict
(1) Vary mask size	Many big masks on the original image	Expensive (mask cost grows with size)
(2) Vary image size	One small mask on resized images	**Preferred** – cheaper. This is an ***image pyramid***.

Cost comparison (an $n \times n$ image with an $m \times m$ mask costs $m^2 n^2$ multiplies):

- Method (1) at double scale: 100×100 image with 6×6 mask = 360,000 multiplies.
- Method (2) at double scale: 50×50 image with 3×3 mask = 22,500 multiplies (16× cheaper).

Down-sampling:

Down-sample by factor s

$$\downarrow_s(I)(i,j) \;=\; I(s\,i,\; s\,j)$$

Take every s-th pixel of the original image.

Aliasing. Naive down-sampling (just picking every s-th pixel) creates ***aliasing***: a single pixel may not be representative of the region it samples (e.g. chess-board patterns become unrecognisable).

Anti-aliasing solution. Smooth with a Gaussian *before* down-sampling: the smoothed pixel reflects a larger region. Larger $\sigma \Rightarrow$ less aliasing.

Gaussian Image Pyramid

A multi-scale representation of one image, built by iteratively Gaussian-smoothing then down-sampling:

$$I_i \;=\; \downarrow_2\!\big(I_{i-1} * G_\sigma\big), \qquad I_0 = I.$$

Same fixed mask can then be applied at each level to detect features at different scales.

Using the cascade identity $G_\sigma * G_\sigma = G_{\sigma\sqrt{2}}$, each successive level effectively smooths the original by a wider Gaussian, so each level corresponds to a different physical scale.

Laplacian Image Pyramid

A multi-scale representation of *intensity discontinuities*, built by subtracting consecutive Gaussian-smoothed images:

$$L_i \;=\; I_i - (I_i * G_\sigma) \;=\; \text{DoG response at level } i.$$

Each L_i behaves like a DoG/LoG response and highlights edges at the corresponding scale.

Pyramid construction (recursive): Gaussian level i is built by smoothing-and-downsampling Gaussian level $i-1$. The Laplacian level i is the difference between Gaussian level i and its smoothed version.

Spot it in a question

"Multi-scale representation of intensity discontinuities" $\Rightarrow$ **Laplacian** pyramid. "Multi-scale representation of an image" $\Rightarrow$ **Gaussian** pyramid. "How is aliasing avoided when down-sampling?" $\Rightarrow$ **Smooth with a Gaussian first.**

3.3 Worked Examples

Example 1: Convolution by hand (no padding)

Q: Compute $H * I_1$ using only locations where the mask fits entirely inside the image, where

$$H = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad I_1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

A: Rotate H by 180°:

$$H_{\text{rot}} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

The mask fits at four valid output locations. Centring the rotated mask (top-left aligned with each valid position) and computing the multiply-accumulate gives

$$H * I_1 = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

(Each non-zero output corresponds to one element of H_{rot} aligning with the single 1 in I_1.)

Example 2: Zero-padded convolution, output same size as I

Q: With

$$H = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}, \quad I = \begin{pmatrix} 0.25 & 1 & 0.8 \\ 0.75 & 1 & 1 \\ 0 & 1 & 0.4 \end{pmatrix},$$

compute $H * I$ by zero-padding I.

A: Rotate H to put the 1 in the middle-left position. Convolving with this rotated mask shifts I one column to the right; the leftmost column becomes zero (padding):

$$H * I = \begin{pmatrix} 0 & 0.25 & 1 \\ 0 & 0.75 & 1 \\ 0 & 0 & 1 \end{pmatrix}.$$

Insight: a single off-centre 1 in a mask just shifts the image; the rotation in convolution flips the direction of that shift.

Example 3: Separable mask via outer product

Q: For $h = [1,\ 0.5,\ 0.1]$, compute $h * h^T$ and verify it equals $h^T \times h$. Hence find $I * H$ when I is the all-ones 3×3 image.

A: The convolution of a row vector with a column vector equals their outer product:

$$h * h^T \ = \ h^T \times h \ = \ \begin{pmatrix} 1 \\ 0.5 \\ 0.1 \end{pmatrix} [1 \ \ 0.5 \ \ 0.1] = \begin{pmatrix} 1 & 0.5 & 0.1 \\ 0.5 & 0.25 & 0.05 \\ 0.1 & 0.05 & 0.01 \end{pmatrix} = H.$$

By associativity, $I * H = (I * h) * h^T$. Computing the two 1D steps with zero-padding gives

$$I * h = \begin{pmatrix} 1.5 & 1.6 & 0.6 \\ 1.5 & 1.6 & 0.6 \\ 1.5 & 1.6 & 0.6 \end{pmatrix}, \quad (I * h) * h^T = \begin{pmatrix} 2.25 & 2.4 & 0.9 \\ 2.4 & 2.56 & 0.96 \\ 0.9 & 0.96 & 0.36 \end{pmatrix}.$$

Take-away: a separable 3×3 filter needs $3 + 3 = 6$ multiplies/pixel instead of 9.

Example 4: Causes of intensity discontinuities

Q: List the categories of image features that can produce intensity-level discontinuities.

A: D–O–R–I:

- **Depth** discontinuities (surfaces at different distances).
- **Orientation** discontinuities (surfaces meeting at an angle).
- **Reflectance** discontinuities (different surface materials/colours).
- **Illumination** discontinuities (e.g. shadow boundaries; usually not useful for recognition).

Example 5: Derivative masks

Q: Write down convolution masks that approximate (a) $-\partial/\partial x$, (b) $-\partial/\partial y$, (c) $-\partial^2/\partial x^2$, (d) $-\partial^2/\partial y^2$, (e) $-\partial^2/\partial x^2 - \partial^2/\partial y^2$ (Laplacian).

A:

- (a) $[-1 \ \ 1]$ (b) $\begin{pmatrix} -1 \\ 1 \end{pmatrix}$
- (c) $[-1 \ \ 2 \ \ -1]$ (d) $\begin{pmatrix} -1 \\ 2 \\ -1 \end{pmatrix}$
- (e) Laplacian: $\begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$ or $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$.

Example 6: Convolving a derivative mask with itself

Q: Convolve $[-1 \;\; 1]$ with itself to produce a 3-pixel result; interpret.

A: Pad to $[0 \;\; -1 \;\; 1 \;\; 0]$; rotated mask is $[1 \;\; -1]$. The output is

$$[\,(0)(1)+(-1)(-1),\;\; (-1)(1)+(1)(-1),\;\; (1)(1)+(0)(-1)\,] \;=\; [1,\; -2,\; 1].$$

Hence $\big(-\partial/\partial x\big) * \big(-\partial/\partial x\big) = \partial^2/\partial x^2$: applying the 1st-derivative mask twice gives the 2nd-derivative mask.

Example 7: Modified Laplacian preserves the input

Q: Let $L = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$ and $L_1 = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{pmatrix}$. (a) Express $I * L$ in derivatives. (b) Hence express $I * L_1$.

A: (a) $I * L \approx -\big(\partial^2 I/\partial x^2 + \partial^2 I/\partial y^2\big)$.

(b) Note $L_1 = L + L_2$ where L_2 is the identity mask (a 1 at the centre, zeros elsewhere). Using $h_1 * I + h_2 * I = (h_1 + h_2) * I$:

$$I * L_1 \;=\; I * L_2 + I * L \;\approx\; I \;-\; \left(\frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}\right).$$

This is the original image *enhanced* by adding edge information (a sharpening filter). The version with 1/8 weights and centre 1 gives the same form with a different scale of the Laplacian term.

Example 8: 3×3 Gaussian numerical approximation

Q: Build a 3×3 Gaussian mask with $\sigma = 0.46$ pixels using $G(x,y) = \frac{1}{2\pi\sigma^2}\exp\!\left(-\frac{x^2+y^2}{2\sigma^2}\right)$, rounded to 2 d.p.

A: Evaluate at offsets $(x,y) \in \{-1,0,1\}^2$. With $\sigma^2 = 0.2116$ and $2\pi\sigma^2 \approx 1.329$:

- Centre $(0,0)$: $G \approx 0.7522 \to 0.75$.
- Edges $(\pm 1, 0)$ and $(0, \pm 1)$: $G \approx 0.0708 \to 0.07$.
- Corners $(\pm 1, \pm 1)$: $G \approx 0.0067 \to 0.01$.

$$H \;=\; \begin{pmatrix} 0.01 & 0.07 & 0.01 \\ 0.07 & 0.75 & 0.07 \\ 0.01 & 0.07 & 0.01 \end{pmatrix}.$$

Example 9: Down-sample by 2 (no smoothing)

Q: Down-sample

$$I = \begin{pmatrix} 0.3 & 0.2 & 0.1 & 0.5 & 0.5 & 0.4 \\ 0.4 & 0.1 & 0.3 & 0.6 & 0.2 & 1 \\ 0.6 & 0.3 & 0.8 & 0.2 & 0.5 & 0 \\ 0.3 & 0.3 & 0 & 0.5 & 0.6 & 0.9 \\ 0.6 & 0.4 & 0.9 & 1 & 0.7 & 0.9 \\ 0.7 & 0.5 & 0.7 & 0.5 & 0.4 & 0.8 \end{pmatrix}$$

by a factor of 2.

A: Take every 2nd pixel (rows/columns indexed $0, 2, 4$):

$$\downarrow_2(I) \;=\; \begin{pmatrix} 0.1 & 0.6 & 1 \\ 0.3 & 0.5 & 0.9 \\ 0.5 & 0.5 & 0.8 \end{pmatrix}.$$

Pitfall: this is naive sub-sampling and *will* alias. In a Gaussian pyramid we Gaussian-smooth *before* this step.

Example 10: LoG vs DoG

Q: (a) How are a Laplacian and a Gaussian "combined" to make an LoG? (b) Why is this advantageous for edge detection? (c) What other function approximates an LoG?

A:

- (a) By **convolution**: $\text{LoG}_\sigma = G_\sigma * L = \nabla^2 G_\sigma$.
- (b) The Laplacian alone is sensitive to noise as well as real edges; the Gaussian suppresses noise; convolving them yields a single mask sensitive to genuine intensity discontinuities at scale σ, not single-pixel noise.
- (c) A **Difference of Gaussians** (DoG) with $\sigma_2 > \sigma_1$ approximates an LoG.

Example 11: Aliasing and pyramids

Q: What is aliasing and how is it avoided when down-sampling images to create an image pyramid?

A: Aliasing is the distortion or misrepresentation that arises when sampling an image too coarsely (a single pixel is not representative of its neighbourhood). It is avoided by **Gaussian-smoothing the image before down-sampling**, so each retained pixel summarises a wider region.

Example 12: Gaussian vs Laplacian pyramid

Q: Briefly describe (a) a Gaussian image pyramid and (b) a Laplacian image pyramid.

A:

- (a) A multi-scale representation of a single image at different resolutions, built by iteratively Gaussian-smoothing and down-sampling.
- (b) A multi-scale representation of *intensity discontinuities*, built by iteratively Gaussian-smoothing, subtracting the smoothed image from the previous level (DoG response), and down-sampling the smoothed image.

3.4 Summary

Key Takeaways

1. **Convolution** $I'(i,j) = \sum_{k,l} I(i+k, j+l)\, H(-k,-l)$: rotate mask, slide, multiply-accumulate. Commutative and associative; preferred over cross-correlation.
2. **Mask families:** smoothing (positive, sum-to-1) vs differencing (mixed sign, sum-to-0).
3. **Separability:** $H = h * h^T$ turns one 2D convolution into two 1D convolutions; the Gaussian is separable.
4. **Edge detection requires both** a smoothing operator (Gaussian, scale σ) and a differencing operator (1st or 2nd derivative).
5. **LoG/DoG/Derivative-of-Gaussian** are the three standard edge masks; derivative-of-Gaussian is the cleanest (used by Canny).
6. **Canny pipeline:** smooth → derivative → magnitude/direction → NMS → recursive hysteresis thresholding.
7. **Causes of edges (D–O–R–I):** depth, orientation, reflectance, illumination.

8. **Pyramids:** fix the mask, resize the image. Gaussian smoothing before down-sampling avoids aliasing. Laplacian pyramid = difference of Gaussian-pyramid levels.

Final comparison: filters and what they do.

Mask	**Type**	**Sum**	**Effect**
Box $\frac{1}{9}\mathbf{1}_{3\times 3}$	Smoothing	1	Mean filter; blurs; sharp mask edges
Gaussian G_σ	Smoothing	1	Gentle blur, separable, scale $= \sigma$
$[-1 \;\; 1]$	1st derivative $(-\partial/\partial x)$	0	Vertical-edge response
$[-1 \;\; 2 \;\; -1]$	2nd derivative $(-\partial^2/\partial x^2)$	0	Vertical-edge response (zero-crossings)
Laplacian (8-neighbour)	2nd derivative, all dirs	0	Edges in any orientation; noise-prone
LoG $= \nabla^2 G_\sigma$	Smoothed Laplacian	0	Edges at scale σ, all dirs
DoG $= G_{\sigma_1} - G_{\sigma_2}$	Centre-surround	0	Cheap LoG approximation
$\partial G_\sigma/\partial x,\ \partial G_\sigma/\partial y$	Derivative of Gaussian	0	Oriented edges; basis of Canny

Final comparison: pyramids.

Pyramid	**Construction**	**What it represents**
Gaussian	Iterate: Gaussian-smooth, down-sample by 2	Multi-scale image (low-pass)
Laplacian	Subtract consecutive Gaussian-pyramid levels (DoG)	Multi-scale intensity discontinuities (band-pass)

4 Week 4: Low- and Mid-Level Biological Vision

4.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Describe the pathway from retina to V1 via the LGN, and the "what/where" streams beyond V1.
- Identify the stimulus selectivities of V1 cells (colour, orientation, motion, frequency, eye of origin, disparity, position).
- Distinguish simple, complex, and hyper-complex cells, and describe the hypercolumn organisation of V1.
- Define the 2D Gabor function, list its 5 parameters, and explain how it models simple/complex cell responses.
- Explain why Gabors arise as the components of natural images under a sparsity constraint and link this to efficient coding.
- Distinguish classical and non-classical receptive fields and explain contextual effects (collinear facilitation, surround suppression, pop-out, texture segmentation).
- Distinguish low-, mid- and high-level vision; define grouping/segmentation; compare top-down and bottom-up influences.
- State and recognise the Gestalt laws of grouping.
- Define border ownership and explain the role of V2 lateral connections.

Big Picture. This week steps through the ***biological visual pipeline***: light reaches the retina, retinal ganglion cells project via the LGN to primary visual cortex (V1), where neurons compute richer features (orientation, motion, disparity). ***Gabor functions*** provide a clean mathematical model of V1 simple-cell receptive fields and—under a sparsity prior—turn out to be the natural-image dictionary the cortex has learned. We then move beyond the ***classical receptive field***: lateral connections in V1 produce ***contextual effects*** (contour integration, pop-out, texture segmentation) that drive bottom-up grouping. Finally, ***mid-level vision*** groups image elements into objects: ***Gestalt laws*** describe bottom-up grouping principles, and ***border ownership*** (computed in V2) decides which side of an edge is foreground.

What Examiners Like to Ask:

- List the image properties to which V1 cells are selective (Q1).
- Define a hypercolumn and compute neuron counts inside one (Q2, Q15).
- Distinguish simple vs complex (vs hyper-complex) cells; model them with convolution + Gabor (Q3, Q4, Q5).
- Compute Gabor values at $(0, 0)$ and complex-cell energy responses (Q14).
- State the sparsity constraint and link it to efficient coding (Q6, Q16).
- Define cRF vs ncRF; describe the association field (Q7, Q8).
- Explain how lateral connections give rise to contour integration, pop-out, texture segmentation (Q9, Q12).
- Compare top-down vs bottom-up grouping (Q10).
- Identify Gestalt laws from images (Q11).
- Define border ownership (Q13).

4.2 Core Concepts

Roadmap:

1. Biological visual system: retina → LGN → V1; what/where pathways.
2. V1 physiology: selectivities, simple/complex/hyper-complex cells, hypercolumns, retinotopy.
3. Gabor functions: definition, parameters, modelling simple/complex cells, efficient coding.
4. Non-classical receptive fields: association field, contextual effects.
5. Mid-level vision: grouping vs segmentation, top-down vs bottom-up.
6. Gestalt laws of bottom-up grouping.
7. Border ownership and V2.

4.2.1 The Biological Visual System

From eye to cortex:

Retina → Optic nerve → Optic chiasm → ***LGN*** (thalamus) → Primary visual cortex ***V1***

Lateral Geniculate Nucleus (LGN)

A thalamic relay station between retina and V1. LGN cells have centre-surround RFs, like retinal ganglion cells. Traditionally viewed as a passive relay, but recent evidence suggests it does more—exact computation unknown.

Visual field → cortex routing:

- The **right visual field (RVF)** (as seen by *both* eyes) projects to the **left LGN** and then to the **left V1** (and vice versa).
- Ganglion cells from the left side of *either* retina project to the left LGN; those from the right side of the right eye ***cross over*** at the **optic chiasm**.
- It is *not* the case that one eye projects to one hemisphere—each hemisphere sees the opposite visual field.

Exam Tip: RVF → Left V1

A common error is to write "right eye → left V1". The correct statement is "right *visual field* (seen by both eyes) → left V1".

Cerebral cortex basic facts:

- Folded sheet $\sim 1.7\,\mathrm{mm}$ thick, area $\sim 0.25\,\mathrm{m}^2$.
- $\sim 10^{10}$ neurons ($\sim 10^5$ neurons/mm^3), $\sim 4 \times 10^{13}$ synapses.
- $\sim 2\%$ of body mass, $\sim 20\%$ of energy consumption.
- $\sim 50\%$ of cortex is devoted to vision; ~ 30 distinct visual areas.

What and Where pathways:

Beyond V1 the cortical visual system is hierarchical (simple/local RFs → complex/large RFs) and splits into two streams:

Pathway	Route	Function
Where (How)	V1 → Parietal cortex	Spatial / motion information
What	V1 → Inferotemporal cortex	Identity / category information

---

4.2.2 Primary Visual Cortex (V1)

V1 (a.k.a. ***striate cortex*** / ***area 17***) performs the initial low-level processing of visual input. It contains a wider range of RF types than retina/LGN.

V1 Cell Selectivities

V1 cells have receptive fields selective for:

- Colour
- Orientation
- Direction of motion
- Spatial frequency
- Eye of origin
- Binocular disparity
- Position (always—stimulus must fall in the cell's RF)

Colour, eye-of-origin, position appear in retina/LGN too. Orientation, direction of motion, and binocular disparity appear *first* in V1.

Centre-surround / colour cells (also seen in retina/LGN):

- **Broad-band centre-surround**: detect intensity changes (DoG-like).
- **Colour-opponent centre-surround**: e.g. R+/G−.
- **Colour-opponent centre-only**.
- **Double-opponent (DO) cells**—first appear in V1: e.g. R+G− centre with G+R− surround. Detect places where colour itself changes (red object on green background). Larger RF than simple opponent cells.

Orientation-selective cells:

V1 cells (unlike retinal/LGN cells) are sharply tuned to a preferred orientation: response falls to zero when a line is tilted ±15–30° off the preferred angle.

Cell type	Optimum response	Function
Simple	Correctly oriented stimulus, *specific contrast polarity*, at a *specific position* in the RF	Edge / bar detector at fixed phase
Complex	Correctly oriented stimulus, *any contrast polarity*, *anywhere* in the RF	Edge / bar detector with positional and phase invariance; often direction-selective
Hyper-complex (end-stopped)	Correct orientation *and* length matched to RF width	Bar / corner / end-of-line detector

Exam Tip: Simple vs Complex in One Line

"Simple = position- and contrast-locked; Complex = position- and contrast-invariant; Hyper-complex = also length-tuned (end-stopping)."

Spatial frequency tuning: V1 cells have a preferred spatial frequency (cycles/degree) and respond more weakly to gratings above and below it.

Eye of origin and disparity:

- **Monocular cells**: receive input from one eye only (retina, LGN, and V1 cells driven by LGN).
- **Binocular cells** (in V1): two matched RFs (one per eye). Same orientation/direction preference, both simple or both complex.
- **Disparity-tuned cells**: respond maximally when the same stimulus appears with a specific offset between the two eyes—encodes ***depth*** via binocular disparity (near vs far via crossed/uncrossed disparity).

Conjunctions: most cells are selective for *several* properties simultaneously (e.g. orientation + direction of motion + position).

V1 organisation:

Hypercolumn

A $\sim 1\,\mathrm{mm}^2$ patch of V1 containing the full range of RF types (orientations, frequencies, eye of origin, etc.) for one retinal location. Contains the neural machinery to analyse *all* attributes of an image at one localised position.

Retinotopic Organisation

Adjacent hypercolumns process adjacent retinal locations: V1 preserves the topological layout of the retina. The map is distorted by ***cortical magnification***—more cortex is devoted to the fovea than to the periphery.

4.2.3 Gabor Functions

2D Gabor Function

A Gaussian envelope multiplied by a sinusoid:

$$G(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos(2\pi f x' + \psi)$$

where the rotated coordinates are

$$x' = x\cos\theta + y\sin\theta, \qquad y' = -x\sin\theta + y\cos\theta.$$

The 5 parameters:

Symbol	Name	Effect
σ	Std. dev. of Gaussian	Spatial extent of the RF
f	Frequency of sinusoid	Preferred spatial frequency
ψ	Phase of sinusoid	Even-symmetric ($\psi = 0$) vs odd-symmetric ($\psi = \pi/2$)
θ	Orientation	Preferred edge/bar angle
γ	Spatial aspect ratio	Elongation of envelope along the bar

Exam Tip: Why Gabors?

Real V1 simple-cell RFs (mapped experimentally) look like oriented Gaussian-windowed sinusoids. Gabors are the standard mathematical model: simple, two "parts" (Gaussian + sinusoid), and they fit the data well.

Modelling V1 cells with convolution:

Simple Cell Model

The response of all simple cells with the same parameters $(\sigma, f, \psi, \theta, \gamma)$, across all hypercolumns, is approximated by convolving the image with a Gabor mask G:

$$R_{\text{simple}}(x, y) = (I * G)(x, y).$$

Repeating with different $(\sigma, f, \psi, \theta, \gamma)$ simulates the rest of V1's simple cells.

Complex Cell Energy Model

Pool the outputs of a ***quadrature pair*** of Gabors with the same parameters except ψ differs by $\pi/2$:

$$R_{\text{complex}}(x, y) = \sqrt{(I * G_{\psi=0})^2 + (I * G_{\psi=\pi/2})^2}.$$

Result is approximately *phase-invariant* (and hence contrast-polarity-invariant), like real complex cells.

Multi-orientation edge detection: sum complex-cell outputs across multiple θ to detect edges of any orientation.

Multi-scale (wavelet transform): a family of Gabors with different (f, σ) behaves like a continuous wavelet transform. V1 simple cells $\approx$ a biological wavelet transform.

Gabors as image components:

Represent an image $\mathbf{x}$ (vectorised) as a linear combination of components (columns of A):

$$A\,\mathbf{y} \approx \mathbf{x}$$

where A is $m \times n$ (components in columns), $\mathbf{y}$ is $n \times 1$ (activations), $\mathbf{x}$ is $m \times 1$ (pixel values). For a dataset of images: $AY \approx X$.

Finding A and Y is ***ill-posed***—additional constraints pick out particular factorisations (PCA, ICA, NMF, ...).

Gabors from Sparsity

Take random natural-image patches as columns of X and find A, Y subject to:

1. **Reconstruction**: $\|X - AY\|$ minimised.

2. **Sparsity**: minimise the number of non-zero entries per column of Y.

The resulting components (columns of A) look like Gabor functions: Gabors capture the intrinsic structure of natural images.

Sparsity Constraint

Each image is represented using as few active components as possible. If components are encoded by neurons, this means *few neurons fire per image*—an ***efficient code*** (low energy, low bandwidth).

Computer-vision applications of image-component models:

Application	Idea
Compression (JPEG)	Reconstruct each patch from a fixed dictionary (cosines, not Gabors); store quantised activations $\mathbf{y}$ instead of $\mathbf{x}$
Denoising	Reconstruct noisy patches with a Gabor (or curvelet/wedgelet) dictionary; reconstruction is smoother and less noisy
Inpainting	Learn A from non-corrupted parts; find activations $\mathbf{y}$ that minimise $\|\mathbf{x} - A\mathbf{y}\|$ over visible pixels; use $A\mathbf{y}$ to fill in the missing pixels

---

4.2.4 Non-Classical Receptive Fields

Classical vs Non-classical RF

- **Classical RF (cRF)**: region of visual space (or stimulus property set) that can elicit a response from the neuron on its own.
- **Non-classical RF (ncRF)**: region/properties that *modulate* the response but cannot drive it alone—requires concurrent input to the cRF.

Association Field

The pattern of long-range lateral (and feedback) connections received by an orientation-selective V1 neuron—the neural substrate of its ncRF.

- **Excitation** from neighbours that are ***collinear*** or ***co-circular*** with the same orientation preference (continuation along smooth contours).
- **Inhibition** from nearby ***iso-oriented*** parallel neighbours (similar but not aligned).

Three rules of thumb:

1. Context has *no effect in isolation* (ncRF cannot drive the cell).
2. Context can *enhance* the response (excitation).
3. Context can *reduce* the response (inhibition).

Contextual effects driven by lateral connections:

Effect	Mechanism	Perceptual consequence
Collinear facilitation	Excitation between collinear iso-oriented cells	Aligned Gabor patches are easier to see
Contour integration	Excitation among co-circular/collinear cells	Smooth contours pop out of noise
Surround suppression	Inhibition between iso-oriented neighbours	Targets among similar distractors are harc to see
Pop-out (feature search)	Inhibition between similar-feature cells	Dissimilar element among similar distract stands out
Texture segmentation	Inhibition within uniform texture regions	Boundary cells get less inhibition → relativ higher response → visible border

Exam Tip: One-Word Mnemonic

Excitation → contours (continuity, similarity-of-alignment).
Inhibition → boundaries (pop-out, texture segmentation, similarity-based grouping).

Pure convolution-based edge detectors (e.g. Canny applied alone) detect *all* edges including texture noise. ncRF-style operations are needed to select *meaningful* edges (object boundaries).

4.2.5 Introduction to Mid-Level Vision

Mid-Level Vision

The set of processes that bridge low-level vision (noise suppression, edge enhancement, feature extraction, efficient coding) and high-level vision (object recognition, classification, localisation, scene understanding). Its central job is to ***group*** elements that "belong together" and ***segment*** them from everything else.

Elements (or tokens) can be anything we need to group: pixel intensities, pixel colours, edges, features.

Two Sides of the Same Coin

Grouping (assemble pieces of one object) and **segmentation** (separate one object from another) are dual: doing one implies the other. The terms are used interchangeably.

Other names for the same process: image parsing, perceptual organisation, binding, perceptual grouping. ***Figure-ground segmentation*** is the restricted case where we segment one object (the figure) from everything else (the ground).

Two complementary approaches:

Approach	Cue source
Top-down	Internal knowledge / prior experience: elements belong together because they lie on the same *known* object (familiarity, expectation, meaningfulness).
Bottom-up	Image properties: elements belong together because they are *similar* or *locally coherent* (Gestalt laws).

These approaches are *not* mutually exclusive. General-purpose segmentation requires *both*.

4.2.6 Gestalt Laws (Bottom-up Grouping Principles)

Gestalt Laws

Bottom-up heuristics ("rules of thumb," not strict laws) describing which image elements humans tend to group together. Proposed by Gestalt psychologists in the early 20th century; extended by later work.

Original Gestalt laws:

Law	Tendency
Proximity	Nearby elements are grouped together.
Similarity	Similar elements are grouped (similarity in shape, orientation, size, colour, luminance, motion, ...).
Closure	Elements that form a closed boundary group; even *potential* closure suffices.
Continuity (good continuation)	Elements forming straight or smoothly curving lines/surfaces group; can produce ***illusory contours***.
Common fate	Elements moving coherently are grouped (Similarity for motion).
Symmetry	Symmetric configurations are grouped.

Later additions (not strictly Gestalt):

Law	Tendency
Common region	Elements inside the same closed region are grouped.
Connectivity	Elements joined by another element are grouped.

Exam Tip: Overlapping Laws

Several laws overlap: proximity = similarity-of-location; common fate = similarity-of-motion. There are no strict rules for resolving conflicts between laws (e.g. proximity vs similarity).

Neural implementation via V1 lateral connections:

- **Similarity** (for orientation, frequency, etc.): mutual lateral *inhibition* between cells representing similar elements. At borders between dissimilar regions inhibition is reduced, so the border "pops out".
- **Continuity**: mutual lateral *excitation* between cells with collinear/co-circular orientation preferences enhances responses to elements on smooth contours.

This is exactly the association field of Section *Non-Classical Receptive Fields*—the same mechanism explains both contextual modulation *and* the bottom-up Gestalt biases.

4.2.7 Border Ownership

Border Ownership

The boundary between two image regions is perceived as belonging to *one* of them (the foreground / figure). The foreground region acquires a definite shape; the background appears *borderless and shapeless* and seems to continue behind the figure.

Examples:

- Tennis player's leg has a border; the grass does not—the grass appears to continue behind the leg.
- FedEx logo: the arrow between "E" and "x" is invisible because the border is owned by the letters; the white region (which contains the arrow) has no shape.
- Bistable fish/birds figure: alternates—only the currently-perceived figure "owns" the borders.
- Rubin face/vase: which side owns the border determines whether you see two faces or a vase.

V2 border-ownership cells:

- V2 is the next cortical area after V1 in the visual hierarchy.
- Some V2 cells are *orientation-selective and side-selective*: they respond strongly only when the figure is on a specific side of the edge in the cell's RF.
- In experiments where the local stimulus inside the cRF is *identical* but the wider context flips foreground/background, the response changes—so border ownership is an ***ncRF effect***.

Mechanistic model (V2 lateral connections): At every location and orientation V2 contains a *pair* of border-ownership cells preferring opposite figure sides. They compete via lateral connections:

- **Excitatory** connections link cells encoding edge segments *consistent* with a probable single object.
- **Inhibitory** connections link cells encoding segments *inconsistent* with a single object.

The network settles into a globally consistent assignment of borders to objects.

Exam Tip: Border Ownership Cues

Border-ownership decisions use cues from *outside* the classical RF: convexity, surroundedness, symmetry, T-junctions, occlusion patterns, familiarity. The local edge alone does not determine which side is figure.

4.3 Worked Examples

Example 1 (LGT Q1, Q2, Q3): V1 Selectivities, Hypercolumns, Cell Types

Q: (a) List the image properties to which V1 cells are selective. (b) What is a hypercolumn? (c) Give the stimulus selectivities of simple and complex cells.

A:

- (a) Colour, orientation, direction of motion, spatial frequency, eye of origin, binocular disparity, position.
- (b) A hypercolumn is a $\sim 1\,\mathrm{mm}^2$ region of V1 containing neurons covering the full range of RF types for a single spatial location on the retina.
- (c) Simple cell: optimum response to an appropriately oriented stimulus, of the right contrast polarity, at a specific position in its RF. Complex cell: optimum response to an appropriately oriented stimulus, with *any* contrast polarity, anywhere in its RF.

Example 2 (LGT Q4, Q5): Modelling V1 Cells with Convolution

Q: How can simple- and complex-cell responses be modelled using convolution?

A:

- **Simple cells**: each simple-cell RF is well approximated by a Gabor function. Convolving the image with a Gabor mask simulates the response of all simple cells with that parameter set across all hypercolumns. Repeating with different Gabor parameters (orientation, frequency, phase, aspect ratio) simulates the full population.
- **Complex cells** (energy model): convolve with a ***quadrature pair*** of Gabors (same parameters but ψ differing by $\pi/2$); take $\sqrt{R_1^2 + R_2^2}$. The result is approximately phase-invariant, matching the contrast-polarity invariance of real complex cells.

Example 3 (LGT Q14): Gabor at the Origin and Energy Response

Q: Compute $G(0,0)$ for a Gabor with $\theta = 0$, $\psi = 0$, $\sigma = 2$, $f = 0.5$, $\gamma = 0.5$. Then compute the energy response of a quadrature pair ($\psi = 0$, $\pi/2$) at $(0,0)$.

A: At $(0,0)$, $x' = y' = 0$, so:

$$\text{Gaussian} = \exp(0) = 1, \qquad \cos(2\pi f \cdot 0 + \psi) = \cos(\psi).$$

- Filter 1 ($\psi = 0$): $G_1 = 1 \cdot \cos(0) = 1$.
- Filter 2 ($\psi = \pi/2$): $G_2 = 1 \cdot \cos(\pi/2) = 0$.
- Energy: $\sqrt{G_1^2 + G_2^2} = \sqrt{1+0} = 1.0$.

Example 4 (LGT Q15): Neurons per Hypercolumn

Q: The cortex is $\sim 1.7\,\text{mm}$ thick with neuron density $\sim 10^5$ neurons/mm^3, and a hypercolumn covers $\sim 1\,\text{mm}^2$. (a) How many neurons are in one hypercolumn? (b) If a hypercolumn must encode 6 orientations $\times$ 2 directions of motion $\times$ 4 spatial frequencies $\times$ 2 ocular dominance columns, how many neurons per feature combination?

A:

- (a) Volume $= 1\,\text{mm}^2 \times 1.7\,\text{mm} = 1.7\,\text{mm}^3$. Neurons $= 1.7 \times 10^5 = 170{,}000$.
- (b) Feature combinations $= 6 \times 2 \times 4 \times 2 = 96$. Neurons per combination $\approx 170{,}000/96 \approx 1{,}770$.

Example 5 (LGT Q6, Q16): Sparsity, Efficient Coding, Dictionary Sizes

Q: (a) State the sparsity constraint and link it to efficient coding. (b) For 8×8 image patches, give the dimensions of A for a complete and a $4\times$ overcomplete dictionary. (c) With the overcomplete dictionary and a 10% sparsity constraint, how many components are active per patch?

A:

- (a) Sparsity = use the minimum number of non-zero activations per image. If components are encoded by neurons, only a few neurons need to fire to represent any image $\Rightarrow$ efficient (low-energy, low-bandwidth) code.
- (b) $m = 8 \times 8 = 64$ pixels. Complete: A is 64×64. Overcomplete: $n = 4 \times 64 = 256$, so A is 64×256.
- (c) Active components $= 0.1 \times 256 = 25.6 \approx 26$.

Example 6 (LGT Q7, Q8): Classical RF, Non-classical RF, Association Field

Q: Briefly define the classical RF (cRF) and non-classical RF (ncRF). What is an association field, and what does it look like for an orientation-selective V1 cell?

A:

- **cRF**: region of visual space (or stimulus properties) that can elicit a response from the neuron.
- **ncRF**: region/properties that can *modulate* the cRF response but cannot generate a response in the absence of cRF input.
- **Association field**: pattern of long-range lateral connections received by an orientation-selective V1 cell—it defines the ncRF. The cell receives lateral *excitation* from neighbours with similar orientation preferences that are ***collinear*** or ***co-circular*** with it, and lateral *inhibition* from other (parallel iso-oriented) neighbours with similar preferences.

Example 7 (LGT Q9, Q12): Lateral Connections and Contextual Effects

Q: How do lateral connections in V1 give rise to (a) contour integration, (b) pop-out, (c) texture segmentation? Then explain similarity and continuity Gestalt biases.

A:

- (a) **Contour integration**: lateral *excitation* between cells with collinear/co-circular orientation preferences mutually enhances responses; smooth contours become more visible than the noise around them.
- (b) **Pop-out**: lateral *inhibition* between cells with similar feature preferences mutually suppresses similar elements; the dissimilar element receives less inhibition and stands out.
- (c) **Texture segmentation**: cells inside a uniform-texture region inhibit each other and are damped; cells along the boundary between two textures receive inhibition from only one side and are relatively more active—the boundary becomes visible.
- **Similarity**: lateral inhibition mutually suppresses similar elements; borders between dissimilar elements receive less inhibition and are enhanced.
- **Continuity**: lateral excitation enhances elements forming continuous (collinear/co-circular) contours.

Example 8 (LGT Q10, Q11, Q13): Top-down vs Bottom-up, Identifying Gestalt Laws, Border Ownership

Q: (a) Compare top-down and bottom-up grouping. (b) Identify Gestalt laws for example images: (i) similar shapes group, (ii) elements inside the same enclosure group, (iii) close-together elements group, (iv) elements joined by a line group, (v) elements forming a closed shape group, (vi) elements on a smooth curve group. (c) Define border ownership.

A:

- (a) Top-down: comes from prior knowledge and experience—elements grouped because of expectations about what belongs to the same object. Bottom-up: comes from image properties—elements grouped because they are similar or locally coherent.
- (b) (i) Similarity, (ii) Common region, (iii) Proximity, (iv) Connectivity, (v) Closure, (vi) Continuity.
- (c) Border ownership: the boundary between two image regions is perceived as belonging to one region (the foreground) and not the other (the background). The foreground has a defined shape (delineated by the border); the background appears shapeless and seems to continue behind the figure. Implemented in V2 via cells whose response depends on which side of the edge the figure is on (an ncRF effect).

4.4 Summary

Key Takeaways

1. **Pathway**: Retina → LGN (centre-surround relay) → V1 (striate cortex). RVF → left V1 (and vice versa). Beyond V1: *What* (ventral, identity) and *Where* (dorsal, spatial/motion) streams.
2. **V1 selectivities**: colour, orientation, direction of motion, spatial frequency, eye of origin, binocular disparity, position. Orientation/disparity/motion direction first appear in V1.
3. **Cell types**: simple (position- and contrast-locked) → complex (phase-/contrast-invariant) → hyper-complex (also length-tuned, end-stopped).
4. **Organisation**: hypercolumn ($\sim$1 mm^2, full RF range for one retinal location) × retinotopic map (cortically magnified for fovea).
5. **Gabor model**: Gaussian × sinusoid with parameters $\sigma, f, \psi, \theta, \gamma$. Simple cell = Gabor convolution; complex cell = energy of a quadrature pair; multi-orientation × multi-scale = wavelet transform.
6. **Sparsity**: Gabors are the components of natural images under a sparsity prior ⇒ efficient code (few active neurons per image).
7. **cRF vs ncRF**: ncRF cannot drive the cell alone but modulates its response. The *association field* (excitation along collinear/co-circular axis, inhibition between iso-oriented parallel neighbours) explains contour integration, pop-out, texture segmentation.
8. **Mid-level vision**: grouping/segmentation. *Top-down* (knowledge) + *bottom-up* (image properties) are both needed.
9. **Gestalt laws**: proximity, similarity, closure, continuity, common fate, symmetry (+ common region, connectivity). Mostly heuristics; can conflict.
10. **Border ownership**: which side of an edge is foreground. Computed in V2 via lateral connections (excitation among consistent edge segments, inhibition among inconsistent ones).

Summary table: V1 cell types and their model.

Cell type	Selectivity	Model
Centre-surround (broad-band)	Intensity changes	Difference of Gaussians (DoG)
Centre-surround (colour-opponent)	Colour edges	DoG on opponent channels (R+G− etc.)
Double-opponent (V1 only)	Local change in colour	Larger RF; opponent both in centre and surround
Simple cell	Orientation + phase + position	Single Gabor $G(\sigma, f, \psi, \theta, \gamma)$
Complex cell	Orientation, position-/phase-invariant	Energy of quadrature pair: $\sqrt{R^2_{\psi=0} + R^2_{\psi=\pi/2}}$
Hyper-complex (end-stopped)	Orientation + length	Difference of complex-cell responses across length
Disparity-tuned (binocular)	Depth via L/R RF offset	Pair of matched RFs across the two eyes

Summary table: contextual effects and their mechanism.

Effect	Connection
Collinear facilitation / contour integration	Lateral excitation (collinear/co-circular)
Surround suppression / pop-out	Lateral inhibition (iso-oriented similar)
Texture segmentation	Lateral inhibition (within texture)
Border ownership (V2)	Excitation between consistent segments; inhibition between inconsistent

5 Week 5: Mid-Level Vision — Segmentation

5.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Define the role of mid-level vision and explain feature space and similarity/distance measures.
- Apply thresholding (single, hysteresis, local) and morphological clean-up (erosion, dilation, opening, closing).
- Apply region-based segmentation (region growing, region merging, split-and-merge) on small numerical examples.
- Apply clustering for segmentation (k-means and agglomerative hierarchical clustering with single/complete/average/centroid linkage).
- Explain segmentation by graph cutting and the Normalised Cuts (Ncuts) criterion.
- Apply the Hough transform for straight lines and explain its generalisation to circles and arbitrary shapes.
- Describe active contours (snakes) and the energy terms they minimise.

Big Picture. Mid-level vision is about ***grouping***: take low-level image elements (pixels, edges, filter responses) and gather those that "belong together" into segments. Every method in this week is a different answer to the same question: which feature(s) define belonging, and how do we measure similarity? Each pixel becomes a point in a ***feature space***, and segmentation reduces to partitioning that space. Methods differ in (i) what feature/measure they use, (ii) whether they reason in feature space alone or use spatial information, and (iii) what model they assume (uniform region, cluster, parametric shape, smooth curve).

What Examiners Like to Ask:

- "Briefly describe the role of mid-level vision."
- How to choose a threshold from an intensity histogram; explain hysteresis thresholding.
- Apply hysteresis thresholding by hand on a small numerical image.
- Show the result of erosion-then-dilation (opening) and dilation-then-erosion (closing) for 4- and 8-connected neighbourhoods.
- Write pseudo-code for region growing / region merging / split-and-merge / k-means / agglomerative / Ncuts / Hough.
- Apply each algorithm to a 3×3 feature-vector image with SAD as similarity and a numerical threshold.
- Describe single-link / complete-link / group-average / centroid linkage.
- Compute a Hough accumulator array for a small binary image at given θ values.
- Explain the two energy terms used by an active contour (snake).

5.2 Core Concepts

Roadmap:

1. Feature space and similarity / distance measures
2. Thresholding (and morphological clean-up)
3. Region-based methods (growing, merging, split-and-merge)
4. Clustering (k-means, agglomerative hierarchical, graph cuts/Ncuts)

5. Fitting (Hough transform, active contours)

5.2.1 Feature Space and Similarity

Mid-Level Vision (Segmentation)

The task of grouping together image elements that "belong together" (e.g. lie on the same object) and segmenting them from all other image elements.

Common features used for grouping (each motivates a Gestalt grouping law):

- Location (proximity), colour/intensity (similarity), texture (similarity)
- Depth, motion (common fate)
- Not separated by a contour (common region)
- Top-down: forms a known shape, or CNN feature vectors

Feature Space

Each image element is represented as a vector $\mathbf{a} = (a_1, \ldots, a_n)$ in an n-dimensional feature space. Similarity (and hence grouping) is determined by the distance between points in this space.

Similarity measures (high when vectors agree):

$$\text{Affinity} = \exp\left(-\frac{\sum_i (a_i - b_i)^2}{2\sigma^2}\right), \qquad \text{Cross-correlation} = \sum_i a_i b_i$$

$$\text{NCC} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2}\sqrt{\sum_i b_i^2}}, \qquad \text{Corr. coeff.} = \frac{\sum_i (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_i (a_i - \bar{a})^2}\sqrt{\sum_i (b_i - \bar{b})^2}}$$

Distance measures (low when vectors agree):

$$\text{Euclidean} = \sqrt{\sum_i (a_i - b_i)^2}, \quad \text{SSD} = \sum_i (a_i - b_i)^2, \quad \text{SAD} = \sum_i |a_i - b_i|$$

Exam Tip: SAD is the default in tutorial questions

Almost every numerical tutorial question for this week uses **Sum of Absolute Differences (SAD)** as the similarity measure with a threshold on SAD. Learn to compute SAD quickly: just add the absolute element-wise differences.

Feature scaling and weighting. Different features have different scales (e.g. pixel coordinates in $[0, 2000]$ vs. intensity in $[0, 1]$). If left raw, large-scale features dominate the distance and small-scale features look "always similar". Solutions:

- Normalise each feature to a common range.
- Use weighted distance, e.g. $\sqrt{w_1(a_1 - b_1)^2 + w_2(a_2 - b_2)^2 + \ldots}$. Choosing w_i that work well is non-trivial (requires optimisation).

5.2.2 Thresholding

Thresholding

A 1-D feature space segmentation: assign pixel (x, y) to foreground if its feature value $I(x, y)$ exceeds a threshold:

$$I'(x, y) = \begin{cases} 1 & \text{if } I(x, y) > t \\ 0 & \text{otherwise} \end{cases}$$

Produces ***figure-ground*** segmentation. The feature can be intensity, colour channel, or output of a filter (e.g. edge magnitude).

How to choose the threshold:

1. **Average intensity** of the image.
2. **Histogram-based**: plot the intensity histogram, assume it is ***bimodal***, place the threshold in the valley between the two peaks.
3. **Hysteresis thresholding**: use *two* thresholds (one each side of the valley).
4. **Fraction-of-image**: choose t so a known fraction of pixels are foreground (when object size is known).

Hysteresis Thresholding

Define a high threshold t_h and a low threshold t_l (with $t_l < t_h$):

- Pixels with $I > t_h$: classified as figure (foreground).
- Pixels with $I < t_l$: classified as background.
- Pixels with $t_l \leq I \leq t_h$: classified as figure *only if* they are adjacent to a pixel already classified as figure (otherwise background).

Local / block thresholding. A single global threshold rarely works for whole images (uneven illumination, shadows). Split the image into rectangular blocks and apply a different threshold per block — it is the *local* contrast that matters.

Thresholding errors and morphological clean-up. Thresholding usually leaves artefacts: missing pixels (holes in objects) and unwanted pixels (specks of noise). Morphological operations clean these up.

Dilation and Erosion

Both use a ***structuring element*** (typically 3×3) defining a neighbourhood (4-connected: H/V; 8-connected: H/V/diagonal).

- **Dilation**: every background pixel that has a foreground neighbour becomes foreground (1). Expands foreground; fills holes/gaps.
- **Erosion**: every foreground pixel that has a background neighbour becomes background (0). Shrinks foreground; removes bridges/specks.

Opening and Closing

- **Opening** = erosion then dilation. Removes background noise (small foreground blobs disappear); leaves overall shape unchanged.
- **Closing** = dilation then erosion. Fills foreground holes/gaps; leaves overall shape unchanged.

Exam Tip: Opening vs Closing

Mnemonic: "open removes specks, close fills holes". The first operation (erosion or dilation) tells you what is removed; the second restores the surviving objects to roughly their original size.

5.2.3 Region-Based Segmentation

Motivation. Thresholding ignores spatial information. Region-based methods consider pixel *location* as well as feature values, so the feature space is multi-dimensional (e.g. RGB, or RGB + position).

Region Growing

Start from a seed pixel; iteratively add neighbouring pixels whose features are within a similarity threshold of the seed's region.

Pseudo-code (region growing):

1. While not all pixels have a region label:
   (a) Choose an unlabelled pixel at random.
   (b) Assign it the next unused region label.
   (c) For all neighbouring unlabelled pixels:
      i. Calculate similarity to the chosen pixel.
      ii. If within similarity threshold: give it the same region label, then make that pixel the chosen pixel and recurse.

When to use. You have plausible seed locations (or can pick them randomly) and a per-pixel similarity criterion.
Pitfalls. Regions can "leak" through a single weak spot in the boundary. If similarity uses intensity only, regions stop at edges — but only if edges are unbroken.

Region Merging

Start with every pixel as its own region; iteratively merge adjacent regions whose properties (mean feature vector) are similar.

Pseudo-code (region merging):

1. Give every pixel a unique region label.
2. Randomly choose a region not yet marked *final*.
   (a) Compare its properties with each neighbour's.
   (b) For all neighbours within similarity threshold: relabel them to the chosen region's label; recompute properties as the mean of all constituent pixels.
   (c) If no neighbour can be merged: mark the chosen region as *final*.
3. Repeat until all regions are marked *final*.

Pitfall: order-dependence. The result depends on the order in which regions are merged, because the merged region's mean shifts with each merge.

Split-and-Merge

Top-down then bottom-up. Recursively split heterogeneous regions into quadrants (a quadtree) until each region is homogeneous, then merge similar adjacent regions.

Pseudo-code (split-and-merge):

1. Label every pixel with the same region label.
2. For each region: if not all pixels are similar, split into 4 quadrants with new labels.
3. Repeat step 2 until all regions are homogeneous.
4. For each region: compare to neighbours; merge any whose properties are within the similarity threshold (recompute mean).
5. Repeat step 4 until no more merges possible.

Note. If image dimensions are not powers of 2, pad the image so it can be quartered cleanly.

Exam Tip: General problems with region-based methods

"Meaningful" regions are rarely uniform: brightness/colour vary due to lighting, curvature, and texture. Real images are almost never composed of uniform regions of similar intensity/colour. Be ready to state this as a limitation.

5.2.4 Clustering Methods

Clustering

A general class of methods for ***unsupervised classification*** of data — classes are not predefined. Aim: minimise intra-cluster distance, maximise inter-cluster distance.

Two sub-classes:

- **Partitional**: divide data into non-overlapping subsets (e.g. k-means).
- **Hierarchical**: nested clusters organised as a tree (dendrogram) — agglomerative (bottom-up) or divisive (top-down).

---

k-means Clustering

A partitional algorithm assuming there are k clusters. Iteratively assigns points to the nearest cluster centre, then recomputes each centre as the mean of its assigned points, until centres stop moving.

Pseudo-code (k-means):

1. Randomly choose k points to act as cluster centres.
2. For each data point: compute similarity to each cluster centre; allocate to closest.
3. For each cluster centre: set new position to mean of its allocated points.
4. Repeat 2–3 until cluster centres are unchanged (converged).

Problems with k-means:

- Results depend on initial centre positions (different runs $\Rightarrow$ different clusterings).
- Poor when true clusters have very different sizes.
- Poor when true clusters are not roughly "globular".
- Must choose k in advance.

Exam Tip: k-means converges when assignments stop changing

On 3×3 SAD examples, the assignment grid stabilises after one or two iterations. State explicitly: "Allocation is unchanged, so cluster centres are also unchanged and the algorithm has converged."

Agglomerative Hierarchical Clustering

Bottom-up: each data point starts as its own cluster; iteratively merge the two most similar clusters until a stopping criterion (single cluster, target k, or proximity threshold).

Pseudo-code (agglomerative):

1. Assign each data point to a unique cluster.
2. Compute the proximity matrix (distance between every pair of clusters).
3. Repeat:
   (a) Merge the two closest clusters.
   (b) Update the proximity matrix.
4. Until only a single cluster remains (or target reached).

Inter-cluster similarity / linkage methods:

Linkage	Distance between clusters
Single-link	MIN distance between any pair of elements
Complete-link	MAX distance between any pair of elements
Group-average	AVERAGE distance over all pairs of elements
Centroid	Distance between the cluster centroids (means)

Different linkages produce different dendrograms on the same data.

Exam Tip: ties merge together

In tutorial questions on agglomerative clustering, several pairs may be tied at the minimum distance. The model answer merges *all tied pairs in the same step*, then updates the proximity matrix once.

Segmentation by Graph Cutting

Build a weighted graph $G = (V, E)$. Each vertex $v \in V$ is an image element (a feature vector); each edge $e_{pq} \in E$ has weight equal to the similarity between connected elements. Segmentation = partitioning G into disjoint subgraphs.

For two disjoint sets $A, B \subseteq V$:

$$\text{cut}(A, B) = \sum_{p \in A,\, q \in B} e_{pq}$$

Cutting edges with minimum total weight separates the most dissimilar parts of the graph. **Bias problem:** minimum cut favours small subgraphs (cost grows with the number of cut edges).

Normalised Cuts (Shi & Malik)

Normalise the cut cost by the total association of each side with the whole graph:

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}, \quad \text{assoc}(A, V) = \sum_{p \in A,\, q \in V} e_{pq}$$

Minimising Ncut avoids the bias toward small subgraphs.

Pseudo-code (Normalised Cuts):

1. Form a graph in which each vertex is an image element and each edge weight is the similarity between connected elements.
2. For every possible bipartition (A, B) of vertices, compute $\text{Ncut}(A, B)$.
3. Cut the edges between A and B for which Ncut is minimum.
4. Repeat steps 2–3 for each subgraph until either Ncut exceeds a threshold for every subgraph, or the maximum number of segments is reached.

Problems with Ncuts:

- Finding the exact minimum is NP-hard — in practice we use approximate (eigenvector-based) solutions.
- Bias toward partitioning into roughly equal segments.
- Struggles with textured backgrounds (a common weakness across most segmentation algorithms).

5.2.5 Fitting Methods

Fitting

A class of segmentation methods that fit a mathematical model (line, circle, arbitrary parametric shape, smooth curve) to image elements. Elements that fit the model are grouped together; the model's boundary becomes the segmentation.

Hough Transform (for straight lines)

For each candidate edge pixel, compute the parameters (θ, r) of *every* line passing through that pixel and increment a vote in an accumulator array indexed by (θ, r). Peaks in the accumulator correspond to lines supported by many pixels.

Line parameterisation. Avoid $y = mx + c$ (slope m and intercept c go to infinity for vertical lines). Use angle θ (with the horizontal axis) and perpendicular distance r from the origin:

$$y \cos\theta - x \sin\theta = r$$

Note: in image coordinates the y-axis points downwards, so positive θ corresponds to a clockwise rotation.

Pseudo-code (Hough transform for straight lines):

1. Initialise all accumulator entries to 0.
2. For each edge pixel (x, y):
   (a) For each quantised value of θ:
      i. Compute $r = y \cos\theta - x \sin\theta$.
      ii. Increment the accumulator entry nearest (θ, r).
3. Find peaks in the accumulator: each peak is a line.

Sizing the accumulator. The maximum $|r|$ is the image diagonal $\sqrt{W^2 + H^2}$ (round up). Quantise θ and r as required by the question.

Generalised Hough Transform.

- **Circles**: parameterise by centre (a, b) and radius r — 3-D accumulator. Each edge pixel votes for all (a, b, r) corresponding to circles passing through it.
- **Arbitrary (non-parametric) shapes**: store a table of edge offsets relative to a reference point. Each image edge pixel assumes it could be each table entry in turn and votes for the implied reference-point location. Peaks in the reference-point accumulator identify shape locations.

Hough Transform: Trade-offs

Advantages:

- Tolerant to gaps in edges and to occlusion.
- Relatively unaffected by noise.
- Detects multiple instances of the shape in a single pass.

Disadvantages:

- Expensive in memory and computation (especially for ≥ 3 parameters).
- Localisation information lost (line endpoints unknown).
- Peaks hard to locate with noisy or parallel edges.
- Sensitive to accumulator quantisation: too coarse $\Rightarrow$ different lines merge; too fine $\Rightarrow$ noise scatters votes and lines are missed.

Exam Tip: quantisation matters

Tutorial Q19 reuses the same image as Q18 but with $\theta \in \{0, 30, 60, 90, 120, 150\}$ instead of $\{0, 45, 90, 135\}$, and there is no clear peak any more. Be ready to comment that "results are sensitive to the quantisation of the parameters".

Active Contours ("Snakes")

A snake is a spline curve that deforms to minimise an energy function combining a shape (internal) term and an image (external) term. The minimum-energy curve is short, smooth, and lies along intensity discontinuities.

Energy function:

$$E_{\text{snake}} = E_{\text{internal}} + E_{\text{external}}$$

- **Internal energy**: function of the snake's *shape*. Reduced when the curve is short and smooth (penalises bending and stretching).
- **External energy**: function of *image features* near the contour. Reduced when intensity gradients along the curve are high (i.e. the curve sits on edges).

Behaviour. Initialise around an object; the snake "shrink-wraps" onto the boundary as energy is minimised.

Problems:

- Only works for shapes that are smooth and form a closed contour.
- Extremely sensitive to parameters that weight internal vs. external energy.

5.3 Worked Examples

Example 1: Role of Mid-Level Vision (Q1)

Q: Briefly describe the role of mid-level vision.

A: To group together image elements that belong together, and to segment them from all other image elements.

Example 2: Hysteresis Thresholding (Q3)

Q: Apply hysteresis thresholding to the 3×3 image I with $t_h = 0.75$, $t_l = 0.25$, 4-connected neighbours.

$$I = \begin{pmatrix} 0.4 & 0.1 & 0.4 \\ 0.2 & 0.2 & 0.7 \\ 0.3 & 0.8 & 0.9 \end{pmatrix}$$

A: Step 1 — pixels with $I > t_h$ become 1, pixels with $I < t_l$ become 0; mid-range pixels (those with $t_l \leq I \leq t_h$, including $0.4, 0.7, 0.3$) are left undecided:

$$\begin{pmatrix} 0.4 & 0 & 0.4 \\ 0 & 0 & 0.7 \\ 0.3 & 1 & 1 \end{pmatrix}$$

Step 2 — a mid-range pixel becomes 1 *only if it neighbours a pixel already classified as figure* (4-connected). The seeds (1s) are at $(3, 2)$ and $(3, 3)$. Propagating:

- 0.7 at $(2, 3)$: 4-neighbour $(3, 3) = 1 \Rightarrow$ becomes 1.
- 0.3 at $(3, 1)$: 4-neighbour $(3, 2) = 1 \Rightarrow$ becomes 1.
- 0.4 at $(1, 1)$: 4-neighbours $\{0.1, 0.2\}$ are all $0 \Rightarrow$ becomes 0.
- 0.4 at $(1, 3)$: 4-neighbours $\{0.1, 0.7\}$ — the 0.7 neighbour was just promoted to 1, but the model answer treats Step 2 as a single pass over the original mid-range set, so 0.4 at $(1, 3)$ stays 0 (neighbour $0.1 = 0$, 0.7 not yet finalised when row 1 is checked).

Final binary image:

$$I' = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Example 3: Opening vs Closing (Q4)

Q: For a 7×9 binary image: (a) draw the result of erosion then dilation with 4-connected neighbours; (b) draw the result of dilation then erosion with 8-connected neighbours.

A:

- **(a) Opening (4-conn)**: erosion first removes thin isolated foreground pixels and bridges (any FG pixel touching a BG pixel disappears). Dilation then re-grows the surviving cores. Net effect: small specks of foreground noise are removed; thicker shapes are preserved with roughly their original silhouette.
- **(b) Closing (8-conn)**: dilation first fills small holes and gaps (every BG pixel adjacent to FG — including diagonals — becomes FG). Erosion then trims the outer halo back. Net effect: foreground holes/cracks are filled; outer shape is preserved.

Exam Tip: When asked to draw, sketch the foreground silhouette

Mark which pixels switch on each pass. With 8-connectivity, dilation reaches diagonally too; opening with 4-connectivity is more aggressive than with 8-connectivity (more pixels touch the background under 4-conn).

Example 4: Region Growing on a 3×3 Image (Q6)

Q: Apply region growing with SAD-similarity threshold $T = 12$, 8-connected neighbours, seed = top-left, to the 3×3 feature-vector image:

$$\begin{matrix} (5,10,15) & (10,15,30) & (10,10,25) \\ (10,10,15) & (5,20,15) & (10,5,30) \\ (5,5,15) & (30,10,5) & (30,10,10) \end{matrix}$$

A: Use SAD between feature vectors. Starting from the top-left (label 1):

- SAD(top-left, mid-left) = 5, SAD(top-left, top-mid) = 25, SAD(top-left, mid-mid) = 10. Add mid-left (5) and mid-mid (10) to region 1.
- Grow from mid-left: SAD(mid-left, bot-left) = 10 ⇒ add bot-left to region 1.
- No further growth from region 1 (all remaining neighbours have SAD ≥ 25). Pick a new seed (top-mid, label 2).
- Grow from top-mid: SAD(top-mid, top-right) = 10 ⇒ add. From top-right SAD(top-right, mid-right) = 10 ⇒ add. Region 2 cannot grow further.
- Pick last seed bot-right (label 3): SAD(bot-right, bot-mid) = 5 ⇒ add bot-mid to region 3.

Final labels:

$$\begin{pmatrix} 1 & 2 & 2 \\ 1 & 1 & 2 \\ 1 & 3 & 3 \end{pmatrix}$$

Example 5: k-means on a 3×3 Image (Q12)

Q: Apply k-means with SAD distance, $k = 2$, initial centres $c_1 = (5,10,15)$, $c_2 = (10,10,25)$ to the same 3×3 feature-vector image as Example 4.

A: Iteration 1. Compute SAD distance from each pixel to each centre:

- Distances to $c_1 = (5,10,15)$: $\begin{pmatrix} 0 & 25 & 15 \\ 5 & 10 & 25 \\ 5 & 35 & 30 \end{pmatrix}$
- Distances to $c_2 = (10,10,25)$: $\begin{pmatrix} 15 & 10 & 0 \\ 10 & 25 & 10 \\ 20 & 40 & 35 \end{pmatrix}$

Allocate to closest centre: $\begin{pmatrix} 1 & 2 & 2 \\ 1 & 1 & 2 \\ 1 & 1 & 1 \end{pmatrix}$.

Recompute centres:

$$c_1 = \tfrac{1}{6}\big[(5,10,15)+(10,10,15)+(5,20,15)+(5,5,15)+(30,10,5)+(30,10,10)\big] = (14.17, 10.83, 12.50)$$

$$c_2 = \tfrac{1}{3}\big[(10,15,30)+(10,10,25)+(10,5,30)\big] = (10, 10, 28.33)$$

Iteration 2. Recompute distances; allocations are unchanged ⇒ centres unchanged ⇒ algorithm has converged.

Example 6: Linkage Methods in Agglomerative Clustering (Q14)

Q: Briefly describe (a) single-link, (b) complete-link, (c) group-average, (d) centroid linkage.

A:

- **Single-link**: distance between clusters is the shortest distance between any pair of elements (MIN).
- **Complete-link**: longest distance between any pair of elements (MAX).
- **Group-average**: average distance over all pairs of elements (AVERAGE).
- **Centroid**: distance between the cluster centroids (means of feature vectors).

Different linkages can produce dramatically different dendrograms on the same data.

Example 7: Hough Transform for a 4×3 Binary Image (Q18)

Q: The image has foreground pixels at $(0,0), (1,1), (2,2), (3,0)$ (using top-left origin, y down). Compute the Hough accumulator for $\theta \in \{0, 45, 90, 135\}$ deg, r rounded to nearest integer.

A: Recall $r = y\cos\theta - x\sin\theta$. Maximum $|r| \leq \sqrt{3^2 + 2^2} \approx 3.6$, so $r \in [-4, 4]$.

For each pixel, compute the four (θ, r) pairs:

Pixel	$\theta = 0$	$\theta = 45$	$\theta = 90$	$\theta = 135$
$(0,0)$	$r = 0$	$r = 0$	$r = 0$	$r = 0$
$(1,1)$	$r = 1$	$r = 0$	$r = -1$	$r \approx -1$
$(2,2)$	$r = 2$	$r = 0$	$r = -2$	$r \approx -3$
$(3,0)$	$r = 0$	$r \approx -2$	$r = -3$	$r \approx -2$

The maximum accumulator entry is 3 at $(\theta, r) = (45, 0)$, contributed by the three diagonal pixels $(0,0), (1,1), (2,2)$. This is the line of three collinear edge pixels.

Follow-up (Q19). Repeat with $\theta \in \{0, 30, 60, 90, 120, 150\}$: *no clear peak emerges*, illustrating the algorithm's sensitivity to angle quantisation.

Example 8: Active Contour Energy (Q20)

Q: Briefly describe the two terms that form the energy function that an active contour attempts to minimise.

A: $E_{\text{snake}} = E_{\text{internal}} + E_{\text{external}}$.

- **Internal energy** is a function of the shape of the contour; it is reduced if the curve is short and smooth (i.e. penalises bending and stretching).
- **External energy** is a function of the image features near the contour; it is reduced if the intensity gradient along the contour is high (i.e. the curve lies on strong edges).

5.4 Summary

Key Takeaways

1. **Mid-level vision = grouping**. Choose a feature, define similarity (SAD/SSD/Euclidean) or affinity, partition feature space.

2. **Thresholding** segments a 1-D feature space. Use the histogram valley; for noisy boundaries use ***hysteresis*** with two thresholds; clean up with morphology (open removes specks, close fills holes).
3. **Region-based** methods use spatial information. Region growing (seed-driven), region merging (bottom-up, order-dependent), split-and-merge (top-down then bottom-up via a quadtree).
4. **k-means** is partitional clustering: needs k, sensitive to initialisation, struggles with non-globular or unequal-size clusters.
5. **Agglomerative hierarchical clustering** merges nearest clusters; linkage choice (single/complete/average/centro shapes the dendrogram.
6. **Graph cuts / Normalised Cuts** treat segmentation as graph partitioning. Min-cut is biased to small subgraphs; Ncut normalises by association. Optimum is NP-hard.
7. **Hough transform** fits parametric shapes by voting; tolerant to gaps and occlusion but expensive and sensitive to quantisation. Use $r = y\cos\theta - x\sin\theta$.
8. **Snakes** minimise internal (smoothness) plus external (image-gradient) energy; only suitable for smooth closed boundaries.

Comparison of segmentation methods:

Method	Feature Space	Spatial Info?	Main Pitfall
Thresholding	1-D (intensity / filter output)	No	Single threshold rarely works globally
Hysteresis thresholding	1-D + adjacency	Yes (adjacency)	Choice of t_l, t_h
Region growing	multi-D + adjacency	Yes	Leaks through weak boundaries
Region merging	multi-D + adjacency	Yes	Order-dependent results
Split-and-merge	multi-D + quadtree adjacency	Yes	Quadtree artefacts; padding needed
k-means	multi-D	Optional (add position)	Init-dependent; needs k; non-globular fails
Agglomerative	multi-D	Optional	Linkage choice changes results; $O(n^2)$ memory
Graph cuts (min-cut)	graph in feature space	Yes (via edges)	Bias toward small subgraphs
Normalised Cuts	graph in feature space	Yes	NP-hard exact; equal-size bias; texture issues
Hough transform	parameter space (vote)	Yes	Memory cost; quantisation sensitivity
Active contours (snakes)	curve over image	Yes (along curve)	Smooth closed shapes only; parameter-sensitive

Exam Tip: Pseudo-code is a recurring marks-grab

Many tutorial questions ask you to write pseudo-code for an algorithm (region growing, region merging, split-and-merge, k-means, agglomerative, Ncuts, Hough). Memorise the canonical step lists in this chapter.

6 Week 6: Feature Matching

6.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- State the ***correspondence problem*** and the scenarios in which it arises.
- Compare ***correlation-based*** and ***feature-based*** matching: methodology, similarity measures, advantages, disadvantages.
- Compute SSD, SAD, normalised cross-correlation and the correlation coefficient by hand.
- Describe the ***Harris corner detector*** pipeline and compute its response R from I_x, I_y.
- Explain why Harris is not scale-invariant; describe ***Harris-Laplacian*** and the ***SIFT*** pipeline (DoG scale-space, keypoint localisation, orientation assignment, descriptor).
- Apply ***RANSAC*** to reject outliers and recover the underlying geometric transformation.

Big Picture. The ***correspondence problem*** is the unifying mid-level vision task of finding matching image elements across multiple images. It underpins stereo (different cameras), motion/tracking (same camera at different times) and object recognition (test image vs. training image). All algorithms answer four questions: *which* elements to match, *where* to search, *what* similarity measure to use, and *how* to discard wrong matches. **Correlation-based** methods slide a window and compare raw intensities (dense, simple, but illumination-/viewpoint-fragile and slow). **Feature-based** methods first detect a sparse set of distinctive interest points (corners or scale-space blobs), build invariant ***descriptors*** around each, and match descriptors with a vector distance (sparse, fast, far more robust). Whichever route is taken, the resulting *putative* matches contain outliers, so geometric verification with ***RANSAC*** is required to recover the true transformation between views.

What Examiners Like to Ask:

- Define the correspondence problem; give three scenarios where it arises.
- Contrast correlation- and feature-based methods (methodology + pros/cons).
- Define "detector" and "descriptor".
- Compute SAD/SSD between feature vectors and pick best matches.
- Calculate the Harris response $R = \det(H) - k\,(\mathrm{tr}(H))^2$ from given I_x, I_y over a 3×3 window.
- Interpret values of R ($R \approx 0$, $R < 0$, $R > 0$).
- Write pseudo-code for RANSAC; apply it to a translation model with given samples and threshold t.
- Explain what makes SIFT scale-, rotation-, illumination-invariant (and what Harris is/is not invariant to).

6.2 Core Concepts

Roadmap:

1. The correspondence problem (sources, requirements, problems, two solution classes).
2. Correlation-based methods (window, search area, similarity measures).
3. Feature-based methods: interest-point detectors (Harris, Harris-Laplacian, SIFT/DoG).
4. Feature descriptors (Harris window, SIFT 128-d vector) and their invariance.
5. The matching problem: putative matches and outlier rejection with RANSAC.

6.2.1 The Correspondence Problem

Correspondence Problem

For each selected element in one image, find the corresponding element (i.e. the projection of the same 3D scene point) in the other image. It is fundamentally a *search* problem.

Three scenarios in which it arises:

- **Stereo** (multiple cameras at the same time) ⇒ recover 3D structure from disparity.
- **Video / motion tracking** (same camera at different times) ⇒ estimate camera/object motion.
- **Object recognition** (current image vs. stored training images) ⇒ judge similarity / identity.

Three design decisions for any algorithm:

1. *Which* elements to match (intensities, edges, corners, descriptors).
2. *How* to search for matches (exhaustive, restricted by task knowledge).
3. *How* to compare elements (similarity measure / distance metric).

Basic requirements: most scene points must be visible in both images, and corresponding regions must look "similar".

Why correspondence is hard:

Problem	Cause
Occlusion	Self-occlusion, occlusion by other objects, or out-of-shot ⇒ no match exists.
False matches	Many regions look similar; only one is the true match.
Changing appearance	Lighting changes intensities; viewpoint changes size/shape (foreshortening).
Large search space	Each element has many candidate matches in the other image.

Methods often add task-specific assumptions (e.g. epipolar geometry in stereo) to constrain the search.

Two classes of solutions:

Class	Starts from	Compares using
Correlation-based	Raw image intensities (a window per pixel)	Similarity measure on intensity values (NCC, SSD, SAD)
Feature-based	Sparse interest points + descriptors	Distance between descriptor vectors

---

6.2.2 Correlation-Based Methods

Algorithm (template matching by sliding window):

1. For each window I_1 in image one (the template),
2. For each window I_2 of the same size in a search area of image two,
3. Compute a similarity between I_1 and I_2.

4. The corresponding point is the centre of the window in image two with the highest similarity (or lowest difference).

Three design choices:

1. **Window size.**
   - Too small: insufficient structure, noise-sensitive, many false matches.
   - Too large: imprecise (blurs the correspondence map), less tolerant to viewpoint changes.
2. **Search-area size.** Full correlation over the whole image is expensive; usually constrained around the source pixel or by task knowledge (e.g. epipolar lines in stereo).
3. **Similarity measure** (next).

Similarity measures. Let I_1, I_2 be the two windows (vectors over pixel positions (i, j)).

Maximise these (similarity)

$$\begin{aligned}
\text{Cross-correlation:} \quad & \sum_{i,j} I_1(i,j)\, I_2(i,j) \;=\; I_1 \cdot I_2 \;=\; \|I_1\|\,\|I_2\| \cos\theta \\
\text{Normalised cross-correlation (NCC):} \quad & \frac{\sum_{i,j} I_1(i,j)\, I_2(i,j)}{\sqrt{\sum I_1^2}\,\sqrt{\sum I_2^2}} \;=\; \frac{I_1 \cdot I_2}{\|I_1\|\,\|I_2\|} \;=\; \cos\theta \\
\text{Correlation coefficient:} \quad & \frac{\sum (I_1 - \bar{I_1})(I_2 - \bar{I_2})}{\sqrt{\sum (I_1 - \bar{I_1})^2}\,\sqrt{\sum (I_2 - \bar{I_2})^2}}
\end{aligned}$$

Minimise these (dissimilarity)

$$\begin{aligned}
\text{Sum of Squared Differences (SSD):} \quad & \sum_{i,j} \big(I_1(i,j) - I_2(i,j)\big)^2 \\
\text{Euclidean distance:} \quad & \sqrt{\textstyle\sum_{i,j} \big(I_1(i,j) - I_2(i,j)\big)^2} \;=\; \sqrt{\text{SSD}} \\
\text{Sum of Absolute Differences (SAD):} \quad & \sum_{i,j} |I_1(i,j) - I_2(i,j)|
\end{aligned}$$

Properties (exam-relevant):

- Cross-correlation is the ***dot product*** of I_1, I_2 viewed as vectors; NCC is the ***cosine*** of the angle between them.
- NCC equals the correlation coefficient when both windows have zero mean.
- SSD= Euclidean distance squared.
- SAD is the cheapest and most popular in practice; the extra cost of NCC/SSD is usually not justified by performance.

Spot it in a question

"Maximise" similarity ⇒ cross-correlation/NCC. "Minimise" difference ⇒ SSD or SAD. "Cosine of the angle between intensity vectors" ⇒ NCC.

Performance.

Aspect	Detail
[+] Implementation	Easy to implement (sliding-window inner products).
[+] Density	Provides a *dense* correspondence map (every pixel).
[–] Cost	Computationally expensive (correlation at every pixel).
[–] Texture	Needs lots of distinct intensity patterns; constant/repetitive regions ⇒ many false matches.
[–] Viewpoint	Fails when viewpoints differ (illumination changes intensities; foreshortening changes window size/shape).

6.2.3 Feature-Based Methods: Detector + Descriptor

Pipeline:

1. Detect ***interest points*** in both images (ideally the *same* 3D points, despite scale/rotation/illumination changes).
2. Build a ***descriptor*** for each interest point.
3. For each interest point in image one, find the interest point in image two that maximises the similarity (minimises the descriptor distance).

Detector vs. Descriptor

- **Detector**: the method that locates image features / interest points suitable for matching.
- **Descriptor**: an array of feature values associated with each interest point. Descriptors are compared (by distance) to decide which points match.

Two requirements:

- **Repeatable detector**: detects the same 3D point in both images, despite scale, rotation, translation, illumination changes.
- **Distinctive descriptor**: complex enough to disambiguate corresponding points with high probability.

Performance summary.

Aspect	Detail
[+] Robustness	Relatively insensitive to illumination and size changes (with good descriptors).
[+] Cost	Cheaper than correlation-based (only matches at selected locations).
[–] Density	Sparse correspondence map; intermediate locations need interpolation.
[–] Texture	Only works where "good" interest points exist; fails on textured/random regions.
[–] Detector choice	Choice of interest points strongly affects results.

What makes a *good* interest point?

- **Easy to localise**: the response is a sharp peak (think corner), not a flat plateau or a ridge.
- **High information content**: the local neighbourhood is distinctive.

A flat region has little information; an edge has high information *across* the edge but is hard to localise *along* the edge; a corner has both.

6.2.4 Corner Detection: Harris

Why corners? Around a corner, the intensity gradient is large in *two* directions. This makes corners easy to localise and information-rich.

Region type	Gradient pattern
Flat	No gradient in any direction.
Edge	Strong gradient in one direction (across edge); none along the edge.
Corner	Strong gradients in two (orthogonal) directions.

Harris matrix H (a.k.a. second-moment / structure tensor)

Compute I_x, I_y by convolving I with derivative-of-Gaussian masks. Sum products of derivatives over a small (Gaussian or equally weighted) window:

$$H = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix}.$$

The eigenvalues λ_1, λ_2 of H are the maximum slope of the intensity gradient along two orthogonal directions.

Region	Eigenvalues	Interpretation
Flat	λ_1, λ_2 both small	No structure.
Edge	$\lambda_2 \gg \lambda_1$ (or vice versa)	One dominant direction.
Corner	λ_1, λ_2 both large, $\lambda_1/\lambda_2 \approx 1$	Two strong directions.

Naive criterion: corner $\Leftrightarrow \min(\lambda_1, \lambda_2) >$ threshold. But computing eigenvalues for every pixel is expensive.

Harris response R

Avoids eigen-decomposition by using $\det H$ and $\operatorname{tr} H$:

$$R = \det(H) - k\left(\operatorname{tr}(H)\right)^2 = \left(\sum I_x^2 \sum I_y^2 - (\sum I_x I_y)^2\right) - k\left(\sum I_x^2 + \sum I_y^2\right)^2,$$

with $k \approx 0.04$–0.06 (empirical).

Value of R	Region type
$\lvert R\rvert$ small / $R \approx 0$	Flat (intensity unchanging).
$R < 0$ (large magnitude)	Edge.
$R > 0$ (large)	Corner.

Algorithm:

1. Compute I_x, I_y via derivative-of-Gaussian masks.
2. For each pixel form H by summing $I_x^2,\ I_y^2,\ I_x I_y$ over the chosen window.
3. Compute R.
4. Threshold: keep pixels with $R >$ threshold.
5. Apply ***non-maximum suppression***: if any neighbour has a larger R, set this pixel's R to 0.
6. Surviving local maxima are the interest points.

Spot it in a question

"Avoids calculating eigenvalues" $\Rightarrow$ Harris R. "Why non-maximum suppression?" $\Rightarrow$ to keep only the strongest corner per neighbourhood and avoid duplicate detections (NMS is a generic CV trick—also used in Canny, object detection, etc.).

Invariance of the Harris detector:

Transformation	Invariant?	Why
Translation	Yes	H depends only on relative gradients.
Rotation	Yes	Eigenvalues (and hence R) are unchanged; eigenvectors rotate.
Illumination	Partial	Affected by overall intensity scaling/offsets.
Viewpoint	Partial	Small viewpoint changes only.
Scale	**No**	Points classed as edges at fine scale become corners at coarse scale.

6.2.5 Scale-Invariant Detectors: Harris-Laplacian and SIFT

Idea. To get scale invariance, run the detector across an ***image pyramid*** and find local maxima in *both* space and scale.

Detector	Spatial response	Scale selection
Harris	Harris R	Single fixed scale (no scale invariance).
Harris-Laplacian	Harris R over scale-space	Local maxima of LoG over scale.
SIFT	DoG response over scale-space	Local extrema of DoG in $3 \times 3 \times 3$ scale-space neighbourhood.

SIFT detector pipeline (Scale Invariant Feature Transform):

1. **Build a Laplacian/DoG pyramid**: convolve image with DoG masks at several resolutions (a Laplacian image pyramid).
2. **Detect scale-space extrema**: for each candidate pixel, check if its DoG response is larger or smaller than *all 26 neighbours* in the $3 \times 3 \times 3$ neighbourhood (8 same-scale + 9 above + 9 below).
3. **Keypoint localisation / filtering**:

- **Contrast threshold**: discard low-contrast points (e.g. from 832 → 729 candidates).
- **Edge response test**: discard points lying on edges (poor localisation along edge). Use the ratio
$$\frac{(\operatorname{tr} H)^2}{\det H} < \frac{(r+1)^2}{r}, \qquad r \approx 10,$$
which is large when one eigenvalue dominates (an edge). Same idea as Harris but expressed via the trace/det ratio.

The output is a set of keypoints, each with a (x, y) location *and* a scale.

Common confusion

A SIFT keypoint is *not* a single pixel; it is a small region whose size is given by the detected scale.

6.2.6 Feature Descriptors

A descriptor turns a neighbourhood into a vector that other points can be compared against.

Harris descriptor (simplest):

- Just a small window of pixel intensities around the interest point.
- Similarity: Euclidean / SSD / SAD between intensity vectors.
- **Robust to translation only**—not to rotation, scale, illumination or viewpoint.

SIFT descriptor (the workhorse): a 128-D vector built from gradient orientations.

SIFT descriptor: three steps

Work in the Gaussian-smoothed image *at the scale of the keypoint*.

1. **Gradient field**: for every pixel surrounding the keypoint, compute approximate gradient magnitude and orientation via pixel differences:
$$\text{mag} = \sqrt{(I_{x_1} - I_{x_2})^2 + (I_{y_1} - I_{y_2})^2}, \quad \text{ori} = \tan^{-1} \frac{I_{y_1} - I_{y_2}}{I_{x_1} - I_{x_2}}.$$
2. **Dominant orientation**: build a histogram of orientations around the keypoint, weighting each sample by gradient magnitude × a Gaussian centred on the keypoint. Take the peak as the dominant orientation; *rotate* all gradients so the dominant orientation points up. (This is what gives rotation invariance.)
3. **Spatial histograms**: split the keypoint neighbourhood into 4×4 sub-windows. In each sub-window, build an 8-bin orientation histogram (samples again weighted by magnitude and a Gaussian on the keypoint). Concatenate $\Rightarrow 4 \times 4 \times 8 = 128$-D vector. Normalise to unit length (illumination invariance).

Matching: Euclidean distance between 128-D descriptors.

Invariance summary:

Descriptor	Translation	Rotation	Scale	Illumination	Viewpoint
Harris (intensity window)	yes	no	no	no	no
SIFT (128-D)	yes	yes	yes	yes	partial

Why each step matters

The *scale* of the keypoint ⇒ scale invariance. Step 2 (rotate to dominant orientation) ⇒ rotation invariance. Step 3's normalisation ⇒ illumination invariance. The Gaussian weighting de-emphasises far-from-centre samples (more robust to small localisation errors).

6.2.7 The Matching Problem: Outliers and RANSAC

Putative matches. Whatever method is used (correlation- or feature-based), the search returns the most-similar candidates; some are correct (***inliers***) and others wrong (***outliers***).

Goal. Recover the underlying geometric transformation between images despite outliers.

High-level pipeline:

1. (Optional) Extract features (if feature-based).
2. Compute putative matches (best-similarity pairing).
3. Find the most likely transformation (*most inliers, fewest outliers*) using ***RANSAC***.

RANSAC: RANdom SAmpling and Consensus

A robust model-fitting procedure for data containing outliers. Requires (i) data = inliers + outliers and (ii) a parametric model that explains the inliers.

RANSAC procedure

1. Randomly choose a *minimal* subset (sample) of data points needed to fit the model.
2. Fit the model to that subset.
3. Test all the other data points; mark those within distance t of the model's prediction as ***inliers***.
4. Count inliers: that count is the model's ***support*** (the size of the ***consensus set***).
5. Repeat from step 1 for N trials.

After N trials, pick the model with the largest support and *re-estimate* the model using *all* points in that consensus set.

Sample size by model:

Model	Min. sample size	Use case
2-D translation	1 pair	Pure $(\Delta x, \Delta y)$ between views.
2-D similarity (translation + rotation + scale)	2 pairs	Planar object, rotated camera.
Affine	3 pairs	Far-away planar surfaces.
Homography	4 pairs	General planar scenes / perspective.
Line in 2-D	2 points	Geometric fitting / segmentation.

Pros and cons:

Aspect	Detail
[+]	Simple and effective; general for many fitting problems (segmentation, stereo geometry, motion).
[+]	Tolerates a high outlier ratio.
[–]	Many iterations needed if outlier fraction is large.
[–]	Several parameters to tune (N trials, threshold t, sample size, success criterion).

Spot it in a question

RANSAC questions almost always boil down to: (1) write the five-step procedure, (2) given a list of putative matches and a sample, compute the model, predict each non-sample match, and *count inliers within t*. The model with the largest consensus wins.

6.2.8 Putting It All Together

The course's five-question summary of the correspondence pipeline:

1. **Which image locations to match**: all (correlation) or selected interest points (Harris, SIFT).
2. **What properties to match**: image intensities or a descriptor (SIFT 128-D).
3. **Where to look**: exhaustive search or restricted (e.g. epipolar lines).
4. **How to evaluate matches**: similarity (NCC, correlation coefficient) or distance (SSD, Euclidean, SAD).
5. **How to find the true correspondence (eliminate false matches)**: **RANSAC**.

Correspondence vs. grouping (Week 5). Both look for similar elements, but:

- **Grouping**: similar elements *within a single image*.
- **Correspondence**: the *same* elements *across multiple images*.

6.3 Worked Examples

Example 1: Definition + scenarios

Q: Describe the correspondence problem and three scenarios where it arises.

A: It is the problem of finding the same 3D location in two or more images. It arises:

1. Multiple cameras (stereo) of one scene from different locations ⇒ recover 3D structure.
2. One camera at different times (video/motion) ⇒ estimate camera/object motion.
3. Comparing an image to stored images ⇒ measure similarity for object recognition.

Example 2: Correlation-based vs. feature-based methodology

Q: Briefly describe both methodologies.

A:

- **Correlation-based**: establish a correspondence *for every pixel* by matching image intensities in a window around each pixel.

- **Feature-based**: establish a correspondence at a *sparse* set of locations (usually corners), using a feature descriptor extracted from around each.

Example 3: Detector vs. descriptor

Q: For feature-based methods, what is a "detector" and a "descriptor"?

A:

- **Detector**: the method that locates image features / interest points suitable for matching.
- **Descriptor**: an array of feature values associated with each interest point. Descriptors are compared (by distance) to decide which points match.

Example 4: SAD matching by hand

Q: Five interest points in the left and right images have feature vectors:

L	**Feat.**	**R**	**Feat.**
L_1	$(10, 4)$	R_1	$(3, 7)$
L_2	$(3, 8)$	R_2	$(1, 1)$
L_3	$(0, 2)$	R_3	$(5, 7)$
L_4	$(6, 9)$	R_4	$(8, 0)$
L_5	$(9, 1)$	R_5	$(1, 2)$

For each L_i, find the best match by SAD.

A: $\text{SAD}(a, b) = |a_1 - b_1| + |a_2 - b_2|$. Compute each row and pick the minimum:

- L_1: R_1=10, R_2=12, R_3=8, R_4=**6**, R_5=11 $\Rightarrow R_4$.
- L_2: R_1=**1**, R_2=9, R_3=3, R_4=13, R_5=8 $\Rightarrow R_1$.
- L_3: R_1=8, R_2=2, R_3=10, R_4=10, R_5=**1** $\Rightarrow R_5$.
- L_4: R_1=5, R_2=13, R_3=**3**, R_4=11, R_5=12 $\Rightarrow R_3$.
- L_5: R_1=12, R_2=8, R_3=10, R_4=**2**, R_5=9 $\Rightarrow R_4$.

Note L_1 and L_5 both claim R_4: at most one is correct—this is what RANSAC will sort out.

Example 5: Disparity = right → left translation

Q: Coordinates for the matched pairs from Example 4 are:

L	**Coord.**	**R**	**Coord.**
L_1	$(187, 168)$	R_1	$(101, 394)$
L_2	$(203, 290)$	R_2	$(115, 186)$
L_3	$(215, 87)$	R_3	$(135, 128)$
L_4	$(234, 28)$	R_4	$(269, 243)$
L_5	$(366, 142)$	R_5	$(336, 178)$

With coplanar image planes, disparity equals the right-to-left translation $L - R$. Compute it.

A:

- $L_1 - R_4 = (187, 168) - (269, 243) = (-82, -75)$.
- $L_2 - R_1 = (203, 290) - (101, 394) = (102, -104)$.
- $L_3 - R_5 = (215, 87) - (336, 178) = (-121, -91)$.
- $L_4 - R_3 = (234, 28) - (135, 128) = (99, -100)$.

- $L_5 - R_4 = (366, 142) - (269, 243) = (97, -101)$.

Inspection: the disparities cluster around (≈ 100, ≈ -100); the two outliers are $L_1 - R_4$ and $L_3 - R_5$. RANSAC will recover the cluster.

Example 6: RANSAC pseudo-code

Q: Write pseudo-code for RANSAC.

A:

1. Randomly choose a minimal subset (sample) of data points necessary to fit the model.
2. Fit the model to this subset.
3. Test all other data points: a point is an inlier iff it lies within distance t of the model's prediction.
4. Count inliers $\Rightarrow$ consensus set; size = model's support.
5. Repeat from step 1 for N trials.

After N trials, select the model with the highest support and re-estimate it using *all* points in that consensus set.

Example 7: RANSAC by hand on the matches above

Q: Apply RANSAC to the matches in Example 5. Model = pure 2-D translation, $t = 20$ pixels, three trials with samples L_1, L_2, L_3 in order (not random). Which trial wins?

A: Sample size = 1 (one match defines the translation).

Trial 1: take $L_1 - R_4 = (-82, -75)$. Predict each other left point as $L_i - (-82, -75)$ and compare to its actual match.

- L_2: predicted $(285, 365)$, actual $R_1 = (101, 394)$. Distance large $\Rightarrow$ outlier.
- L_3: predicted $(297, 162)$, actual $R_5 = (336, 178)$. Outlier.
- L_4: predicted $(316, 103)$, actual $R_3 = (135, 128)$. Outlier.
- L_5: predicted $(448, 217)$, actual $R_4 = (269, 243)$. Outlier.

Consensus set = 0.

Trial 2: take $L_2 - R_1 = (102, -104)$.

- L_1: predicted $(85, 272)$, actual $(269, 243)$. Outlier.
- L_3: predicted $(113, 191)$, actual $(336, 178)$. Outlier.
- L_4: predicted $(132, 132)$, actual $(135, 128) \Rightarrow$ distance $\approx 5 \leq 20 \Rightarrow$ **inlier**.
- L_5: predicted $(264, 246)$, actual $(269, 243) \Rightarrow$ distance $\approx 5.8 \leq 20 \Rightarrow$ **inlier**.

Consensus set = 2.

Trial 3: take $L_3 - R_5 = (-121, -91)$.

- All four predictions are far from their actual matches $\Rightarrow$ consensus = 0.

Result. Trial 2 wins. The true correspondences are the matches for L_2, L_4, L_5 (and the sample L_2). Re-estimate $(\Delta x, \Delta y)$ from those three pairs as the final model.

Example 8: Harris response interpretation

Q: For the Harris corner detector, what type of image feature gives rise to (a) $R \approx 0$, (b) $R < 0$, (c) $R > 0$?

A:

- (a) $R \approx 0$: a region where intensity is approximately unchanging (**flat**).

- (b) $R < 0$ (with large magnitude): an **edge**.
- (c) $R > 0$ (large): a **corner**.

Why: $\det H = \lambda_1\lambda_2$ and $\operatorname{tr} H = \lambda_1 + \lambda_2$. On a corner both eigenvalues are large $\Rightarrow \det H$ dominates $\Rightarrow R > 0$. On an edge one eigenvalue is large and one is small $\Rightarrow \det H$ small, $(\operatorname{tr} H)^2$ large $\Rightarrow R < 0$. On a flat region both are small $\Rightarrow R \approx 0$.

Example 9: Computing R from given derivatives

Q: Given I_x and I_y for a 5×4 image, compute R at a central pixel using $k = 0.05$ and a 3×3 equally weighted summing window. Suppose at one pixel the window gives

$$\sum I_x^2 = 4, \quad \sum I_y^2 = 4, \quad \sum I_x I_y = 0.$$

A:

$$\begin{aligned} \det H &= 4 \cdot 4 - 0^2 = 16, \\ \operatorname{tr} H &= 4 + 4 = 8, \\ R &= 16 - 0.05 \cdot 8^2 = 16 - 3.2 = 12.8. \end{aligned}$$

Interpretation: $R > 0$ (large) $\Rightarrow$ corner. (For a flat window all sums would be 0 and $R = 0$. For an edge along one axis, one of $\sum I_x^2, \sum I_y^2$ is 0, so $\det H = 0$ and $R = -k(\operatorname{tr} H)^2 < 0$.)

Recipe in any exam variant: (1) at the central pixel, list the nine values of $I_x^2, I_y^2, I_x I_y$ in the 3×3 window; (2) sum each separately; (3) plug into $R = \det H - k(\operatorname{tr} H)^2$.

Example 10: Correlation-based pros and cons

Q: Briefly describe the advantages and disadvantages of correlation-based methods.

A:

- **Advantages**: easy to implement; gives a dense correspondence map (every pixel).
- **Disadvantages**:
  - Computationally expensive (correlation at every pixel).
  - Needs lots of distinct intensity patterns; constant or repetitive regions cause many false matches.
  - Fails when viewpoints differ much: illumination changes intensity values; foreshortening changes the size, shape and appearance of corresponding regions.

Example 11: Why NCC over plain cross-correlation? (FAQ)

Q: Two "similar" images with low intensities can have lower plain cross-correlation than two "different" images with high intensities. Explain and resolve.

A: Plain cross-correlation $\sum I_1 I_2$ is the dot product, which scales with the magnitudes $\|I_1\|, \|I_2\|$ as well as with how aligned (similar) they are. Two bright but unrelated patches can therefore beat two dim but identical ones. **Normalised cross-correlation** divides by $\|I_1\|\,\|I_2\|$, yielding $\cos\theta$: it depends only on the *angle* (pattern alignment) between intensity vectors, not their magnitudes, and is therefore the right similarity for matching.

Example 12: Why non-maximum suppression in Harris? (FAQ)

Q: What is non-maximum suppression for in Harris?

A: A genuine corner produces high R across a small neighbourhood of pixels; without NMS we would record many duplicate detections per corner. NMS keeps only the local maximum (set R to 0 if any

neighbour has a larger value), so each corner contributes one interest point, and weak responses near a strong one are ignored. NMS is a generic CV operation, also used by Canny edge detection and modern object detectors.

6.4 Summary

Key Takeaways

1. **Correspondence problem**: find matching elements across images. Underlies stereo, motion and recognition. Four design questions: which / where / similarity / outliers.
2. **Correlation-based** methods: slide a window; maximise NCC / correlation coefficient or minimise SSD / Euclidean / SAD. Dense and simple, but slow and viewpoint-fragile.
3. **Feature-based** methods: detect distinctive interest points (corners, blobs); build invariant descriptors; match by descriptor distance. Sparse but cheap and robust.
4. **Harris corner detector**: build $H = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix}$; corner $\Leftrightarrow$ both eigenvalues large; use $R = \det H - k(\operatorname{tr} H)^2$ to avoid eigen-decomposition. NMS to keep one peak per corner. Translation- and rotation-invariant; *not* scale-invariant.
5. **SIFT**: scale-space DoG extrema $\rightarrow$ contrast/edge filtering $\rightarrow$ dominant orientation $\rightarrow$ $4 \times 4 \times 8 = 128$-D gradient-orientation descriptor, normalised. Translation-, rotation-, scale- and illumination-invariant.
6. **Matching $\Rightarrow$ outliers**: nearest-neighbour search returns putative matches; outliers must be rejected.
7. **RANSAC**: 1. sample minimal subset, 2. fit, 3. count inliers within t, 4. repeat N times, 5. keep model with most support and re-fit on its consensus set. Robust to a high outlier rate.
8. **Correspondence vs. grouping**: across vs. within an image.

Final comparison: matching pipelines.

Method	Locations	What is matched	Comparison	Density
Correlation	every pixel	raw intensity windows	NCC / SSD / SAD	dense
Harris + window	corners	intensity window	SSD / SAD / Euclidean	sparse
SIFT	DoG keypoints	128-D gradient histogram	Euclidean	sparse

Final comparison: detector invariance.

Detector	Translation	Rotation	Scale	Illumination	Viewpoint
Harris	yes	yes	no	partial	partial
Harris-Laplacian	yes	yes	yes	partial	partial
SIFT (DoG)	yes	yes	yes	yes	partial

Final comparison: similarity / distance measures.

Measure	**Formula sketch**	**Optimise**	**Notes**
Cross-correlation	$\sum I_1 I_2$	maximise	Sensitive to intensity magnitude.
NCC	$\frac{\sum I_1 I_2}{\lVert I_1 \rVert \lVert I_2 \rVert}$	maximise	$= \cos\theta$ between intensity vectors.
Correlation coefficient	mean-centred NCC	maximise	= NCC when both means are zero.
SSD	$\sum (I_1 - I_2)^2$	minimise	= Euclidean distance squared.
Euclidean	$\sqrt{\sum (I_1 - I_2)^2}$	minimise	Used by SIFT descriptors.
SAD	$\sum \lvert I_1 - I_2 \rvert$	minimise	Cheap; commonly used in practice.

7 Week 7: Stereo Vision and Depth

7.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Derive the disparity-to-depth relation $z = fB/d$ for coplanar cameras and reason about how depth accuracy varies with baseline B and distance z.
- State the ***stereo correspondence problem*** and the constraints used to make it tractable: epipolar, maximum disparity, continuity, uniqueness, ordering.
- Define the ***epipolar geometry*** terminology (baseline, epipole, epipolar plane, epipolar line, conjugated epipolar lines, horopter) and explain rectification.
- For non-coplanar cameras, compute disparity as $\alpha_L - \alpha_R$ relative to the horopter and use it to localise points.
- Relate ***field of view*** (FoV) to focal length / sensor size and explain the short- vs long-baseline FoV/error trade-off.
- Enumerate and briefly describe the cues to depth: binocular (disparity), oculomotor (accommodation, convergence), monocular (interposition, size familiarity, texture gradients, linear/aerial perspective, shading), motion (parallax, optic flow, accretion/deletion, structure from motion).

Big Picture. A single image loses depth: every 3D point on a line of sight projects to the same 2D pixel. ***Stereo vision*** recovers depth by triangulating the same 3D point in two images taken from different viewpoints. Two sub-problems must be solved:

1. **Correspondence**: which left-image pixel matches which right-image pixel? Search is reduced from 2D to 1D by the ***epipolar constraint***, and further pruned by maximum disparity, continuity, uniqueness and ordering. None of these constraints is universally true – each one fails in characteristic situations (occlusion, slanted surfaces, depth discontinuities).
2. **Reconstruction**: given a correspondence, compute depth z from disparity d. For coplanar (rectified) cameras $z = fB/d$; for non-coplanar (verging) cameras disparity is an angular quantity $\alpha_L - \alpha_R$ measured against a zero-disparity curve called the ***horopter***.

Stereo is only one cue. The visual system fuses many: **binocular** disparity, **oculomotor** (accommodation, convergence), **monocular** pictorial cues, and **motion**-induced cues. Computer-vision systems use the same taxonomy and add **active** sensing (LIDAR, structured light, time-of-flight).

What Examiners Like to Ask:

- Derive $z = fB/d$ from two coplanar pinhole cameras (similar triangles; left vs right projection).
- "Comment on how depth accuracy changes with (a) distance, (b) baseline."
- "Explain the epipolar constraint in one sentence." Then list the other constraints and when each one fails.
- Numerical horopter question: given B, vergence θ, and angles α_L, α_R, find the distance from horopter (Tutorial Q5 pattern).
- "Why are at least two cameras needed to recover 3D?" (DOF counting: 3 unknowns, only 2 image equations per camera.)
- "Describe two oculomotor / four monocular / two motion-induced cues to depth."
- "Why is depth important for object recognition?" (segmentation between objects + shape within an object).
- Code-style: implement a window-based SAD disparity map for rectified images (LGT exercise).

7.2 Core Concepts

Roadmap:

1. Image formation recap and why one camera cannot recover depth.
2. Coplanar (simple) stereo: disparity, $z = fB/d$, accuracy.
3. Stereo correspondence: requirements and the five constraints.
4. Field of view: focal length, sensor, common FoV, baseline trade-off.
5. Non-coplanar (verging) stereo: horopter, angular disparity.
6. Epipolar geometry: baseline, epipoles, epipolar plane/lines, rectification.
7. Cues to depth: binocular, oculomotor, monocular, motion.

7.2.1 Why Stereo? Image Formation Recap

A pinhole camera projects a 3D scene point $P = (x, y, z)$ onto the image plane (assume the principal point is at the origin) by the similar-triangle relation

$$P' = (x', y') = \left(\frac{fx}{z}, \frac{fy}{z}\right).$$

Depth ambiguity. All 3D points on a single line of sight produce the same image pixel: (x, y, z) and $(\lambda x, \lambda y, \lambda z)$ project identically. Imaging therefore loses the depth dimension. A second viewpoint with known geometry breaks this ambiguity by triangulation.

Why two cameras? For one camera the projection equations give two scalar constraints on three unknowns (x, y, z) – under-determined. A second camera adds two more equations, sufficient to solve for (x, y, z). (DLT formalises this with 11 parameters per camera: 6 extrinsic + 5 intrinsic.)

Applications. Path planning and collision avoidance (cars, robots), grasping, virtual advertising, 3D reconstruction (CAD models, VR), and as a pre-processing cue for segmentation and object recognition.

7.2.2 Coplanar Cameras (Simplest Case)

Coplanar Stereo Setup

A pair of cameras with: (i) coplanar image planes, (ii) equal focal lengths f, (iii) optical centres at the same height (collinear x-axes), (iv) parallel z-axes (optical axes meet at infinity). Separation between optical centres is the ***baseline*** B.

Image formation for the pair. Working in the left camera's coordinate frame, a 3D point $P = (x, y, z)$ projects to

$$(x'_L, y'_L) = \left(\frac{fx}{z}, \frac{fy}{z}\right), \qquad (x'_R, y'_R) = \left(\frac{f(x-B)}{z}, \frac{fy}{z}\right).$$

Because the x-axes are collinear, $y'_L = y'_R$.

Disparity (coplanar)

The horizontal difference between the projections of the same 3D point in the two images:

$$d \;=\; x'_L - x'_R \;=\; \frac{fx}{z} - \frac{f(x-B)}{z} \;=\; \frac{fB}{z}.$$

For coplanar cameras disparity is purely horizontal, measured in pixels, and may be positive or negative.

Disparity-to-Depth (coplanar)

$$\boxed{z = \frac{fB}{d}}$$

Depth is inversely proportional to disparity. Given baseline B and focal length f, a measured disparity gives absolute depth; without B, only relative depths from relative disparities.

Disparity map / depth map.

- A pair of stereo images defines a ***field*** of disparity vectors – one per pixel (a *disparity map*).
- Per-pixel disparity is a 2D vector in general; for coplanar cameras only the x component is non-zero.
- Converting disparity to depth produces a *depth map* (commonly: light = close, dark = far).
- False matches produce false-shallow / false-deep regions; texture-poor surfaces are notoriously hard.

Accuracy of depth measurement.

Change	**Effect on** $d = fB/z$	**Accuracy of** z
z decreases (point closer)	d increases	Better (small Δd matters less)
z increases (point farther)	$d \to 0$	Worse (small errors swing z a lot)
B increases (longer baseline)	d increases	Better depth resolution
B decreases (shorter baseline)	d decreases	Worse depth resolution

Spot it in a question

"Comment on accuracy with changing distance / baseline" $\Rightarrow$ argue from $d = fB/z$: larger disparity $\Rightarrow$ small measurement error in d has less impact on z $\Rightarrow$ accuracy improves as object gets closer or baseline gets longer.

7.2.3 The Correspondence Problem

To measure disparity we must *find corresponding points* – pairs of pixels that are projections of the same 3D point.

Two solution families:

Method	**Idea**	**Output**
Correlation-based (window)	Match a fixed window around every pixel by SAD/SSD/NCC	Dense disparity map (a value at every pixel)
Feature-based	Match interest points / descriptors only	Sparse disparity map (values at features)

Basic requirements.

1. Most scene points are visible in both images (no large occlusions).
2. Corresponding image regions "look similar" (similar intensities/textures/descriptors).

These hold approximately when scene depth $z \gg B$ (object far compared with baseline).

Why it is hard. Correspondence is fundamentally ambiguous: for any left-image patch there are usually many right-image patches that are locally similar (repetitive texture, low texture, specularities). Constraints prune the search.

Constraints on Stereo Correspondence

1. **Epipolar.** A point in the left image must lie on the corresponding *epipolar line* in the right image. *For coplanar cameras the epipolar lines are horizontal scan lines*, so $y'_L = y'_R$ and 2D search collapses to 1D.
2. **Maximum disparity.** Limit the search range: for points at distance $\geq z_{\min}$, $|d| \leq d_{\max} = fB/z_{\min}$. *Fails* for points closer than $z_{\min}$.
3. **Continuity (smoothness).** Neighbouring pixels usually have similar disparities (surfaces are smooth in depth). *Fails at depth discontinuities* (object/object or object/background boundaries).
4. **Uniqueness.** Each pixel in one image matches at most one pixel in the other. *Fails* when a slanted surface projects to n pixels in one image and $m \neq n$ in the other, or when one camera's line of sight grazes a surface so one location matches many.
5. **Ordering.** Matching points along conjugated epipolar lines appear in the same order. *Fails* when objects are at very different depths (a near object can swap order with a far one across views).

Bottom line. "We use *imperfect* constraints to narrow many possible solutions down to (hopefully) the correct one."

Window matching (correlation-based). For each left pixel (x, y):

1. Extract a template window of size $w \times w$ around (x, y).
2. For each candidate disparity $d \in [0, d_{\max}]$, extract the window around $(x - d, y)$ in the right image.
3. Compute a similarity / cost (SAD, SSD, NCC).
4. Pick the d minimising cost (or maximising similarity).

Common cost functions:

Cost	Formula (window W)	Notes
SAD	$\sum_{(u,v)\in W} \lvert I_L(u,v) - I_R(u-d,v)\rvert$	Cheap; sensitive to brightness offsets
SSD	$\sum_{(u,v)\in W} (I_L - I_R)^2$	Penalises large differences more
NCC	normalised correlation in W	Robust to linear brightness changes

Window-size trade-off: small windows capture detail but are noisy and ambiguous on textureless regions; large windows are smoother but blur depth boundaries.

Sub-pixel refinement. The minimum of a discrete cost curve is at integer d; fitting a parabola to three neighbouring cost values $\{c(d-1), c(d), c(d+1)\}$ and taking its minimum gives a sub-pixel estimate.

7.2.4 Field of View (FoV)

Field of View

The angular extent of the scene that a camera can image. For a pinhole camera with focal length f and sensor width W (height H),

$$\mathrm{FoV}_x = 2\arctan\left(\frac{W}{2f}\right), \qquad \mathrm{FoV}_y = 2\arctan\left(\frac{H}{2f}\right).$$

Larger f (telephoto) $\Rightarrow$ *narrower* FoV, more magnification. **Smaller** f (wide-angle) $\Rightarrow$ *wider* FoV, more distortion at edges.

Common FoV (cFoV). A stereo system can only recover depth in the intersection of the two cameras' FoVs.

Baseline / FoV / depth-error trade-off (coplanar cameras).

Setup	cFoV	Depth error	Why
Short baseline	Large	Large	Same Δz produces small Δd (from $z = fB/d$)
Long baseline	Small	Small	Same Δz produces larger Δd, but cameras see less common scene

Verging cameras. Rotating the cameras inwards so optical axes intersect (rather than being parallel) increases the cFoV at a given baseline and reduces depth error – at the cost of more complex geometry (non-horizontal epipolar lines).

Panoramas. Stitching multiple narrow-FoV images together produces a wide effective FoV; this is correspondence between rotated views, not stereo.

7.2.5 Non-Coplanar (Verging) Cameras

When the optical axes converge at a ***fixation point*** the geometry generalises:

Vergence and Horopter

Vergence angle θ: the angle between the two optical axes.
Horopter: the locus of 3D points that produce *zero disparity* given the current vergence. Each line of sight from the left camera meets a corresponding right-camera ray at the horopter; equal angular offsets from the optical axes ($\alpha_L = \alpha_R$) define points on it.

Disparity for verging cameras. Disparity is measured in *angles*, not pixels:

$$d = \alpha_L - \alpha_R,$$

where α_L, α_R are the angles a 3D point's line of sight makes with the left/right optical axis.

- $d = 0$ at the fixation point and at all points on the horopter.
- $d > 0$: point lies *outside* (further than) the horopter ("uncrossed" disparity).
- $d < 0$: point lies *inside* (nearer than) the horopter ("crossed" disparity).
- Magnitude of disparity grows with distance from the horopter.

Sign convention. Be explicit about (i) the sign of each angle α relative to the optical axis and (ii) the order of subtraction; otherwise the inside/outside distinction flips.

Recovering depth from non-coplanar cameras. Two routes:

1. **Triangulation.** Determine the intersection of the corresponding view-lines using the known camera parameters.
2. **Rectification + simple case.** Warp both images so that epipolar lines become horizontal scan lines (turning the verging case into a coplanar case), then use $z = fB/d$.

Both require knowledge of intrinsic and extrinsic camera parameters (calibration).

Biological connection. Human eyes verge on a fixation point; cortical neurons tuned to specific retinal disparities (positive, zero, negative) read out depth around the horopter (see Week 4: biological vision).

7.2.6 Epipolar Geometry

Epipolar Geometry: Terminology

- **Baseline.** The line through the two camera projection centres O_L, O_R.
- **Epipole.** The image of one camera's optical centre in the other camera's image plane. Equivalently, the intersection of the baseline with each image plane. Left epipole e_L lives in the left image; right epipole e_R in the right image.
- **Epipolar plane.** For a particular 3D point P, the plane through P and both optical centres O_L, O_R.
- **Epipolar line.** The intersection of the epipolar plane with an image plane. *All epipolar lines in the left image pass through e_L; all epipolar lines in the right image pass through e_R.*
- **Conjugated epipolar lines.** The pair of epipolar lines (one in each image) generated by the same 3D point.

Epipolar Constraint

Corresponding image points must lie on conjugated epipolar lines. Hence the 2D correspondence search is reduced to a 1D search along an epipolar line. For coplanar cameras the epipoles are at infinity and the epipolar lines are horizontal scan lines (so search is along a row).

Rectification. A 2D image warp applied to both images that makes corresponding epipolar lines coincide with the rows of the rectified images. After rectification the system behaves as a coplanar pair: search left/right along the same row, $z = fB/d$ applies.

Why rectify? Faster matching (one 1D search per row), simpler post-processing, and direct application of the simple-case depth formula.

Coplanar vs general (verging) summary:

Property	Simple (coplanar)	General (verging)
Image planes	Coplanar (translation along x)	Translation + rotation
Epipolar lines	Horizontal scan lines	Lines radiating from the epipole
Disparity unit	Pixels	Angles ($\alpha_L - \alpha_R$)
Disparity vs depth	$z = fB/d$	Magnitude grows with distance from horopter
Depth recovery	Direct	Triangulation, or rectify + simple case

---

7.2.7 Cues to Depth

Stereo disparity is just one of many cues. The brain (and modern CV systems) fuse several.

Four Cue Families

Binocular – requires two eyes/cameras.
Oculomotor – proprioceptive feedback from eye/camera muscles or controls.
Monocular – usable with one eye closed; pictorial cues.
Motion-induced – arise from observer or object motion.

Binocular cue.

- **Disparity (stereopsis).** The difference between the left and right images of corresponding scene points. Sufficient on its own: *random-dot stereograms* and *autostereograms* produce vivid depth percepts when all other cues are removed.

Oculomotor cues.

- **Accommodation.** The shape (refractive power) of the lens is adjusted by ciliary muscles to bring objects at different depths into focus. The state of those muscles encodes depth.
- **Convergence.** The angle by which the two eyes (or cameras) rotate inwards to fixate an object encodes that object's depth.

Monocular (pictorial) cues.

- **Interposition (occlusion).** A near object hides part of a farther one; provides ordering, not absolute depth. Manipulating it produces impossible figures (Penrose triangle, Escher, Kanizsa squares).
- **Size familiarity.** For objects of known size, smaller image ⇒ greater depth.
- **Texture gradient.** For uniform surfaces, texture elements get smaller and denser with distance.
- **Linear perspective.** Parallel lines converge to a vanishing point; closer line spacing ⇒ greater depth (e.g. railroad tracks). Strong manipulator: Ames room, Trompe-l'oeil.
- **Aerial perspective.** Atmospheric scattering reduces contrast and saturation with distance (distant mountains look bluish/hazy).
- **Shading.** Distribution of light and shadow encodes shape; the brain assumes light from above (hollow-face illusion shows shading can be overruled by familiarity).

Motion-induced cues.

- **Motion parallax.** As an observer translates sideways, near objects move opposite to the motion, far objects move with it; speed grows with distance from the fixation point.
- **Optic flow.** Forward/backward motion produces an expanding/contracting flow field; close points move faster across the image.
- **Accretion/deletion.** As the camera moves, parts of far surfaces are progressively occluded or revealed by near surfaces; the pattern signals relative depth.
- **Structure from motion (kinetic depth).** A moving rigid object viewed by a stationary camera (or a moving camera viewing a static scene) reveals 3D structure that is invisible in any single static frame.

Active depth sensing (engineering). Beyond the above passive cues, computer-vision systems often add active sensors:

Sensor	Principle	Trade-offs
Stereo (passive)	Triangulate two cameras	No emitter; struggles on textureless / shiny surfaces
Structured light	Project known pattern, measure deformation	Robust on textureless surfaces; indoor only (sunlight overwhelms pattern)
Time-of-flight (ToF)	Emit modulated IR, time the round trip	Direct depth per pixel; modest range and resolution
LIDAR	Pulsed laser scan, ToF	Long range, sparse; expensive; rotating mechanics

Spot it in a question

"Cues with one eye closed" ⇒ *monocular*. "Two oculomotor cues" ⇒ *accommodation, convergence*. "Why is depth important for object recognition?" ⇒ *depth between objects helps segmentation; depth within an object encodes shape*.

7.3 Worked Examples

Example 1: Derive $z = fB/d$ for coplanar cameras

Q: Two cameras have identical focal lengths f, coplanar image planes and collinear x-axes, separated by baseline B. Derive the relationship between point distance z and disparity d.

A: For any pinhole camera, a 3D point (X, Y, Z) in its frame projects to $(u, v) = (fX/Z,\ fY/Z)$. For the left camera (using its frame): $(u_L, v_L) = (fX_L/Z,\ fY_L/Z)$. The right camera is the same scene viewed from a frame translated by $+B$ along x, so the same world point has $X_R = X_L - B$ and $Y_R = Y_L$ in the right frame:

$$(u_R, v_R) = \left(\frac{f(X_L - B)}{Z},\ \frac{fY_L}{Z}\right).$$

Hence the horizontal disparity is

$$d \;=\; u_L - u_R \;=\; \frac{fX_L}{Z} - \frac{f(X_L - B)}{Z} \;=\; \frac{fB}{Z}, \qquad \boxed{Z = \frac{fB}{d}}.$$

Example 2: Accuracy with distance and baseline

Q: Comment on the accuracy with which depth can be measured as (a) distance changes, (b) baseline changes.

A: Accuracy depends on the size of the disparity: large d ⇒ small measurement errors translate into small z errors.

- (a) From $d = fB/z$: as z decreases, d increases. Hence accuracy *increases as the object gets closer*; very far objects have $d \to 0$ and depth becomes unreliable.
- (b) As B increases, d increases. Hence accuracy *increases as the baseline gets longer*. Trade-off: a longer baseline shrinks the common FoV and exacerbates occlusion.

Example 3: Epipolar constraint in one sentence

Q: Briefly explain the epipolar constraint.

A: For a known stereo geometry, the corresponding point of any left-image pixel must lie on a single 1D line in the right image (its *epipolar line*); this reduces correspondence search from 2D to 1D. For a simple coplanar setup, epipolar lines coincide with horizontal image rows. Computing the epipolar line in general requires the intrinsic and extrinsic camera parameters.

Example 4: Other constraints and when they fail

Q: List four further constraints used to solve the stereo correspondence problem and note circumstances where each fails.

A:

- **Maximum disparity.** Restrict search to $|d| \leq fB/z_{\min}$. *Fails* for points closer than $z_{\min}$.
- **Continuity.** Neighbouring pixels have similar disparities. *Fails* at depth discontinuities (object boundaries).
- **Uniqueness.** Each pixel matches at most one in the other image. *Fails* for surfaces angled so that they project a different number of elements to each image, and at occlusions (a pixel visible in only one camera has no match).
- **Ordering.** Matches occur in the same order along conjugated epipolar lines. *Fails* when objects sit at very different depths so the order swaps across views.

Example 5: Horopter distance with verging cameras

Q: A stereo system has baseline $B = 400$ mm and convergence angle $\theta = 60°$, with each z-axis making $60°$ with the baseline. Find the distance of a scene point from the horopter for: (a) $\alpha_L = \alpha_R = 15°$, (b) $\alpha_L = +15°,\ \alpha_R = -15°$, (c) $\alpha_L = -15°,\ \alpha_R = +15°$.

A: Disparity $d = \alpha_L - \alpha_R$. Distance from baseline along the symmetry line: the fixation point sits at $0.5B\tan(60°) = 200 \cdot 1.732 \approx 346.4$ mm.

- **(a)** $d = 15 - 15 = 0$. The point lies on the horopter, so distance from horopter $= 0$.
- **(b)** $d = 15 - (-15) = 30 > 0$. Compute angle $\angle BO_RP = 60 + 15 = 75°$; distance from baseline to P is $0.5B\tan(75°) \approx 200 \cdot 3.732 = 746.4$ mm. Distance to horopter $\approx 746.4 - 346.4 = \mathbf{400}$ mm (point is *outside* / further than horopter).
- **(c)** $d = -15 - 15 = -30 < 0$. Angle $\angle BO_RP = 60 - 15 = 45°$; distance from baseline to P is $0.5B\tan(45°) = 200$ mm. Distance to horopter $= 200 - 346.4 = \mathbf{-146.4}$ mm (point is *inside* the horopter, nearer than fixation).

Sign reading: $d > 0$ means outside, $d < 0$ means inside; the sign of distance from horopter follows the sign of d.

Example 6: Why depth helps object recognition

Q: Give two reasons why recovering depth information is important for object recognition.

A:

- **Between-object depth** provides a cue for segmentation: regions at clearly different depths are likely different objects, supporting the mid-level vision task of grouping pixels into figures.
- **Within-object depth** provides shape information (the 3D surface of the object), which supports high-level recognition by fitting/comparing 3D models or shape descriptors.

Example 7: Two oculomotor cues

Q: Briefly describe two oculomotor cues to depth.

A:

- **Accommodation.** The shape of the lens (or, in a camera, the depth of the image plane) is adjusted to bring an object into focus. The state of the focusing apparatus is therefore correlated with object depth.
- **Convergence.** In a verging system, the angle by which the two eyes/cameras rotate inwards to fixate an object encodes that object's distance. Larger convergence $\Rightarrow$ closer object.

Example 8: Four monocular cues

Q: Briefly describe four monocular (pictorial) cues to depth.

A:

- **Interposition.** A nearer object occludes (interposes itself in front of) a farther one, giving ordinal depth.
- **Size familiarity.** For an object of known size, a smaller image implies greater depth.
- **Texture gradients.** On a uniformly textured surface, texture elements shrink and crowd together with depth.
- **Linear perspective.** Lines parallel in the world converge to a vanishing point in the image; the closer the lines in the image, the greater the depth.

(*Other valid answers:* aerial perspective, shading.)

Example 9: Two motion-induced cues

Q: Briefly describe two motion-induced cues to depth.

A:

- **Motion parallax.** When the observer translates sideways, objects nearer than the fixation point appear to move opposite to the observer while farther objects appear to move with it; speed of image motion grows with distance from the fixation point.
- **Optic flow.** For forward (or backward) motion, the entire image expands (or contracts) about a focus of expansion; points closer to the camera have larger image-plane velocity.

(*Other valid answers:* accretion/deletion, structure from motion.)

Example 10: SAD disparity (LGT coding question)

Q: For a rectified stereo pair, write a window-based algorithm that computes a dense disparity map by minimising SAD along the same image row.

A: Algorithm. For each interior pixel (x, y) in the left image:

1. Extract a $w \times w$ template window centred at (x, y) from the left image.
2. For $d = 0, 1, \ldots, d_{\max}$, extract a $w \times w$ window centred at $(x - d, y)$ from the right image (skip if it falls off the left edge).
3. Compute $\mathrm{SAD}(d) = \sum_{(u,v)\in W} |I_L(u, v) - I_R(u-d, v)|$ in floating point (avoid `uint8` under/overflow).
4. Record the $d^* = \arg\min_d \mathrm{SAD}(d)$.
5. Write $\mathrm{disp}(x, y) = d^*$.

Pseudocode (rectified, half-window h_w):

```
for y in [h_w, H - h_w):
  for x in [h_w, W - h_w):
    template = L[y-h_w:y+h_w+1, x-h_w:x+h_w+1]
    best_d, best_sad = 0, +inf
    for d in [0, d_max):
```

```
    if x - d - h_w < 0: break
    win = R[y-h_w:y+h_w+1, x-d-h_w:x-d+h_w+1]
    sad = sum(|template.float - win.float|)
    if sad < best_sad: best_sad, best_d = sad, d
  disp[y, x] = best_d
```

Pitfalls / pattern checks:

- Convert to `float32` *before* subtracting – otherwise unsigned-integer wrap-around silently corrupts SAD.
- The bounds check `x - d - h_w < 0` avoids reading off the left edge.
- Window-size trade-off: small $w \Rightarrow$ noisy/streaky map; large $w \Rightarrow$ smooth but blurred at depth boundaries.

Example 11: Why two cameras (DOF counting)

Q: Why are at least two cameras needed to reconstruct a 3D position?

A: A single camera's projection $(u, v) = (fx/z, fy/z)$ gives 2 scalar equations in 3 unknowns (x, y, z) – under-determined: an entire ray of 3D points satisfies it. A second calibrated camera adds 2 more equations from a different viewpoint, giving 4 equations in 3 unknowns; intersection of the two view-rays (*triangulation*) recovers (x, y, z). The DLT formulation makes this explicit with 11 parameters per camera (6 extrinsic + 5 intrinsic).

Example 12: Coplanar vs non-coplanar (summary)

Q: Contrast the coplanar and non-coplanar (verging) stereo cases on (a) image planes, (b) epipolar lines, (c) disparity, (d) depth recovery.

A:

- (a) Coplanar: image planes parallel, optical axes parallel (intersect at ∞), x-axes collinear. Verging: image planes related by a translation *and* a rotation; optical axes intersect at a finite fixation point.
- (b) Coplanar: epipolar lines are horizontal scan lines; epipoles at infinity. Verging: epipolar lines radiate from the (finite) epipoles in each image.
- (c) Coplanar: disparity in pixels, $d = x'_L - x'_R$. Verging: disparity in angles, $d = \alpha_L - \alpha_R$, with sign indicating inside/outside the horopter.
- (d) Coplanar: $z = fB/d$ directly. Verging: triangulate via the calibrated geometry, or first *rectify* both images so epipolar lines become rows and reduce to the simple case.

7.4 Summary

Key Takeaways

1. **One camera loses depth.** Every point on a line of sight maps to one pixel; two viewpoints break the ambiguity by triangulation.
2. **Coplanar stereo** (rectified, parallel cameras): disparity $d = x'_L - x'_R = fB/z \Rightarrow z = fB/d$. Disparity is purely horizontal.
3. **Accuracy** of z improves with larger disparity, i.e. closer points and longer baselines; long baselines reduce common FoV.

4. **Correspondence problem.** Find matching pixels. Constraints: **epipolar** (1D search), **maximum disparity**, **continuity**, **uniqueness**, **ordering** – each fails in characteristic situations.
5. **Window matching** (SAD/SSD/NCC) gives dense maps; feature matching gives sparse maps. Window-size trade-off: detail vs noise.
6. **Field of view** $= 2\arctan(W/2f)$. Short baseline $\Rightarrow$ large cFoV but large depth error; long baseline $\Rightarrow$ small cFoV, small depth error.
7. **Verging cameras**: disparity is angular $(\alpha_L - \alpha_R)$, zero on the horopter. $d > 0$ outside the horopter, $d < 0$ inside. Recover depth by triangulation or by rectifying to the simple case.
8. **Epipolar geometry**: baseline, epipoles (e_L, e_R), epipolar plane (per 3D point), epipolar lines (intersection with image plane), conjugated epipolar lines. *Rectification* makes epipolar lines image rows.
9. **Cues to depth**: binocular (disparity), oculomotor (accommodation, convergence), monocular (interposition, size familiarity, texture gradient, linear/aerial perspective, shading), motion (parallax, optic flow, accretion/deletion, structure from motion). Active sensors (LIDAR, structured light, ToF) supplement passive vision.

Final comparison: stereo geometry.

Property	**Coplanar (simple)**	**Verging (general)**
Image planes	Translation along x only	Translation + rotation
Optical axes	Parallel (intersect at ∞)	Intersect at fixation point
Epipoles	At infinity	Finite, in each image
Epipolar lines	Horizontal scan lines	Radiate from epipoles
Disparity unit	Pixels	Angles
Disparity formula	$d = x'_L - x'_R$	$d = \alpha_L - \alpha_R$
Zero disparity	Points at infinity	Horopter (depends on vergence)
Depth formula	$z = fB/d$	Triangulate (or rectify + simple)

Final comparison: depth cues.

Family	**Cue**	**What it encodes**
Binocular	Disparity	Difference in image of corresponding points
Oculomotor	Accommodation	Lens shape / focus state $\rightarrow$ object depth
Oculomotor	Convergence	Inward rotation angle $\rightarrow$ fixation depth
Monocular	Interposition	Near object occludes far (ordering)
Monocular	Size familiarity	Smaller image of known object $\rightarrow$ farther
Monocular	Texture gradient	Texture shrinks/densifies with depth
Monocular	Linear perspective	Parallel lines converge at vanishing point
Monocular	Aerial perspective	Lower contrast/saturation with depth
Monocular	Shading	Light/shadow distribution $\rightarrow$ shape
Motion	Motion parallax	Near vs far image motion under observer translation
Motion	Optic flow	Expansion/contraction during forward/backward motion
Motion	Accretion/deletion	Occlusion changes during camera motion
Motion	Structure from motion	3D shape from view sequence

8 Week 8: Motion and Tracking

8.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Distinguish the ***motion field*** from ***optic flow*** and explain when they differ.
- State the ***brightness constancy*** assumption and derive the ***optic flow constraint equation*** $I_x u + I_y v + I_t = 0$.
- Explain the ***aperture problem*** and describe how it can be overcome.
- Compute depth, time-to-collision, ego-motion direction, and segmentation from optic flow.
- Compare direct, dense methods (Lucas–Kanade, Horn–Schunck) with feature-based and block-matching methods.
- Segment moving objects via ***image differencing*** and ***background subtraction*** (offline / moving average).
- Define object ***tracking***, contrast tracking-by-detection with model-based tracking, and describe the ***predict–update*** cycle.
- Outline the ***Kalman filter***, ***particle filter***, ***mean-shift***, and ***KLT*** trackers, including their assumptions and failure modes.

Big Picture. A video is a sequence of images $I(x, y, t)$ acquired at discrete times $t_k = t_0 + k\Delta t$. Two new things become possible compared with stereo: we can infer *self*/object ***motion***, and we can ***segment*** moving objects without recovering depth. The unifying object is the ***optic flow field*** (u, v): the image-plane motion of each scene point. Optic flow is only an *approximation* to the true motion field (uniform rotating sphere $\Rightarrow$ MF $\neq 0$, OF $= 0$); but since the motion field cannot be observed, we estimate it by tracking brightness patterns. From (u, v) we recover depth, time-to-collision, ego-motion, and segmentation. When high-level features (objects) are followed across many frames we call this ***tracking***, which adds a ***predict–update*** (Bayesian) loop on top of detection: Kalman (linear–Gaussian), particle filter (multimodal/non-linear), mean-shift (mode-seek on appearance), and KLT (sparse feature optic flow).

What Examiners Like to Ask:

- "Define optic flow and motion field; give a case where they differ."
- "State and explain the aperture problem; how is it overcome?"
- "List constraints for the video correspondence problem and when they fail."
- Numerical: depth from $Z = -V_x x_1/\dot{x}$ (perpendicular motion) or $Z_2 = V_z x_1/\dot{x}$ (along optical axis); time-to-collision $= x_1/\dot{x}$.
- Apply image differencing or moving-average background subtraction to a 1D pixel patch over several frames with threshold T.
- Derive the optic flow constraint equation from $I(x, y, t) = I(x + u\,dt,\ y + v\,dt,\ t + dt)$.
- Describe Lucas–Kanade vs Horn–Schunck (local vs global; assumptions; output dense?).
- "What is tracking? Why predict before measure?" Compare tracking-by-detection with model-based tracking.
- Sketch the Kalman predict–update equations and state when they apply.

8.2 Core Concepts

Roadmap:

1. Motion: video, motion field vs optic flow, brightness constancy, optic flow constraint, aperture problem.
2. Estimating optic flow: feature-based, Lucas–Kanade, Horn–Schunck, block matching.
3. Applications of optic flow: depth, time-to-collision, ego-motion, motion segmentation.
4. Motion segmentation: image differencing and background subtraction.
5. Tracking: definition, predict–update cycle, tracking-by-detection vs model-based.
6. Trackers: Kalman filter, particle filter, mean-shift, KLT; occlusion handling.

8.2.1 Video, Motion Field, and Optic Flow

Video

A series of N images (frames) acquired at discrete times $t_k = t_0 + k\Delta t$, $k = 0, 1, \ldots, N-1$, with fixed time interval Δt. A pixel intensity is a function of both space and time: $I(x, y, t)$.

Stereo vs video. Both deal with ≥ 2 images. Stereo = multiple cameras at one time. Video = one camera at multiple times. Video additionally enables: (i) inference of 3D structure (as with stereo); (ii) segmentation of objects from background *without* recovering depth; (iii) inference of self- and object-motion.

Motion Field (MF)

The true image motion of a scene point: the actual 2D projection on the image plane of the relative 3D motion between camera and scene. Cannot be observed directly.

Optic Flow (OF)

The apparent image motion of a scene point, inferred from changes in image brightness. The ***optic flow field*** is the collection of all OF vectors (***sparse*** if only at features, ***dense*** if everywhere).

MF $\neq$ OF in general. Optic flow only approximates the motion field. Three classical counter-examples:

Scene	MF	OF	Why
Rotating uniform Lambertian sphere	$\neq 0$	$= 0$	Surface moves but image is constant
Stationary specular sphere, moving light	$= 0$	$\neq 0$	Highlight moves but surface does not
Rotating barber's pole	horizontal	vertical	Aperture problem on the stripes

Despite this, MF cannot be observed, so we always *estimate* MF by computing OF.

8.2.2 Brightness Constancy and the Optic Flow Constraint

Brightness Constancy Assumption

The intensity of a scene point does not change between consecutive frames as it moves: if a point at (x, y) at time t moves to $(x + u\,dt,\ y + v\,dt)$ at time $t + dt$, then

$$I(x, y, t) \;=\; I(x + u\,dt,\ y + v\,dt,\ t + dt).$$

Derivation of the OFCE. Taylor-expand the right-hand side and keep first-order terms:

$$I(x, y, t) \;=\; I(x, y, t) + I_x\, u\, dt + I_y\, v\, dt + I_t\, dt + O(dt^2).$$

Cancelling and dividing by dt gives:

Optic Flow Constraint Equation (OFCE)

$$I_x\, u \;+\; I_y\, v \;+\; I_t \;=\; 0, \qquad \text{i.e.} \qquad \nabla I \cdot (u, v)^T \;=\; -I_t.$$

I_x, I_y are spatial derivatives, I_t is the temporal derivative, (u, v) is the unknown flow.

Assumptions: (1) brightness constancy, (2) small motion (so the linearisation is accurate), (3) image is differentiable in space and time.

Spot it in a question

"One equation, two unknowns" $\Rightarrow$ *one OFCE per pixel cannot determine* (u, v): this *is* the aperture problem in equation form. We can only recover the component of flow *parallel to* ∇I (the ***normal flow***); the component perpendicular to ∇I (along the edge) is undetermined.

8.2.3 The Aperture Problem

Aperture Problem

Through a small "aperture" (image patch) the direction of motion of an extended edge is ambiguous: only the component of motion *perpendicular* to the edge is recoverable; any component *parallel* to the edge is invisible because the intensity does not change along the edge.

Why we "see" perpendicular motion. Among all consistent motions, the perpendicular one (i) is the average of all possibilities and (ii) predicts the slowest movement; the brain (and many algorithms) defaults to it.

Two strategies for solving it:

1. **Integrate** local measurements across space (combine many edge patches with different orientations).
2. **Restrict** measurements to image locations where the structure is unambiguous: ***corners***. SIFT / Harris detectors are good both for stereo and for video correspondence (the basis of KLT).

8.2.4 The Video Correspondence Problem

To measure OF, find corresponding points across frames. Two families of methods:

Family	Idea	Verdict
Feature-based	Extract descriptors at interest points; match to next frame	Sparse OF; works for *large* motion
Direct (differential)	Recover motion from I_x, I_y, I_t via OFCE / spatio-temporal filters	Dense OF; sensitive to appearance change; needs *small* motion

Constraints used to find correspondences:

Constraint	Statement	Fails when
Spatial coherence	Neighbouring flow vectors are similar (smooth surfaces)	Depth/motion discontinuities
Small motion	Flow magnitude is small relative to image size	Fast motion or low frame rate

---

8.2.5 Estimating Optic Flow

The OFCE gives one equation per pixel and two unknowns. We need additional constraints. Three classical strategies:

(1) Lucas–Kanade (local windowed least squares).

Lucas–Kanade

Assume the flow (u, v) is *constant* over a small window W around a pixel. Stack one OFCE per pixel in W:

$$\underbrace{\begin{pmatrix} I_{x,1} & I_{y,1} \\ \vdots & \vdots \\ I_{x,n} & I_{y,n} \end{pmatrix}}_{A} \begin{pmatrix} u \\ v \end{pmatrix} = \underbrace{\begin{pmatrix} -I_{t,1} \\ \vdots \\ -I_{t,n} \end{pmatrix}}_{b}.$$

Solve in least squares:

$$\begin{pmatrix} u \\ v \end{pmatrix} = (A^T A)^{-1} A^T b, \qquad A^T A = \begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix}.$$

Conditions / pitfalls: $A^T A$ must be invertible and well-conditioned, i.e. the window must contain enough *texture in two different directions*. This is exactly the ***Harris corner*** criterion: LK works well at corners, fails on flat regions and along edges (aperture problem returns).

(2) Horn–Schunck (global smoothness).

Horn–Schunck

Solve a global energy minimisation:

$$E(u, v) = \int \underbrace{(I_x u + I_y v + I_t)^2}_{\text{data: OFCE}} + \alpha^2 \underbrace{\left(\|\nabla u\|^2 + \|\nabla v\|^2\right)}_{\text{smoothness regulariser}} \, dx\, dy.$$

α trades off data fit against smoothness. Solved iteratively (Jacobi/Gauss–Seidel).

Output: dense flow field. **Pitfalls:** oversmooths real motion discontinuities (object boundaries); slower than LK.

(3) Block matching (feature/correlation). For each block in frame t, search a window in frame $t + 1$ for the best-matching block (SSD / SAD / NCC). The displacement vector is the OF for that block. Used in MPEG video coding and for large displacements where differential methods fail.

Comparison.

Method	Density	Constraint	Best for	Weakness
Lucas–Kanade	Sparse (at corners)	Local constancy	Small motion at corners	Fails on flat / edges
Horn–Schunck	Dense	Global smoothness	Smooth flow fields	Blurs at boundaries
Block matching	Sparse–dense	Patch similarity	Large displacements	Aperture, expensive
Feature-based	Sparse	Descriptor match	Wide baselines	Requires distinctive features

8.2.6 Applications of Optic Flow

Depth from OF, known ego-motion (case 1: motion $\perp$ optical axis). If camera moves with velocity V_x along the camera x-axis,

$$Z = -\frac{V_x\, x_1}{\dot{x}}, \qquad \dot{x} = \frac{x_2 - x_1}{\Delta t}.$$

Case 2: motion along optical axis V_z.

$$Z_2 = \frac{V_z\, x_1}{\dot{x}}.$$

Image coordinates here are measured *from the principal point*, not the corner of the image.

Time-to-collision (Vz unknown). Eliminating V_z from the case-2 equation:

$$\boxed{\tau = \frac{x_1}{\dot{x}} = \frac{\alpha}{\dot{\alpha}} = \frac{2A}{\dot{A}}}$$

where α is the angle subtended by the object and A its image area. Computable from image alone — no knowledge of camera speed, object size, or distance. Used by birds and insects for landing.

Ego-motion patterns.

Field	Pattern	Camera motion
Parallel ($V_z = 0$)	All vectors parallel; OF opposite to camera direction; speed $\propto \lvert\mathbf{v}\rvert$	Pure translation perpendicular to optical axis (turn/translate left/right)
Radial ($V_z \neq 0$)	Vectors point to/away from a vanishing point (FOE/FOC)	Move forward (FOE) / backward (FOC); FOE = destination

Relative depth. In a parallel field, depth $\propto 1/|\mathbf{v}|$ (motion parallax with fixation at infinity). In a radial field, depth of point $p \propto 1/(|\mathbf{v}| \cdot \text{distance from FOE})$.

Segmentation from OF. Discontinuities in the OF field indicate different depths and hence different objects. Useful even when stereo depth is unavailable.

8.2.7 Motion Segmentation: Image Differencing and Background Subtraction

(1) Image differencing (static camera). For consecutive frames,

$$\delta(x, y, t) = |I(x, y, t) - I(x, y, t-1)| > T \;\Rightarrow\; \text{moving pixel.}$$

Pitfall (*double-view problem*): the largest changes occur where background is replaced by object (or vice versa); pixels covered by the object in *both* frames may not change. The output looks like a hollow "ghost" / double silhouette.

---

(2) Background subtraction. Maintain a background model $B(x, y)$ and threshold $|I(x, y, t) - B(x, y)| > T$. Three ways to learn B:

Method	Update	Properties
Adjacent frame difference	$B(x, y) = I(x, y, t-1)$	Same as image differencing; double-view bug
Offline average (mean+threshold)	$B(x, y) = \frac{1}{N}\sum_{t=1}^{N} I(x, y, t)$	Needs a clean training phase; static
Moving (running) average	$B(x, y) \leftarrow (1-\beta)B(x, y) + \beta I(x, y, t)$	Adapts to slow illumination changes; *most common*

Typical algorithm (per frame t):

1. Update background: $B \leftarrow (1-\beta)B + \beta I_t$.
2. Compute $\delta(x, y, t) = \|I(x, y, t) - B(x, y)\|$.
3. Threshold: $\delta_T(x, y, t) = \delta(x, y, t) > T$.
4. Noise removal: $\delta_T \leftarrow \text{close}(\delta_T)$ (morphological dilation+erosion).

Foreground pixels are where $\delta_T \neq 0$.

What a good background model should do: *exclude* interesting changes (objects, even if temporarily stopped); *incorporate* uninteresting changes (illumination drift, swaying trees, sun/clouds, lights on/off).

8.2.8 Tracking

Tracking

Matching high-level features (typically objects) across many frames of a video. Differs from optic flow in scope: trackers carry an explicit *state* (location, velocity, appearance) for each tracked object and refine it over time.

Predict–update cycle. Tracking algorithms use previous frames to *predict* the location of the object in the next frame, then *update* (correct) the prediction with a measurement.

1. **Predict.** From the previous state and a motion model, compute a prior estimate of where the object will be next.
2. **Measure.** Detect the object in the predicted region (smaller search ⇒ less work, less ambiguity).
3. **Update.** Combine prediction and measurement into the new state, weighting each by its uncertainty.

Why predict first? (i) restrict the search region (faster, fewer false matches); (ii) average out measurement noise across frames.

Two paradigms:

Paradigm	Idea	Pros / Cons
Tracking-by-detection	Run a detector on each frame; associate detections across frames (e.g. via a Kalman/particle filter)	Robust to drift; relies on a strong detector; data association is hard
Model-based tracking	Maintain a target appearance/shape model and search for it next frame (KLT, mean-shift, correlation filters)	Cheaper per frame; can drift; struggles with occlusion/appearance change

8.2.9 Trackers: Kalman, Particle, Mean-Shift, KLT

Kalman filter.

Kalman Filter (linear Gaussian)

State $\mathbf{x}_t$, measurement $\mathbf{z}_t$. Linear model:

$$\mathbf{x}_t = F\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, Q); \qquad \mathbf{z}_t = H\mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, R).$$

Predict: $\hat{\mathbf{x}}_{t|t-1} = F\hat{\mathbf{x}}_{t-1|t-1}$, $P_{t|t-1} = FP_{t-1|t-1}F^T + Q$.
Update: Kalman gain $K_t = P_{t|t-1}H^T(HP_{t|t-1}H^T + R)^{-1}$; state $\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + K_t(\mathbf{z}_t - H\hat{\mathbf{x}}_{t|t-1})$; covariance $P_{t|t} = (I - K_tH)P_{t|t-1}$.

Assumptions: linear dynamics, linear measurement, Gaussian noise. **Strengths:** closed-form, optimal in the MMSE sense under those assumptions, very cheap. **Failure:** non-linear motion (use EKF/UKF) or multimodal posteriors (target hidden behind another).

Particle filter (sequential Monte Carlo).

Particle Filter

Represent the posterior $p(\mathbf{x}_t \mid \mathbf{z}_{1:t})$ by N weighted samples (***particles***) $\{(\mathbf{x}_t^{(i)}, w_t^{(i)})\}$.

1. **Predict (sample):** draw $\mathbf{x}_t^{(i)} \sim p(\mathbf{x}_t \mid \mathbf{x}_{t-1}^{(i)})$ from the motion model.
2. **Update (weight):** $w_t^{(i)} \propto w_{t-1}^{(i)}\, p(\mathbf{z}_t \mid \mathbf{x}_t^{(i)})$ (likelihood of observation).
3. **Resample:** draw N new particles in proportion to weights to avoid degeneracy.

Strengths: arbitrary non-linear, non-Gaussian, *multimodal* posteriors (handles ambiguity, e.g. tracking through clutter). **Cost:** $O(N)$ particles per frame; needs many in high dimensions (curse of dimensionality).

Mean-shift tracking. Build an appearance model (typically a colour histogram) of the target. At each frame, start at the previous location and iteratively move the search window to the local *mode* of the back-projected likelihood (a histogram-based density). **Pros:** simple, real-time, deals with non-rigid deformation. **Cons:** fixed window scale (CamShift fixes this); fails if the target leaves the window or the colour distribution overlaps the background.

KLT tracker (Kanade–Lucas–Tomasi). Sparse model-based tracker:

1. Detect "good features to track" (corners with two strong eigenvalues of $A^T A$ from the LK system).
2. Track each feature into the next frame with Lucas–Kanade (small window, brightness constancy, small motion).
3. Drop features that fail a residual / dissimilarity test; redetect periodically.

KLT is essentially LK + corner selection + bookkeeping over many frames. It assumes *small* motion, so for fast motion it is run at multiple scales (a Gaussian pyramid: see Week 3).

Comparison.

Tracker	State	Posterior	Best when
Kalman	Linear (e.g. pos + vel)	Gaussian (unimodal)	Smooth linear motion, low noise
Particle filter	Arbitrary	Arbitrary, multimodal	Clutter, non-linear, ambiguous
Mean-shift	Window position	Mode of appearance	Distinctive colour, non-rigid target
KLT	Sparse feature locations	Implicit (LK)	Textured target, small motion, short videos

Occlusion handling. The predict–update view also tells you what to do during occlusion: when no measurement is available (or the likelihood is poor), *skip the update* and keep *predicting* forward, letting the covariance grow until the target reappears. Long occlusions are typically resolved by a re-detection step (tracking-by-detection) or by maintaining multiple hypotheses (particle filter).

8.3 Worked Examples

Example 1: Constraints on the video correspondence problem

Q: List the constraints typically applied to solving the video correspondence problem and note when they fail.

A:

- **Spatial coherence** (neighbouring points have similar flow). Fails at depth/motion discontinuities (object boundaries, two surfaces moving differently).
- **Small motion** (flow magnitude is small). Fails when relative motion is fast or the frame rate is low.

Example 2: Aperture problem

Q: Define the aperture problem and suggest how it can be overcome.

A: The direction of motion of an extended edge seen through a small aperture is ambiguous: only the component of motion *perpendicular* to the edge is recoverable; the parallel component cannot be determined because the intensity does not change along the edge. Overcome by (1) integrating local motion measurements across many image patches with different orientations, or (2) restricting measurements to image locations where the structure is unambiguous (***corners***, e.g. Harris/SIFT interest points).

Example 3: Barber's pole

Q: A traditional barber's pole rotates about its vertical axis. (a) In what direction is the motion field? (b) The optic flow? (c) Explain via the aperture problem.

A:

- (a) MF is **horizontal** (the painted surface rotates around the vertical axis).
- (b) OF is **(predominantly) vertical**.
- (c) The diagonal stripes are extended edges, so locally only the perpendicular component is recoverable. Corners exist only at the top/bottom (small horizontal component) and at the sides (large vertical component). Either by averaging local cues or by relying on unambiguous corner cues, the aggregate motion is seen as vertical.

Example 4: Depth from perpendicular ego-motion

Q: Frames at t and $t+1\,\mathrm{s}$. Point $(50, 50, t)$ matches $(25, 50, t+1)$. Camera moves at $V_x = 0.1\,\mathrm{m/s}$ along the camera x-axis. Focal length $f = 35\,\mathrm{mm}$, pixel size $0.1\,\mathrm{mm/pixel}$. Find depth Z.

A: Use $Z = -V_x x_1/\dot{x}$.

Image-point velocity: $\dot{x} = (25-50)/1 = -25\,\mathrm{pixels/s} = -25 \times 10^{-4}\,\mathrm{m/s} = -0.0025\,\mathrm{m/s}$.

Convert $x_1 = 50\,\mathrm{pixels} \to 50 \times 0.1\,\mathrm{mm} = 5\,\mathrm{mm} = 0.005\,\mathrm{m}$. (In the slide solution this is left as the focal-length-scaled coordinate $fx_1 = 0.035 \times 0.1\,\mathrm{m} \cdot \mathrm{pix}$.) Plugging in:

$$Z = -\frac{0.035 \times 0.1}{-0.0025} = 1.4\,\mathrm{m}.$$

Example 5: Depth from along-axis ego-motion

Q: Point $(50, 70, t) \to (45, 63, t+1)$, $V_z = 0.1\,\mathrm{m/s}$ along the optical axis, image centre at $(100, 140)$. Find depth.

A: Re-express coordinates relative to the principal point: $(-50, -70) \to (-55, -77)$. Use $Z_2 = V_z x_1/\dot{x}$.

$\dot{x} = (-55 - (-50))/1 = -5\,\mathrm{pixels/s}$. So

$$Z_2 = \frac{0.1 \times (-50)}{-5} = 1\,\mathrm{m}.$$

Cross-check with y: $\dot{y} = -7\,\mathrm{pix/s}$, $Z_2 = (0.1 \times -70)/(-7) = 1\,\mathrm{m}$. ✓

Example 6: Time-to-collision

Q: Give an equation for time-to-collision that does not require depth, and apply it to the previous question.

A: $\tau = x_1/\dot{x}$. For the previous point: $\tau = -50/-5 = 10\,\mathrm{s}$.

Sanity check: the camera was $1\,\mathrm{m}$ away moving at $0.1\,\mathrm{m/s}$, which gives the same answer.

Example 7: Image differencing on a 1D patch

Q: Four frames of a 1×5 patch:

$$I_{t_1} = [190, 200, 90, 110, 90], \quad I_{t_2} = [110, 170, 160, 70, 70],$$

$$I_{t_3} = [100, 60, 170, 200, 90], \quad I_{t_4} = [90, 100, 100, 190, 190].$$

Apply image differencing with threshold $T = 50$.

A: Compute $|I_{t_{k+1}} - I_{t_k}|$ then threshold:

- $|I_{t_1} - I_{t_2}| = [80, 30, 70, 40, 20] \rightarrow [1, 0, 1, 0, 0]$.
- $|I_{t_2} - I_{t_3}| = [10, 110, 10, 130, 20] \rightarrow [0, 1, 0, 1, 0]$.
- $|I_{t_3} - I_{t_4}| = [10, 40, 70, 10, 100] \rightarrow [0, 0, 1, 0, 1]$.

Note the double-view artefact: a single moving feature lights up two pixels per pair of frames.

Example 8: Moving-average background subtraction

Q: Same 4 frames. Use moving-average $B \leftarrow (1-\beta)B + \beta I_t$ with $\beta = 0.5$, B initialised to zero, threshold $T = 50$.

A: (Update B *then* compare to I_t.)

- t_1: $B = 0.5I_{t_1} = [95, 100, 45, 55, 45]$. $|I_{t_1} - B| = [95, 100, 45, 55, 45] \rightarrow [1, 1, 0, 1, 0]$.
- t_2: $B = 0.5B + 0.5I_{t_2} = [102.5, 135, 102.5, 62.5, 57.5]$. $|I_{t_2} - B| = [7.5, 35, 57.5, 7.5, 12.5] \rightarrow [0, 0, 1, 0, 0]$.
- t_3: $B = [101.25, 97.5, 136.25, 131.25, 73.75]$. $|I_{t_3} - B| = [1.25, 37.5, 33.75, 68.75, 16.25] \rightarrow [0, 0, 0, 1, 0]$.
- t_4: $B = [95.625, 98.75, 118.125, 160.625, 131.875]$. $|I_{t_4} - B| = [5.625, 1.25, 18.125, 29.375, 58.125] \rightarrow [0, 0, 0, 0, 1]$.

Take-away: initialising B to zero makes the first frame mostly foreground (cold start); steady state takes a few frames depending on β.

Example 9: Coding – moving-average tracker (LGT skeleton)

Q: Sketch the body of `process_video_live(video_path, alpha=0.05, threshold=30, min_area=500)` that does (i) moving-average background subtraction; (ii) contour-based bounding-box tracking.

A: Pseudo-code:

1. Read first frame → `gray`; init `background = gray.astype(float32)`.
2. Each subsequent frame: `gray = grayscale(frame)`.
3. `diff = abs(gray - background)` (in float32 to avoid overflow).
4. `fg_mask = (diff > threshold) * 255` (uint8 binary mask).
5. `background = (1 - alpha) * background + alpha * gray`.
6. Find external contours of `fg_mask`; for each contour with area > `min_area`, draw `boundingRect` on a copy of the frame.

Pitfalls (exam-relevant): use `float32` (uint8 wraps around at 255); update background *after* computing `diff`, otherwise the moving target gets baked into B; `RETR_EXTERNAL` avoids nested holes producing duplicate boxes; `min_area` suppresses noise.

Example 10: Why predict before measure?

Q: Tracking algorithms predict the next location before measuring it. Give two reasons.

A:

- **Restrict the search.** Less work per frame and fewer false matches in the surrounding clutter.
- **Average measurement noise.** The state estimate combines many noisy measurements through the dynamics, so the trajectory is smoother and more accurate than per-frame detection.

Example 11: Choosing a tracker

Q: For each scenario, pick Kalman, particle filter, mean-shift, or KLT, and justify.

A:

- **Radar-style point target moving smoothly:** Kalman — linear dynamics, Gaussian noise, single mode.
- **Pedestrian in cluttered crowd, frequently occluded:** particle filter — multimodal posterior, non-linear motion, occlusion handled by predicting through gaps.
- **Distinct-coloured ball on uniform background:** mean-shift — colour histogram is highly discriminative; cheap.
- **Textured rigid object in a short clip with small motion:** KLT — corners give unambiguous flow; LK works because motion is small.

Example 12: Lucas–Kanade derivation in one line

Q: Why does Lucas–Kanade fail along an edge?

A: On an edge, I_x and I_y are nearly parallel within the window, so the rows of A are nearly collinear and $A^T A$ is rank-deficient (one small eigenvalue). The least-squares solution is unstable and recovers only the normal flow — the aperture problem in matrix form. LK fails on textureless regions for the same reason ($\nabla I \approx 0$).

8.4 Summary

Key Takeaways

1. **Motion field $\neq$ optic flow**: MF is the true projected motion (unobservable); OF is what we infer from brightness changes. Counter-examples: rotating Lambertian sphere, specular sphere with moving light, barber's pole.
2. **Brightness constancy $\Rightarrow$ OFCE** $I_x u + I_y v + I_t = 0$. One equation, two unknowns $\Rightarrow$ **aperture problem**: only normal flow recoverable.
3. **Solve OF** by adding constraints: Lucas–Kanade (local constancy + LSQ), Horn–Schunck (global smoothness), block matching, feature matching.
4. **Aperture problem solutions:** integrate across patches OR use corners (LK, KLT).
5. **Applications of OF:** depth (with ego-motion), time-to-collision $\tau = x_1/\dot{x}$ (no depth needed), ego-motion direction (FOE/FOC), motion segmentation.
6. **Motion segmentation:** image differencing (double-view artefact) vs background subtraction (offline mean / moving average $B \leftarrow (1-\beta)B + \beta I_t$).
7. **Tracking = predict + update**; tracking-by-detection (detect + associate) vs model-based (KLT, mean-shift).
8. **Trackers**: Kalman (linear–Gaussian, unimodal), particle filter (non-linear, multimodal), mean-shift (mode of colour histogram), KLT (corners + LK).
9. **Occlusion**: keep predicting, skip update; growing covariance signals lost track.

Final comparison: optic flow methods.

Method	Density	Extra constraint	Failure mode
Lucas–Kanade	Sparse (corners)	Local constant flow over window	Flat / edge regions
Horn–Schunck	Dense	Global smoothness $\alpha^2\|\nabla u\|^2$	Blurs motion boundaries
Block matching	Sparse-dense	Best patch match (SSD/NCC)	Aperture, expensive
Feature-based	Sparse	Descriptor distance	Few/repetitive features

Final comparison: trackers.

Tracker	Posterior	Motion model	Best for
Kalman	Gaussian (unimodal)	Linear	Smooth, predictable trajectories
Particle filter	Arbitrary, multimodal	Any	Clutter, occlusion, ambiguity
Mean-shift	Mode of histogram	None (greedy)	Distinctive colour, deformable
KLT	Implicit (LK)	Small motion	Textured rigid targets

9 Week 9: Artificial Object Recognition (High-Level Vision)

9.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Distinguish the three object-recognition tasks: ***identification***, ***classification***, ***localisation***, and relate them to the category hierarchy.
- State the requirements of object recognition (sensitivity vs invariance) and the standard sources of nuisance variation (viewpoint, lighting, deformation, clutter, occlusion).
- Describe and contrast the nine classical methods covered: ***template matching***, ***sliding window***, ***edge matching***, ***model-based*** recognition, ***intensity histograms***, ***Implicit Shape Model (ISM)***, ***SIFT feature matching***, ***bag-of-words***, ***geometric invariants***.
- Compute standard similarity measures (cross-correlation, NCC, correlation coefficient, SSD, SAD).
- Explain Lowe's ratio test and the role of RANSAC / Generalised Hough Transform in feature-based recognition.
- Classify methods along three axes: top-down vs bottom-up, representation (pixel intensity / features / geometry), 2D vs 3D, local vs global.

Big Picture. Object recognition is the high-level task of associating image evidence with object identities or categories. Every recognition pipeline has the same skeleton: *(i) image data*, *(ii) a representation of the objects*, and *(iii) a matching technique*. Methods differ in *what* representation is used (pixel intensities, features, or geometry; 2D vs 3D; local vs global) and *how* matching is performed (top-down/generative "hypothesise and verify" vs bottom-up/discriminative "extract and look up"). The fundamental tension is ***sensitivity*** (small relevant differences must be distinguishable) vs ***invariance*** (large irrelevant differences must be ignored). Classical pipelines trade these off in different ways; the lecture surveys nine of them. Modern recognition is dominated by neural networks, but the classical building blocks (similarity measures, feature descriptors, voting schemes, geometric verification) still appear inside or alongside deep models.

What Examiners Like to Ask:

- Define identification vs classification vs localisation; place them in the category hierarchy.
- List the requirements of object recognition (D-O-R-I-style invariance lists: viewpoint, lighting, deformation, within-category, clutter, occlusion).
- Describe the template matching pipeline; state similarity measures and which are maximised vs minimised.
- Explain why template matching fails: scale, rotation, viewpoint, occlusion, threshold trade-off, tractability.
- Compare sliding window with template matching; explain how segmentation reduces the search space.
- Describe the bag-of-words pipeline (parsing → ignore stop-words → stems → ID → histogram) for documents and its image analogue (interest points → SIFT → k-means → codebook → histogram).
- Describe the ISM pipeline and the voting / Generalised Hough Transform step.
- Describe SIFT feature matching, including Lowe's ratio test and geometric verification with RANSAC.
- State the geometric invariant of each transformation group: Euclidean (length, angle, area), similarity (length ratios, angles), affine (parallelism, length ratios on a line, area ratios), projective (cross-ratio).

- Compare local vs global representations: which is robust to occlusion / viewpoint, which can distinguish similar objects, which generates more false positives / negatives.

9.2 Core Concepts

Roadmap:

1. Recognition tasks and the category hierarchy.
2. Recognition requirements (sensitivity vs invariance) and the generic procedure.
3. Template matching + similarity measures.
4. Sliding window.
5. Edge matching.
6. Model-based recognition.
7. Intensity histograms.
8. Implicit Shape Model (ISM).
9. SIFT feature matching.
10. Bag-of-words (BoW).
11. Geometric invariants.
12. Classification of methods (top-down vs bottom-up; representation; local vs global).

9.2.1 Recognition tasks

Three object-recognition tasks

- **Identification.** Determine the identity of an *individual instance* of an object (e.g. Clinton vs Bush; Galaxy On8 vs iPhone 7+).
- **Classification (categorisation).** Determine the *category* of an object (e.g. human vs chimp; telephone vs calculator).
- **Localisation.** Determine the *presence and/or location* of an object in the image (bounding boxes / labels).

Semantic segmentation. If localisation is sufficiently fine-grained *and* run for a sufficiently large set of categories, the result becomes a per-pixel labelling — ***semantic segmentation***. Note: semantic segmentation differs from the binary segmentation of earlier weeks (which separated one foreground from background); semantic segmentation assigns class labels at every pixel.

Category hierarchy. Classification can take place at multiple levels of abstraction:

Level	Examples	Properties
Superordinate	object, animal, mammal, fruit	Few features in common across members
Basic level	cat, dog, cow, apple, banana	**Members share many features; easy to discriminate from peers**
Subordinate	poodle, doberman, Bramley apple, Cox apple	Members very similar within class
Identification	"Rex", "Fido"	Single instance; extreme of hierarchy

Why basic level matters

The basic level is the level at which:

- Humans are *fastest* at recognising category members.
- Humans usually classify *first* before doing identification.
- Children learn category names *first*.
- Members share many features (perceptually homogeneous) yet are distinct from peers (easy to discriminate).

It is the highest level at which members share many features, and the lowest level at which members have features distinct from other peer categories.

9.2.2 Recognition requirements and generic procedure

The sensitivity / invariance trade-off

Object recognition simultaneously requires:

- **Sensitivity** to (possibly small) image differences relevant to distinguishing one identity / category from another.
- **Insensitivity (invariance)** to (possibly large) image differences that do *not* affect identity / category.

Required invariances:

Source of variation	Example
Background clutter	Telephone amongst tangled cables
Occlusion	Person partially hiding a telephone
Viewpoint	Same chair seen from different angles
Lighting	Same face under bright vs dim illumination
Non-rigid deformation	Running horse, where the body bends
Within-category variation	Many different sofa-chair shapes

Generic recognition pipeline (off-line / on-line):

Object recognition procedure

1. **Off-line (training).** Extract a representation from each training image and store it as a feature vector $(a, b, c, \ldots)$.
2. **On-line (test).** Extract the representation from the input image in the same way.
3. **Match** the test representation against the stored ones to determine class / identity.

The recipe needs: image data, a representation of objects, and a matching technique.

Methods vary along three axes:

- **Features used.** Local features (parts) vs global features (whole-shape descriptions).
- **Representation.** 2D image-based vs 3D object-based; pixel intensities vs feature vectors vs geometry.

- **Matching procedure.** Top-down (generative, expectation-driven) vs bottom-up (discriminative, stimulus-driven); choice of similarity measure and decision criterion.

9.2.3 Template matching

Template matching

Representation: a ***template*** I_1 is an array of pixel intensities depicting the object to be recognised. **Matching:** for every template, slide it across the image; at each location compute a similarity score against the underlying patch I_2; report the best score (or all scores above a threshold).

Recognising multiple objects $\Rightarrow$ **multiple templates** (one per object / part: mouth, eye, nose, ...).

Similarity measures. Recall (from the correspondence-problem lecture):

Measure	Formula	Optim.
Cross-correlation	$\sum_{i,j} I_1(i,j)\, I_2(i,j) = I_1 \cdot I_2 = \lVert I_1 \rVert \lVert I_2 \rVert \cos\theta$	maximise
Normalised cross-correlation (NCC)	$\frac{\sum I_1 I_2}{\sqrt{\sum I_1^2}\sqrt{\sum I_2^2}} = \frac{I_1 \cdot I_2}{\lVert I_1 \rVert \lVert I_2 \rVert} = \cos\theta$	maximise
Correlation coefficient	$\frac{\sum (I_1 - \bar{I}_1)(I_2 - \bar{I}_2)}{\sqrt{\sum (I_1 - \bar{I}_1)^2}\sqrt{\sum (I_2 - \bar{I}_2)^2}}$	maximise
Sum of squared differences (SSD)	$\sum (I_1 - I_2)^2$	minimise
Euclidean distance	$\sqrt{\sum (I_1 - I_2)^2}$	minimise
Sum of absolute differences (SAD)	$\sum \lvert I_1 - I_2 \rvert$	minimise

Implementation note. Template matching using cross-correlation can be implemented using *convolution with a rotated template* (i.e. cross-correlation = convolution after 180° rotation of the kernel; cf. Week 3). The correlation coefficient equals NCC when both means are zero.

Why template matching fails (collected):

Problem	Why it bites
True vs false matches	Multiple peaks; ambiguous what counts as a match.
Template must be very similar	Even small changes in scale, rotation, viewpoint produce weak matches indistinguishable from false ones.
Within-category variation	Need many templates per object (e.g. many car viewpoints).
Threshold trade-off	High threshold $\Rightarrow$ miss true matches; low threshold $\Rightarrow$ many false positives.
Tractability	250×200 image, 5 objects, 30 viewpoints, 5 scales $\Rightarrow$ **37.5M** matches.
Occlusion	A partially hidden object fails to match.

Underlying issue. The pixel-intensity comparison is fundamentally not robust to appearance changes; invariance is hard to achieve.

Spot it in a question

"Search every region, compute similarity, threshold the best peak" ⇒ template matching. "Use NCC because it is invariant to linear illumination changes" ⇒ exam-favourite reason to choose NCC over plain cross-correlation.

9.2.4 Sliding window

Sliding window

Like template matching, but with two changes:

1. For each image patch, use a **classifier** (more tolerant to appearance changes) instead of a fixed-template similarity score.
2. Choose patches at *different shapes and sizes* and **resize** them before feeding to the classifier (further increases tolerance to scale).

Tractability. Classifying every patch of every size and shape (∼100k+ regions) is very expensive. Practical fix: pre-process the image with ***segmentation*** to propose candidate regions (∼1k regions) likely to contain an object.

9.2.5 Edge matching

Edge matching

Representation: like template matching, but *both* template and input are pre-processed to extract edges (Canny etc., cf. Week 3).
Matching: for every edge template T and every image region with edge image I, compute the average minimum point-to-edge distance:

$$D(T, I) \;=\; \frac{1}{|T|} \sum_{t \in T} d_I(t),$$

where $d_I(t)$ is the distance from template point t to the nearest image edge. Choose the minimum value as the best match.

Pros: more tolerant to lighting / colour changes than raw pixels (edges are intrinsic). **Cons:** still sensitive to scale, rotation, viewpoint; an edge map is noisy and contains many irrelevant edges (reflectance / illumination, cf. D-O-R-I from Week 3).

9.2.6 Model-based recognition

Model-based recognition

General idea (top-down / generative):

1. **Hypothesise** object identity and pose.
2. **Render (back-project)** the object into the image plane.
3. **Compare** to the image; accept if score is good.

Representation: 2D or 3D model of object shape.

Comparison scores:

Score	Question asked, and verdict
Edge score	Are there image edges near predicted object edges? *Very unreliable* – in textured regions the answer is usually yes regardless.
Oriented edge score	Same, but require matching orientations. Better, but still hard to tune.

9.2.7 Intensity histograms

Intensity / colour histograms

Representation: a histogram of pixel intensities (greyscale or colour).
Matching: compare histograms (e.g. via χ^2, Bhattacharyya, or other distance) to find the closest match.

Trade-offs:

Property	Comment
+ Fast, easy to compute	Simple histogram.
+ Insensitive to small viewpoint changes	Unlike templates.
− Sensitive to illumination	Pixel values shift with lighting.
− Sensitive to intra-class variation	Different colour distributions for the same class.
− Insensitive to spatial configuration	Two very different images can share the same histogram (an exam-favourite trap).

Application: face / skin detection. Skin has a small range of (intensity-independent) colours, so colour histograms are commonly used as part of face detection / recognition pipelines (segment skin pixels first, then look for face structure).

9.2.8 Implicit Shape Model (ISM)

ISM representation

Two components:

- **Parts**: 2D image fragments (patches around interest points).
- **Structure**: the configuration (spatial layout) of parts relative to the object centre.

Training pipeline:

1. For every training image, locate ***interest points*** (e.g. Harris detector).
2. Extract a 2D patch around each interest point; collect all such patches.

3. Cluster the patches (e.g. ***hierarchical agglomerative clustering***); store cluster centres as the ***appearance codebook***.
4. Match codebook entries to training images and, for each codebook entry, record the offset(s) to the object centre.

Matching pipeline:

1. Detect interest points in the input image.
2. For each interest point, find matching codebook entries.
3. Each matched feature ***votes*** for possible object centres (equivalent to the ***Generalised Hough Transform***).
4. Object centres with maximum votes are taken as hypotheses; back-project the contributing patches to localise the object.

Strengths and limits. Robust to translation, rotation, partial occlusion (since votes are local). *Failure mode*: ignores spatial constraints between parts, so spurious centres can arise (e.g. a phantom "cow" between two real cows in the worked example).

9.2.9 SIFT feature matching

SIFT descriptor (recap)

A **128-element** histogram of intensity-gradient orientations, formed by binning gradients in 4×4 pixel windows around the interest point into 8 orientations ($4 \times 4 \times 8 = 128$), normalised to unit length, and rotated so the dominant orientation is vertical.

Why SIFT works:

- **Locality**: features are local $\Rightarrow$ robust to occlusion and clutter (no prior segmentation).
- **Distinctiveness**: each 128-D vector can be matched against a large database.
- **Quantity**: many features even for small objects.
- **Efficiency**: close to real-time.

Matching protocol:

1. For each test keypoint descriptor, find its top *two* nearest neighbours in the training database (BFMatcher knn with $k = 2$).
2. Apply ***Lowe's ratio test***: accept the match if

$$\frac{d_1}{d_2} < \tau, \qquad \tau \approx 0.75,$$

   where d_1, d_2 are distances to the first / second nearest descriptors. The idea: a true match should be much closer than the runner-up; if both are similar, the match is ambiguous.

Geometric verification. Given three non-collinear model points P_1, P_2, P_3 and three image points p_1, p_2, p_3, there is a unique transformation (rotation, translation, scale) aligning model to image. Use ***RANSAC*** or the ***Generalised Hough Transform*** to check that the matched locations are geometrically consistent — i.e. explained by a single transformation. In practice (cf. the coding exercise), one fits a ***homography*** via `cv2.findHomography(..., cv2.RANSAC, 5.0)` and uses the inlier mask.

Spot it in a question

"Multiple matches per descriptor with a ratio threshold" ⇒ Lowe's ratio test. "Verify matches with a single homography" ⇒ RANSAC or Generalised Hough Transform.

9.2.10 Bag-of-Words (BoW)

Bag-of-words for documents (the analogy):

1. Parse documents into words ("the cat sat on the mat. a dog is sitting on the cat").
2. Ignore stop-words (the, a, on, it, ...) ⇒ "cat sat mat dog sitting cat".
3. Reduce to stems (sitting / sat / sits → sit) ⇒ "cat sit mat dog sit cat".
4. Assign each stem an ID (1 = cat, 2 = sit, 3 = mat, ...).
5. Represent each document as a K-dim frequency vector (K = vocabulary size), e.g. $(2, 2, 1, 1, 0, \ldots)$.

Matching: for query documents containing ≥ 1 query word, compute the angle (i.e. $\cos^{-1}$ NCC) between query and document vectors; rank.

Bag-of-words for images

1. **Choose features.** Sample image locations via a regular grid, an interest-point detector, or randomly.
2. **Encode features.** Compute a SIFT (or other) descriptor on the patch around each location.
3. **Build dictionary.** Cluster descriptors with ***k-means*** into K groups; cluster centres are ***visual words*** (codewords). Optionally drop the most frequent codewords (analogue of stop-words).
4. **Represent images.** Each image becomes a K-bin histogram of visual-word occurrences.
5. **Match.** Compare histograms by distance / angle.

Mapping to the document analogy:

Documents	**Images**
Word	Image patch
Stem of a word	k-means cluster centre (visual word)
Vocabulary	Codeword dictionary
Stop-word removal	Drop most-frequent codewords
TF vector per document	Histogram of visual words per image

Strengths. Robust to occlusion / clutter (local features), simple histogram representation, scales to large databases. **Limit.** Discards spatial layout (just like intensity histograms): two images with the same parts in different arrangements look identical.

9.2.11 Geometric invariants

Geometric invariant

A property of an object in the scene that does *not* change under a given group of viewpoint transformations.

Hierarchy of transformation groups (most → least restrictive):

Group	Transformations	Invariants
Euclidean	translation, rotation	lengths, angles, areas
Similarity	+ uniform scale	ratios of lengths, angles
Affine	+ shear (non-uniform scale)	parallelism, ratios of lengths *along a line*, ratios of areas
Projective	+ foreshortening (full perspective)	**cross-ratio**

Cross-ratio

For four collinear points P_1, P_2, P_3, P_4:

$$\mathrm{CR}(P_1, P_2, P_3, P_4) \;=\; \frac{\|P_3 - P_1\|\,\|P_4 - P_2\|}{\|P_3 - P_2\|\,\|P_4 - P_1\|}.$$

Under any perspective projection, the cross-ratio of the four points and of their projected images p_1, p_2, p_3, p_4 are equal.

Recognition with invariants.

- **Representation**: value of the cross-ratio (or other invariant).
- **Matching**: compare the measured invariant to a database of training values.

Problems. Occlusion (need all four collinear points visible), availability and distinctiveness of points (a single cross-ratio is not very discriminative on its own).

9.2.12 Classification of methods

Axis 1: matching procedure.

Style	Examples	Idea
Top-down (generative)	model-based, templates, edge matching	Hypothesise an object and look for it; expectation-driven; "fitting".
Bottom-up (discriminative)	SIFT, BoW, intensity histograms, ISM	Extract a description and look it up in the database; stimulus-driven.

Axis 2: representation used.

Representation	Methods
Pixel intensities	templates, edge matching, intensity histograms, ISM
Feature vectors	SIFT feature matching, bag-of-words
Geometry	model-based, geometric invariants
2D (image-based)	templates, sliding window, ISM, bag-of-words
3D (object-based)	3D models, geometric invariants, SIFT (used for consistency check)
Local features	SIFT (and BoW), ISM, part templates
Global features	whole-object templates, sliding window, 3D models

Local vs global, head-to-head.

Aspect	Local features	Global features
Idea	Object = collection of simple features (e.g. "A = /+\+−").	Object = description of overall shape (e.g. "A = A").
+ Tolerance	Viewpoint, within-class variation, occlusion.	Can distinguish similar objects with the same parts.
− Failure 1	Different objects can share the same set of features (T, L, + all = "| + −").	Sensitive to viewpoint and within-class variation (need exact alignment).
− Failure 2	With many objects in the image, no part-to-object assignment (e.g. TA vs IV give the same multiset).	Sensitive to occlusion (every part of the description matters).
Failure rate	Many false positives.	Many false negatives.

Compromise. Use features of *intermediate complexity*, or a *hierarchy* of features at different complexities — combining locality (robustness) with structure (discriminative power).

9.3 Worked Examples

Example 1: Three recognition tasks

Q: Distinguish identification, classification, and localisation. Where does "semantic segmentation" sit?

A:

- **Identification**: which *instance*? (Clinton vs Bush.)
- **Classification**: which *category*? (human vs chimp.)
- **Localisation**: *where* in the image? (bounding box / label.)

Semantic segmentation arises when localisation is fine-grained *and* performed for many categories: every pixel gets a class label. It differs from earlier-week (binary) segmentation by labelling multiple classes simultaneously.

Example 2: Basic-level category

Q: Why is the "basic level" privileged in human object recognition? Place the categories {animal, dog, poodle, Rex} on the hierarchy.

A: The basic level is the *highest* level at which members share many features and the *lowest* level at which they are distinct from peer categories — making it perceptually homogeneous and easy to discriminate. Humans recognise basic-level categories fastest, classify there before identifying, and learn those names first as children.
Hierarchy: *animal* (superordinate) → *dog* (basic) → *poodle* (subordinate) → *Rex* (identification).

Example 3: Pick a similarity measure

Q: You template-match a logo across an image where lighting may have rescaled all intensities by an unknown factor. Which similarity measure should you use, and why?

A: Use **NCC** (or the correlation coefficient if mean-shift is also possible). NCC equals $\cos\theta$ between the two patches viewed as vectors, so it is invariant to a positive multiplicative factor on either vector — exactly the linear-illumination case. Plain cross-correlation is not, since multiplying intensities by k scales the score by k.

Example 4: Why template matching fails on chairs

Q: A chair template gives a clean NCC peak in image A but a noisy response in image B (same chair, different viewpoint). Explain. Suggest two fixes consistent with the methods covered.

A: Template matching's pixel-level metric is *not* robust to scale, rotation, viewpoint or within-class variation. Image B differs from the template in viewpoint, so every patch is a poor match and the peak vanishes into the noise.
Fixes: (i) use **multiple templates** per object covering different viewpoints and scales (with the trade-off of ∼37.5M comparisons in the lecture's example); (ii) move to a **sliding window** with a learned classifier that tolerates appearance changes, or to a **local-feature** pipeline (SIFT / BoW / ISM) that does not rely on global pixel alignment.

Example 5: Lowe's ratio test

Q: For a SIFT descriptor in the test image, the two best matches in the database have L2 distances $d_1 = 0.30$ and $d_2 = 0.36$. (a) With ratio threshold $\tau = 0.75$, is this match accepted? (b) If $d_1 = 0.30, d_2 = 0.42$, is it accepted? (c) Why is the ratio used rather than d_1 alone?

A:

- (a) $d_1/d_2 = 0.83 > 0.75 \Rightarrow$ **rejected** (ambiguous: the runner-up is almost as close).
- (b) $d_1/d_2 = 0.71 < 0.75 \Rightarrow$ **accepted**.
- (c) An absolute threshold on d_1 is unreliable because descriptor distances vary across keypoints. The *ratio* encodes *distinctiveness* of the match relative to its competitors; a true match should be much closer than the second-best.

Example 6: Building a BoW representation

Q: You have 1000 training images. Outline the steps to build a 500-word visual vocabulary and represent a new image as a feature vector.

A:

1. For each training image, sample feature locations (regular grid, Harris / SIFT detector, or random) and compute a SIFT descriptor at each.
2. Pool all descriptors and run **k-means** with $K = 500$; cluster centres are the visual words / codeword dictionary.
3. Optionally drop the most frequent codewords (BoW analogue of stop-words).
4. For a new image: compute SIFT descriptors at sampled locations, assign each to its nearest codeword, and form the 500-bin frequency **histogram**.
5. Match by comparing histograms (e.g. via $\cos\theta = \text{NCC}$ between the two TF vectors).

Caveat: histogram representations discard spatial layout — two images with the same parts arranged differently will look identical (a known weakness of BoW and intensity histograms).

Example 7: ISM voting

Q: In an ISM, three matched codebook entries vote for object centres at pixel locations $(50, 60), (51, 59), (200, 30)$. There is one true cow at the first cluster of votes and one false hypothesis. Explain how voting localises the cow and why a phantom cow can appear between two real cows.

A: Each codebook match contributes a ballot for the predicted object centre (Generalised Hough Transform). Genuine matches from the same cow vote consistently for nearby pixels, producing a peak; the back-projected patches at that peak reconstruct the cow.
With *two* cows present, parts of cow 1 may also vote (via codebook entries) for centres consistent with cow 2's position, and vice versa. The combined votes can pile up at a location *between* the two real cows, creating a phantom hypothesis. ISM votes do not enforce the spatial relationship between distinct parts, so this failure is structural.

Example 8: Geometric invariants under transformations

Q: For each transformation group, state which of {lengths, angles, length ratios, parallelism, cross-ratio} are invariant.

A:

- **Euclidean** (translation, rotation): lengths, angles, length ratios, parallelism, cross-ratio all preserved.
- **Similarity** (+ uniform scale): lengths *not* preserved (scaled), but length ratios, angles, parallelism, cross-ratio preserved.
- **Affine** (+ shear / non-uniform scale): angles *not* preserved, but parallelism, length ratios *along a line*, area ratios, cross-ratio preserved.
- **Projective** (+ foreshortening): only the **cross-ratio** preserved.

The hierarchy is nested: the cross-ratio survives all four; angles only survive similarity-and-below.

Example 9: Cross-ratio computation

Q: Four collinear scene points have positions $1, 2, 3, 5$ along a ruler. Their projected images sit at $0, 1, 1.5, 2$ on the image line. (a) Compute the cross-ratio in scene and image. (b) What does the result tell you?

A:

- **Scene:** $\mathrm{CR} = \dfrac{(P_3 - P_1)(P_4 - P_2)}{(P_3 - P_2)(P_4 - P_1)} = \dfrac{(3-1)(5-2)}{(3-2)(5-1)} = \dfrac{2 \cdot 3}{1 \cdot 4} = \dfrac{6}{4} = 1.5.$
- **Image:** $\mathrm{CR} = \dfrac{(1.5-0)(2-1)}{(1.5-1)(2-0)} = \dfrac{1.5 \cdot 1}{0.5 \cdot 2} = \dfrac{1.5}{1} = 1.5.$

The two cross-ratios match, as expected: the cross-ratio is a projective invariant. To recognise the same configuration of four collinear points across viewpoints, store CR = 1.5 in the database and compare.

Example 10: Top-down vs bottom-up classification

Q: Classify each method as top-down or bottom-up, and justify.

1. Model-based (3D CAD) recognition.
2. SIFT + Lowe + RANSAC.
3. Bag-of-words + nearest-neighbour histogram.
4. Edge matching with a stored template.

A:

- (1) **Top-down** (generative): hypothesise pose, render, compare. Expectation-driven.
- (2) **Bottom-up** (discriminative): extract SIFT descriptors, look them up; geometric verification only confirms a hypothesis built from the data.
- (3) **Bottom-up**: extract a histogram and look it up.

- (4) **Top-down**: a stored object template is hypothesised in each image region; the comparison is the verification step.

Example 11: Coding – template matching with NCC

Q: (LGT Coding Q1) Complete `find_object_template` using OpenCV's `matchTemplate` with NCC and report the bounding box of the best match.

A:

```
def find_object_template(image, template):
    h, w = template.shape[:2]
    result = cv2.matchTemplate(image, template, cv2.TM_CCOEFF_NORMED)
    _, _, _, max_loc = cv2.minMaxLoc(result)
    top_left     = max_loc
    bottom_right = (top_left[0] + w, top_left[1] + h)
    return top_left, bottom_right
```

Why `TM_CCOEFF_NORMED`. It is the correlation coefficient (mean-subtracted NCC), so it is invariant to additive *and* multiplicative linear illumination changes. We take the *maximum* (NCC is a maximise-this measure), unlike SSD/SAD where we would take the minimum.

Example 12: Coding – SIFT + Lowe + RANSAC

Q: (LGT Coding Q2/Q3) Sketch the pipeline that (a) finds robust SIFT matches between two images and (b) localises the object via a homography.

A:

```
# (a) SIFT + Lowe's ratio test
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
matches = bf.knnMatch(des1, des2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]

# (b) Homography via RANSAC, then project model corners
src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
h, w = img1.shape
corners = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-1, 1, 2)
proj = cv2.perspectiveTransform(corners, H)
```

Why each piece. `NORM_L2` is the correct distance for SIFT (Euclidean on 128-D unit vectors). `knnMatch` with $k = 2$ is needed for Lowe's ratio. `findHomography` with RANSAC discards outliers (need ≥ 4 point pairs for a homography). `perspectiveTransform` maps the model corners into the scene to draw the bounding polygon.

9.4 Summary

Key Takeaways

1. **Three recognition tasks**: identification (instance), classification (category), localisation (where). Fine-grained, multi-class localisation = semantic segmentation.
2. **Sensitivity vs invariance**: must distinguish small relevant differences yet ignore large irrelevant ones (clutter, occlusion, viewpoint, lighting, deformation, within-class).
3. **Generic recipe**: image data + representation + matching. Methods differ on the three axes: top-down vs bottom-up, representation type, local vs global features.
4. **Template matching**: simple but brittle. Use NCC for linear illumination invariance. Multiple templates needed for multiple viewpoints/scales (combinatorial blow-up).
5. **Sliding window**: classifier instead of similarity; resize patches; use segmentation to propose candidate regions ($\sim$1k vs $\sim$100k+).
6. **Edge matching**: pre-process with edges; minimise average distance from template edge points to image edges.
7. **Model-based**: hypothesise pose $\rightarrow$ back-project $\rightarrow$ compare. Edge or oriented-edge score; oriented is better.
8. **Intensity / colour histograms**: fast, viewpoint-tolerant, but discard spatial layout and are sensitive to illumination. Used for face / skin detection.
9. **ISM**: parts + structure; codebook by clustering; voting via Generalised Hough Transform; robust to translation/rotation/occlusion but ignores inter-part spatial constraints.
10. **SIFT matching**: 128-D descriptors + Lowe's ratio test + RANSAC / GHT geometric verification; locality, distinctiveness, quantity, efficiency.
11. **Bag-of-words**: documents-to-images analogy; k-means visual vocabulary; histogram representation; loses spatial layout.
12. **Geometric invariants**: Euclidean (length, angle, area), similarity (length ratios, angles), affine (parallelism, length ratios, area ratios), projective (cross-ratio). Cross-ratio is the only invariant under full perspective.
13. **Local vs global**: local = many false positives, global = many false negatives; intermediate-complexity / hierarchical features compromise.

Final comparison: nine classical methods.

Method	Style	Repr.	Key strength / weakness
Template matching	top-down	pixels (global)	− scale/rotation/occlusion brittle
Sliding window	either	pixels $\rightarrow$ classifier	+ tolerant via classifier; − tractability
Edge matching	top-down	edges (global)	+ illumination tolerant; − noisy edges
Model-based	top-down	2D/3D shape	+ explicit pose; − unreliable scores
Intensity histograms	bottom-up	global pixel stats	+ fast; − ignores spatial layout
ISM	bottom-up	local patches + votes	+ occlusion-robust; − phantom centres
SIFT matching	bottom-up	local features (128-D)	+ scale/rotation invariant; needs verification
Bag-of-words	bottom-up	local features $\rightarrow$ histogram	+ scalable; − no spatial info
Geometric invariants	either	geometry	+ viewpoint invariant; − occlusion / point availability

Final comparison: similarity measures.

Measure	**Optim.**	**Invariances**
Cross-correlation	max	none beyond same-scale comparison
NCC	max	invariant to multiplicative illumination changes
Correlation coefficient	max	invariant to additive *and* multiplicative changes
SSD / Euclidean	min	none
SAD	min	none; cheaper than SSD; less penalty for outliers

10 Week 10: Biological Object Recognition (High-Level Vision)

10.1 Topic Summary

Learning Objectives

By the end of this week, you will be able to:

- Distinguish ***object-based*** (3D, object-centred) from ***image-based*** (2D, viewer-centred) theories of object recognition.
- Describe ***recognition-by-components*** (Biederman) and define a ***geon*** and a ***structural description***.
- Distinguish ***configural*** (global/holistic) from ***featural*** (local) processing and explain the ***face inversion effect***.
- Compare ***rules***, ***prototypes***, and ***exemplars*** as theories of categorisation, and link them to nearest-mean and (k-)nearest-neighbour classifiers.
- Describe the cortical ***what*** (ventral, V1→IT) and ***where*** (dorsal, V1→parietal) pathways.
- Explain how RF size, location sensitivity, and stimulus complexity change along the ventral pathway.
- Describe ***HMAX*** and ***CNN*** feed-forward hierarchies, the role of alternating S- and C-layers, and the SUM/MAX operations.
- Distinguish feed-forward from ***recurrent*** models and contrast ***bottom-up*** (stimulus-driven, discriminative) with ***top-down*** (knowledge-driven, generative) processes.
- State ***Bayes' theorem***, identify posterior/likelihood/prior/evidence in a vision context, and use the posterior ratio to choose between hypotheses.

Big Picture. This week tackles *high-level vision* from a biological/cognitive perspective: how does the brain go from a retinal image to an object label? Two big debates frame the field. (1) ***Object-based vs image-based***: do we store one 3D model per object (recognition by components, geons, structural descriptions) or multiple 2D views (template / multiple-views models)? (2) ***Configural vs featural***: do we represent objects globally (holistic, e.g. upright faces) or locally (independent features, e.g. inverted faces)? In categorisation, three psychological accounts—***rules***, ***prototypes***, ***exemplars***—map onto standard ML classifiers (decision trees, nearest-mean, k-NN). The cortical machinery is hierarchical: along the ventral pathway (V1→V2→V4→IT) RFs grow larger, less location-sensitive and more complex. ***HMAX*** and CNNs make this concrete using alternating ***simple*** (SUM, "and"-like, increase selectivity) and ***complex*** (MAX, "or"-like, increase invariance) layers. Pure feed-forward is not enough: ***recurrent*** (lateral + feedback) connections combine bottom-up evidence with top-down priors, formalised by ***Bayesian inference***: $p(\text{obj} \mid I) \propto p(I \mid \text{obj})\, p(\text{obj})$.

What Examiners Like to Ask:

- Define viewer-centred vs object-centred; geon; structural description (LGT Q1, Q2).
- Classify a new feature vector with nearest mean, 1-NN and k-NN; compute distances by hand (LGT Q3).
- State how RF size, location sensitivity, and stimulus complexity change along the ventral pathway (LGT Q4).
- Describe inputs to simple vs complex cells and the operation each performs (LGT Q5, Q6).
- Write Bayes' theorem and interpret each term for vision (LGT Q7).
- Use Bayes / posterior ratio to decide which object an ambiguous image depicts (LGT Q8).
- Compare bottom-up (discriminative) vs top-down (generative); explain why vision is an ill-posed inverse problem.

- Compare HMAX with CNN; explain MAX pooling as biological motivation for CNN sub-sampling.

10.2 Core Concepts

Roadmap:

1. Theories of object recognition: object-based (3D) vs image-based (2D); configural vs featural.
2. Theories of categorisation: rules, prototypes, exemplars; mapping to ML classifiers.
3. Cortical pathways: what (ventral) vs where (dorsal); hierarchy of receptive fields.
4. Feed-forward hierarchical models: HMAX, CNN.
5. Recurrent models: lateral + feedback; bottom-up vs top-down.
6. Bayesian inference: posterior, likelihood, prior, evidence; priors and ill-posed vision.

10.2.1 Object-Based vs Image-Based Theories

Object-based (3D, object-centred)

Each object is represented by storing a single 3D model in an ***object-centred*** reference frame (independent of viewpoint). Recognition decomposes the input image into 3D components and matches them against stored 3D descriptions.

Image-based (2D, viewer-centred)

Each object is represented by storing multiple 2D ***views*** (images) in a ***viewer-centred*** reference frame. Recognition is template-matching against the stored views (with interpolation for novel viewpoints).

Recognition-by-Components (Biederman). Hypothesis: a small "alphabet" of 3D primitives (***geons***) and their inter-relations form the building blocks of object recognition—like letters forming words.

Geon

A simple, view-invariant 3D shape primitive: cubes, spheres, cylinders, cones, wedges, etc. Geons are chosen to be (i) sufficiently distinct from each other, (ii) robust to noise/partial occlusion, and (iii) view-invariant (look similar from most viewpoints).

Structural Description

A representation of an object as a list of its component geons *plus* their relative locations and sizes (e.g. "cube above cylinder"; "lamp shade left-of, below, larger-than fixture"). Different views of an object share the same set of geons in the same arrangement $\Rightarrow$ ***viewpoint invariance***.

RBC pipeline: Early image processing $\rightarrow$ part segmentation $\rightarrow$ part modelling (assign each part a geon) $\rightarrow$ build structural description $\rightarrow$ match against stored structural descriptions.

Problems with object-based / RBC:

- Hard to decompose an image into components (segmentation is ill-posed).
- Many natural objects (a tree, a face, a cloud) have no clean parts-based decomposition.
- Cannot capture *fine* details $\Rightarrow$ cannot discriminate individuals or similar objects (e.g. two specific faces).

Image-based variants:

Variant	Idea / Status
Template matching	Early version: store one 2D template per object. Too rigid for the flexibility of human recognition.
Multiple views	Modern version: store multiple stored 2D views per object; novel viewpoints handled by *interpolation* between stored views.

Spot it in a question

"Object-centred", "single 3D model", "geons", "structural description" ⇒ object-based. "Viewer-centred", "multiple 2D views", "template", "interpolation between stored views" ⇒ image-based.

10.2.2 Configural vs Featural Theories

Featural (local)

Features are processed independently; relationships between features are ignored. Local properties dominate.

Configural (global / holistic)

Object is processed as a whole; the spatial *configuration* (relationships between features) is critical.

Face inversion effect. Upright faces are processed ***configurally***; inverted faces are processed ***featurally***. This explains why we struggle to spot grotesque alterations in upside-down faces (Thatcher illusion): configural processing is disabled, and individual features look fine in isolation.

Trade-off. Local (featural) and global (configural) representations have complementary strengths and weaknesses:

Representation	Strength	Weakness (failure modes)
Simple / local (featural)	Robust to occlusion, viewpoint	Many *false positives*: confuses objects with similar features in different arrangements; clutter
Complex / global (configural)	Captures whole-object structure	Many *false negatives*: fragile to occlusion, viewpoint, within-class variation

Solutions: (1) use features of *intermediate* complexity, or (2) use a *hierarchy* of features with a range of complexities—which is exactly what cortex appears to do.

10.2.3 Categorisation: Rules, Prototypes, Exemplars

How are category boundaries defined? How are new stimuli assigned?

Rules

Category membership is defined by abstract rules (e.g. "has three sides ⇒ triangle"; "has four legs and barks ⇒ dog"). Anything satisfying the rule(s) is in the category.

For: explains over-extension of rules in language ("goed", "mouses"). **Against:** category membership is graded (a bear is a better mammal than a whale; a pigeon is a better bird than a penguin) which rules cannot capture.

Prototypes

Compute the average of all instances per category to form a ***prototype***. Assign a new stimulus to the category whose prototype is nearest.

For: prototypical members are accessed faster and learnt more easily. **Against:** variations within a class cannot be represented (only the mean is stored).

Exemplars

Store specific individual instances (***exemplars***) of each category. Assign a new stimulus to the category of the nearest stored exemplar.

For: successfully predicts certain mis-categorisations (e.g. a whale as a fish, by similarity). **Against:** graded membership still tricky to reproduce purely from raw exemplars.

Mapping to machine learning (these are all supervised: class labels of training points are known):

Psychology	ML method	Stored
Rules	Decision tree	A tree of feature thresholds
Prototypes	Nearest mean classifier (NMC)	One mean vector per class
Exemplars	Nearest neighbour (1-NN)	All training feature vectors
Exemplars	K-nearest neighbours (k-NN)	All training feature vectors

Nearest Mean Classifier (Prototype)

For each class, compute the ***mean*** of the feature vectors of all training points in that class. Assign a new stimulus to the class whose mean is closest. **Decision boundary is linear** $\Rightarrow$ only suitable when classes are linearly separable.

Nearest Neighbour Classifier (Exemplar, 1-NN)

Save all training feature vectors. Assign a new stimulus to the class label of the closest training exemplar. **Decision boundary is non-linear (piecewise linear)** and forms a ***Voronoi partitioning*** of feature space. Sensitive to outliers.

k-Nearest Neighbours Classifier (k-NN)

Save all training feature vectors. Assign a new stimulus to the class label of the *majority* among its k closest training exemplars. **Decision boundary is non-linear.** k is typically small and *odd* (to break ties); larger k reduces the effect of outliers but smooths fine boundaries.

Similarity / distance measures (used for "closest"):

- Minimise distance: Sum of Squared Differences (SSD), Euclidean distance, Sum of Absolute Differences (SAD = Manhattan).
- Maximise similarity: cross-correlation, normalised cross-correlation, correlation coefficient.

Exam Tip

"Linear decision boundary" or "mean of each class" ⇒ NMC (prototype). "Voronoi partitioning" or "stores all training points" ⇒ 1-NN (exemplar). "Majority vote among nearest" ⇒ k-NN. Always state Euclidean distance unless specified otherwise.

Supervised vs unsupervised reminder. Prototype/exemplar methods are *supervised* (class labels known in training). ***Clustering*** (k-means, agglomerative, graph cuts—seen in segmentation week) is *unsupervised*: labels assigned by similarity alone.

10.2.4 Cortical Visual System: Pathways

Beyond V1, cortex has two pathways:

Pathway	Route	Function
What (ventral)	V1 → V2 → V4 → IT (inferotemporal)	Object identity / category
Where / How (dorsal)	V1 → parietal cortex (via MT)	Spatial / motion information

Both pathways are ***hierarchically organised***: simple, local RFs at V1; complex, large RFs in higher areas.

Hierarchy of receptive fields (along ventral pathway):

Three trends along the ventral pathway

As we move from peripheral (V1) toward higher areas (IT):

1. **Receptive field size** *increases*.
2. **Sensitivity to stimulus location** *decreases* (greater positional invariance).
3. **Complexity of preferred stimulus** *increases*.

Concrete cell hierarchy (LGN to V1, then beyond):

$$\text{Eye} \to \text{LGN} \to \text{V1}: \quad \text{Centre-surround cells} \to \text{Simple cells} \to \text{Complex cells} \to \cdots$$

Cell type	Preferred stimulus
Centre-surround (LGN)	Isolated spots of contrasting intensity / colour.
Simple (V1)	Edges/bars at a specific orientation, with specific contrast polarity, at a specific location.
Complex (V1)	Edges/bars at a specific orientation with *any* polarity at *any* position within a small region.
⋮	Increasingly complex feature combinations (corners, junctions, parts, faces).
End of ventral (IT)	Very complex stimuli (e.g. faces, body parts), large RFs, location-invariant.

How more complex RFs are built. Each level combines outputs from cells at the previous level with simpler RFs. For example:

- A **simple cell** sums multiple co-aligned centre-surround cells ⇒ responds when a bar/edge at the right orientation activates them all.

- A **complex cell** pools several simple cells with the same orientation preference but different positions ⇒ responds whenever any one of those simple cells fires (orientation kept, position tolerated).

Spot it in a question

"Multiple co-aligned centre-surround cells" as input ⇒ simple cell. "Multiple simple cells with the same orientation, slightly different positions" as input ⇒ complex cell. "Larger RF, more invariance" is the general direction along the ventral pathway.

10.2.5 Feed-Forward Hierarchical Models: HMAX and CNN

Overall idea. Image is processed by layers of neurons with progressively more complex RFs at progressively less specific locations. Features at one stage are built from features at earlier stages. Can be thought of as ***hierarchical template matching***.

HMAX (Riesenhuber & Poggio).

HMAX: alternating S- and C-layers

Two distinct mathematical operations are needed: one to increase *selectivity* (build more complex RFs), and one to increase *invariance* (tolerate position/scale changes). HMAX therefore alternates two layer types, named after V1 cell types:

Unit	Computation	Logical role	Effect
Simple "S-cells"	SUM	"and"-like	Increased selectivity
Complex "C-cells"	MAX	"or"-like	Increased invariance

Wiring:

- S-cells in one layer respond to *conjunctions* of C-cells in the previous layer (template / "and").
- C-cells in one layer respond to *any* S-cell in a small neighbourhood of the previous layer (pooling / "or").

Two regimes inside HMAX:

Stage	Description
V1 → IT	Generic, reusable representations of shape components. "Hard-wired".
IT → PFC	Task-specific. Trained using supervised learning (a classifier on top).

CNN as a near-equivalent of HMAX.

CNN as hierarchical model

A Convolutional Neural Network alternates two operations:

- **Convolution** (with learned filters) – equivalent to the SUM / "and" / S-cell operation.
- **Sub-sampling / pooling** (often *max* pooling over a small neighbourhood) – equivalent to the MAX / "or" / C-cell operation, reducing location specificity.

This is the same hierarchical recipe as HMAX, implemented with standard image-processing primitives.

Confusing CNN vs HMAX naming

In CNN literature, "C layer" = *convolution* layer (which corresponds to HMAX's S-cell layer); "S layer" = *sub-sampling* layer (which corresponds to HMAX's C-cell layer). The *names* are flipped relative to HMAX. The *operations* (sum vs max) match.

Status. Both HMAX and CNNs are examples of *purely serial, feed-forward* models of cortical information processing. Deep CNNs achieve state-of-the-art on many vision tasks.

10.2.6 Recurrent Models and Top-Down Processing

Real cortex is not strictly feed-forward. Two kinds of recurrent connections exist:

1. **Lateral connections within a region** (excitatory + inhibitory): allow neurons in the same population to interact (basis of the contextual / non-classical RF effects from Week 4—contour integration, surround suppression, pop-out).
2. **Feedback connections from higher to lower areas**: convey information from higher cortical regions back to primary sensory areas; allow bottom-up evidence to interact with top-down knowledge.

Bottom-Up vs Top-Down

Bottom-up: use the information *in the stimulus itself* to identify what is there. Stimulus-driven; ***discriminative***.

Top-down: use ***context***, prior knowledge and expectation to aid identification. Knowledge-driven; ***generative***.

Process	Driven by	Models
Bottom-up	Stimulus	Discriminative (model the posterior directly)
Top-down	Knowledge / context / priors	Generative (model likelihood and prior)

Recurrent networks combine the two; ***Bayesian inference*** formalises the optimal way to combine them.

10.2.7 Bayesian Inference

Bayes' Theorem

$$p(A \mid B) \;=\; \frac{p(B \mid A)\, p(A)}{p(B)}.$$

The bar "|" reads "given". The two conditional probabilities $p(A \mid B)$ and $p(B \mid A)$ are *not* equal in general (e.g. $p(\text{rain} \mid \text{wet pavement}) < 1$ but $p(\text{wet pavement} \mid \text{rain}) = 1$).

Vision is an inverse problem. We observe an image I (the outcome) and want the object obj_j (the cause). $p(\text{obj}_j \mid I)$ is hard to compute directly (inverse problem). But $p(I \mid \text{obj}_j)$ is comparatively *easy*

– it follows from the forward image-formation model (project the 3D object, simulate illumination, etc.). Bayes inverts:

$$\boxed{p(\text{obj}_j \mid I) \;=\; \frac{p(I \mid \text{obj}_j)\, p(\text{obj}_j)}{p(I)}}$$

Bayes nomenclature for vision

- **Posterior** $p(\text{obj}_j \mid I)$ – what we want: probability that object j is present given image I.
- **Likelihood** $p(I \mid \text{obj}_j)$ – forward model: probability of observing I given that object j is present.
- **Prior** $p(\text{obj}_j)$ – prior knowledge: probability that object j is present in this environment, before seeing I.
- **Evidence** $p(I)$ – the same for all hypotheses about *this* image; can be ignored when comparing them.

If we ignore $p(I)$ (or set $p(I) = 1$), then **posterior** $\propto$ **likelihood** $\times$ **prior**.

Posterior ratio (binary hypothesis "object vs not object"). Useful when we just want a yes/no decision:

$$\underbrace{\frac{p(\text{obj}_j \mid I)}{p(\neg\text{obj}_j \mid I)}}_{\text{posterior ratio}} = \underbrace{\frac{p(I \mid \text{obj}_j)}{p(I \mid \neg\text{obj}_j)}}_{\text{likelihood ratio}} \cdot \underbrace{\frac{p(\text{obj}_j)}{p(\neg\text{obj}_j)}}_{\text{prior ratio}}$$

Decide "object" iff posterior ratio > 1; the evidence $p(I)$ cancels out.

Why priors matter (vision is ill-posed). Multiple causes can produce the same image $\Rightarrow$ likelihood alone is ambiguous. The visual system uses ***priors*** (assumptions / constraints on the world) to choose the most likely interpretation. Examples:

- Prior: *light comes from above* $\Rightarrow$ infer convex/concave shape from shading.
- Prior: *texture is circular and homogeneous* $\Rightarrow$ infer 3D shape and depth.
- Prior: *faces are convex* $\Rightarrow$ even a concave (hollow) mask appears convex.
- Prior: *size is constant* $\Rightarrow$ infer depth from retinal size (e.g. Ponzo illusion).
- Prior: *neighbouring / similar / connected features are related* $\Rightarrow$ Gestalt grouping (Week 4).
- Prior: *strings of letters form words* $\Rightarrow$ infer letter identity from context.

10.2.8 Big Picture: How Cortex Combines These Ideas

Hierarchy of features at multiple complexities. Cortex uses a hierarchy of features whose complexity grows with depth, modelled by alternating layers that increase *selectivity* (sum / "and") and *invariance* (max / "or"). This solves the local-vs-global trade-off by spanning all scales of complexity simultaneously.

Bottom-up + top-down via recurrence. Lateral and feedback connections allow stimulus-driven evidence (likelihoods) to combine with knowledge-driven priors. Bayes' theorem describes the optimal combination.

Discriminative vs generative. Bottom-up / discriminative methods directly model the posterior (e.g. a CNN classifier output). Top-down / generative methods model the likelihood and prior separately and use Bayes to obtain the posterior.

10.3 Worked Examples

Example 1: Viewer-centred vs object-centred (LGT Q1)

Q: What is the difference between the "viewer-centred" and "object-centred" approaches to object recognition?

A:

- **Viewer-centred (image-based):** the 3D object is modelled as a set of 2D images, showing different views of the object.
- **Object-centred (object-based):** a single 3D model is used to describe the object, independent of viewpoint.

Example 2: Geon and structural description (LGT Q2)

Q: What is a geon? What is a structural description?

A:

- **Geon:** a simple three-dimensional shape such as a sphere, cube, cylinder, cone or wedge; chosen to be sufficiently distinct, robust, and view-invariant.
- **Structural description:** a representation of an object in terms of its component geons *and* their relative locations and sizes (e.g. "cube above cylinder").

Example 3: NMC, 1-NN and 3-NN classification (LGT Q3)

Q: Four objects from two classes are encoded as:

Object	Class	Feature vector
1	A	$(7,7)$
2	A	$(7,4)$
3	B	$(3,4)$
4	B	$(1,4)$

A new object has feature vector $(3,7)$. Classify it using (1) nearest mean, (2) 1-NN, (3) 3-NN. Use Euclidean distance.

A:

(1) Nearest mean (Prototype). Prototypes:

$$\mu_A = \left(\tfrac{7+7}{2}, \tfrac{7+4}{2}\right) = (7, 5.5), \quad \mu_B = \left(\tfrac{3+1}{2}, \tfrac{4+4}{2}\right) = (2, 4).$$

Distances from $(3,7)$:

$$d(\mu_A, (3,7)) = \sqrt{(7-3)^2 + (5.5-7)^2} = \sqrt{16 + 2.25} = \sqrt{18.25} \approx 4.27$$

$$d(\mu_B, (3,7)) = \sqrt{(2-3)^2 + (4-7)^2} = \sqrt{1+9} = \sqrt{10} \approx 3.16$$

μ_B is closer $\Rightarrow$ **class B**.

(2) 1-NN (Exemplar). Distances from $(3,7)$ to each training point:

Object	Class	Dist
1 $(7,7)$	A	$\sqrt{16+0} = 4$
2 $(7,4)$	A	$\sqrt{16+9} = 5$
3 $(3,4)$	B	$\sqrt{0+9} = 3$
4 $(1,4)$	B	$\sqrt{4+9} \approx 3.6$

Closest exemplar is object 3 (class B) ⇒ **class B**.

(3) 3-NN. Three closest are 3 (B), 4 (B), 1 (A). Majority = B ⇒ **class B**.

Example 4: Trends along the ventral pathway (LGT Q4)

Q: How do the following properties of cortical neurons change when moving along the ventral pathway from peripheral to higher areas: (1) RF size, (2) sensitivity to stimulus location, (3) complexity of preferred stimulus?

A:

- (1) RF size **increases**.
- (2) Sensitivity to stimulus location **decreases** (greater invariance).
- (3) Complexity of preferred stimulus **increases**.

Example 5: Inputs to simple vs complex cells (LGT Q5)

Q: Describe the inputs to (a) a simple cell and (b) a complex cell, and explain how these give rise to the observed response properties.

A:

- **(a) Simple cell:** input is from several centre-surround cells whose RFs lie on a common line. When a bar/edge at the correct orientation falls on this line, all the centre-surround inputs are activated together, so the simple cell responds. Hence simple cells respond to oriented bars/edges at a specific orientation *and* location.
- **(b) Complex cell:** input is from several simple cells that share the same orientation preference but are tuned to slightly different positions within a small spatial region. A bar/edge at the right orientation will activate *some* of these simple cells regardless of exact position, so the complex cell responds. Hence complex cells respond to oriented edges with tolerance to exact location (positional invariance).

Example 6: HMAX operations (LGT Q6)

Q: Describe the mathematical operation performed by neurons in the HMAX model in (1) simple cells, (2) complex cells.

A:

- (1) Simple cells output the **SUM** of their inputs ("and"-like; increase *selectivity*).
- (2) Complex cells output the **MAXIMUM** of their inputs ("or"-like; increase *invariance*).

Example 7: Bayes for vision (LGT Q7)

Q: Write down Bayes' theorem and explain each term in relation to a computer vision system.

A:

$$p(H \mid E) \;=\; \frac{p(E \mid H)\, p(H)}{p(E)}.$$

- $p(H \mid E)$ – **posterior**: probability that hypothesis H (e.g. "object j is present") is true given the image evidence E. This is what the vision system needs to evaluate (we want the most likely H).
- $p(E \mid H)$ – **likelihood**: probability that, if H were true, the image would contain the observed evidence E. Comes from our knowledge of the image-formation process (geometry, illumination, surface properties).

- $p(H)$ – **prior**: probability of H before seeing the image; if H is intrinsically improbable, stronger evidence is required.
- $p(E)$ – **evidence**: probability of seeing E regardless of whether H is true. The same for all hypotheses about a fixed image, so it cancels when comparing them.

Example 8: Posterior with priors – robot sorter (LGT Q8)

Q: A production line yields two objects, objA and objB, sorted by a robot. From most viewpoints they are distinguishable, but at orientations oriA and oriB respectively their images are *indistinguishable*. The line produces 3 objA per 1 objB. The probability of objA at oriA is 0.1; the probability of objB at oriB is 0.2. Into which bin should the robot sort an indistinguishable image to minimise errors?

A: Class priors: $p(\text{objA}) = 0.75$, $p(\text{objB}) = 0.25$ (3:1 ratio). Likelihoods of observing the indistinguishable image:

$$p(I \mid \text{objA}) = 0.1, \qquad p(I \mid \text{objB}) = 0.2.$$

Posteriors (with $k = 1/p(I)$ a common factor):

$$p(\text{objA} \mid I) = k \cdot 0.1 \cdot 0.75 = 0.075\,k, \quad p(\text{objB} \mid I) = k \cdot 0.2 \cdot 0.25 = 0.050\,k.$$

$0.075 > 0.050 \Rightarrow$ **sort into the objA bin**.

(Optional) Normalising: $p(I) = 0.075 + 0.050 = 0.125$, so $p(\text{objA} \mid I) = 0.6$, $p(\text{objB} \mid I) = 0.4$.

Take-away: the higher likelihood (objB) is overruled by the much higher prior (objA).

Example 9: Posterior ratio – zebra detector

Q: Use the posterior ratio to decide whether each image contains a zebra, given $p(\text{zebra}) = 0.01$ and $p(\neg\text{zebra}) = 0.99$.

Image 1: $p(I \mid \text{zebra}) = 0.07$, $p(I \mid \neg\text{zebra}) = 0.0005$.

$$\frac{p(\text{zebra} \mid I)}{p(\neg\text{zebra} \mid I)} = \frac{0.07}{0.0005} \cdot \frac{0.01}{0.99} \approx 140 \cdot 0.0101 \approx 1.41 \;>\; 1 \;\Rightarrow\; \textbf{zebra}.$$

Image 2 (motorcycle): $p(I \mid \text{zebra}) = 0.003$, $p(I \mid \neg\text{zebra}) = 0.85$.

$$\frac{0.003}{0.85} \cdot \frac{0.01}{0.99} \approx 0.00353 \cdot 0.0101 \approx 3.6 \times 10^{-5} \;<\; 1 \;\Rightarrow\; \textbf{not zebra}.$$

Take-away: the prior ratio (low for zebras) means strong likelihood evidence is needed to overturn the default "not a zebra".

Example 10: HMAX vs CNN

Q: (a) What two operations are alternated in HMAX, and what does each achieve? (b) How does a CNN realise the same recipe? (c) State a confusing naming clash.

A:

- (a) HMAX alternates **S-cells** (SUM, "and"-like, increase *selectivity*) and **C-cells** (MAX, "or"-like, increase *invariance*).
- (b) A CNN alternates **convolution** (with learned filters, equivalent to S-cell SUM/conjunction) and **sub-sampling / max-pooling** (equivalent to C-cell MAX, reducing location specificity).
- (c) In CNN literature, the convolution layer is called the "C layer" but matches HMAX's S-cells; the sub-sampling layer is called the "S layer" but matches HMAX's C-cells. Names flipped, operations equivalent.

Example 11: Configural vs featural – face inversion

Q: Why is it harder to spot grotesque alterations in an inverted face than in an upright face?

A: Upright faces are processed **configurally** (holistically, taking spatial relations between features into account). Inversion disrupts configural processing and forces **featural** processing, in which each feature is evaluated independently. Local features may look fine on their own even when the configuration is grossly altered (e.g. Thatcher illusion).

Example 12: Why purely featural or purely configural fails

Q: Explain why simple (local) and complex (global) feature representations each fail in characteristic ways, and how cortex resolves this.

A:

- **Local / featural** features generate many *false positives*: they fail to distinguish objects with similar features in different arrangements, and fail in clutter.
- **Global / configural** features generate many *false negatives*: they fail under occlusion, viewpoint changes, and within-class variation.
- **Cortex's solution:** use a *hierarchy* of features with a range of complexities, modelled by alternating layers that increase selectivity (SUM) and invariance (MAX). This spans both regimes simultaneously.

Example 13: Coding – nearest mean and k-NN classifiers

Q: Sketch `nearest_mean_classifier` and `k_nearest_neighbors_classifier` (Euclidean distance, 2D feature vectors).

A (key lines):

```
nearest_mean:
for cls in unique_classes:
    cls_features = train_features[train_labels == cls]
    class_mean = np.mean(cls_features, axis=0)
    dist = euclidean_distance(new_stimulus, class_mean)
    if dist < min_dist:  best_class = cls; min_dist = dist

k-NN:
distances = [(euclidean(new_stimulus, x), y) for x,y in zip(train_features, train_labels)]
distances.sort(key=lambda t:  t[0])
k_nearest = [y for _,y in distances[:k]]
return Counter(k_nearest).most_common(1)[0][0]
```

Take-away: NMC stores one mean per class (linear boundary); k-NN stores all training points and votes (non-linear, Voronoi-like).

Example 14: Coding – simulating S- and C-cells

Q: Sketch `simulate_s_cell` (Gabor convolution) and `simulate_c_cell` (max pooling).

A (key lines):

```
S-cell (selectivity, ``and''-like):
gabor = cv2.getGaborKernel((ksize,ksize), 4.0, theta, 10.0, 0.5, 0, ktype=cv2.CV_32F)
s_response = cv2.filter2D(image, cv2.CV_32F, gabor)

C-cell (invariance, ``or''-like, MAX):
c_response = scipy.ndimage.maximum_filter(s_response, size=pool_size)
```

Take-away: Gabor convolution gives orientation-selective response (S-cell); spatial max pooling tolerates small position shifts (C-cell). Same structure as a CNN convolution + max-pooling pair.

Example 15: Coding – posterior ratio

Q: Sketch `calculate_posterior_ratio`.

A (key lines):

```
likelihood_ratio = p_img_given_obj / p_img_given_not_obj
prior_ratio = p_obj / p_not_obj
posterior_ratio = likelihood_ratio * prior_ratio
is_recognised = posterior_ratio > 1.0
return posterior_ratio, is_recognised
```

Take-away: for a binary decision, you never need $p(I)$; multiply likelihood ratio by prior ratio and compare to 1.

10.4 Summary

Key Takeaways

1. **Object-based vs image-based.** Object-based stores one 3D model per object (RBC, geons, structural descriptions, viewpoint-invariant). Image-based stores multiple 2D views (template / multiple-views, viewer-centred). Each has characteristic failure modes.
2. **Configural vs featural.** Holistic / global processing (upright faces) vs independent-feature processing (inverted faces). Inversion effect demonstrates the distinction.
3. **Categorisation.** Rules (decision trees), Prototypes (NMC, linear boundary), Exemplars (1-NN, k-NN; non-linear, Voronoi). NMC stores one mean per class; (k-)NN stores all training points and votes.
4. **Ventral pathway hierarchy.** V1→V2→V4→IT: RFs grow larger, less location-specific, more complex. Centre-surround → simple → complex → $\cdots$ → face cells.
5. **HMAX.** Alternating S-cells (SUM, "and", selectivity) and C-cells (MAX, "or", invariance). V1→IT generic + hard-wired; IT→PFC task-specific + supervised.
6. **CNNs.** Same recipe: convolution (= S/SUM) + max pooling (= C/MAX). The "C/S" naming is flipped relative to HMAX.
7. **Recurrent cortex.** Lateral connections within an area + feedback connections from higher to lower areas. Combine bottom-up (stimulus-driven, discriminative) with top-down (knowledge-driven, generative).
8. **Bayes' theorem.** $p(\text{obj} \mid I) \propto p(I \mid \text{obj})\, p(\text{obj})$; posterior $\propto$ likelihood $\times$ prior. Vision is an ill-posed inverse problem; priors disambiguate.
9. **Posterior ratio.** For binary decisions: posterior ratio = likelihood ratio $\times$ prior ratio; choose "object" iff > 1 (evidence cancels).

Final comparison: theories of object recognition.

Theory	What is stored	Strengths / weaknesses
Object-based / RBC	One 3D model per object (geons + relations)	Viewpoint-invariant; struggles with non-decomposable objects and fine within-class differences
Image-based template	One 2D view per object	Too rigid; no flexibility
Image-based multi-view	Multiple 2D views + interpolation	Handles novel viewpoints; viewer-centred
Configural (face area)	Whole-object spatial config	Holistic; broken by inversion
Featural	Independent local features	Robust to occlusion; many false positives

Final comparison: psychology ↔ ML mapping.

Theory	ML method	Decision boundary	Storage
Rules	Decision tree	Axis-aligned	Tree of thresholds
Prototypes	Nearest mean classifier	Linear	1 mean / class
Exemplars	1-NN	Voronoi (piecewise)	All training points
Exemplars	k-NN	Non-linear, smoother	All training points + k

Final comparison: cortical models.

Model	Connectivity	Computation
HMAX (feed-forward)	V1→IT→PFC, no feedback	Alternate SUM (S, selectivity) and MAX (C, invariance); top classifier supervised
CNN (feed-forward)	Layers, no feedback	Alternate convolution (= S/SUM) and max-pooling (= C/MAX); end-to-end supervised
Recurrent cortex	Lateral + feedback	Combine bottom-up evidence with top-down priors; Bayesian inference

Final comparison: bottom-up vs top-down.

Direction	Driven by	Models	Bayes term
Bottom-up	Stimulus	Discriminative	Models $p(\text{obj} \mid I)$ directly
Top-down	Context, priors	Generative	Models $p(I \mid \text{obj})$ and $p(\text{obj})$